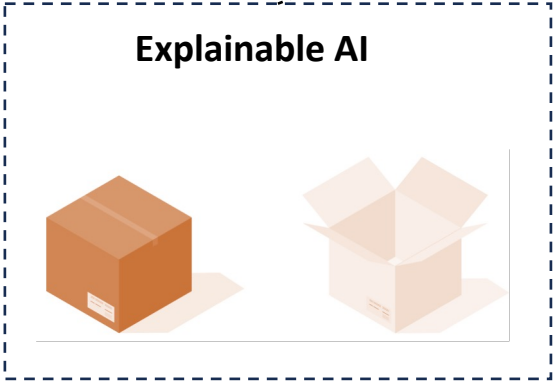
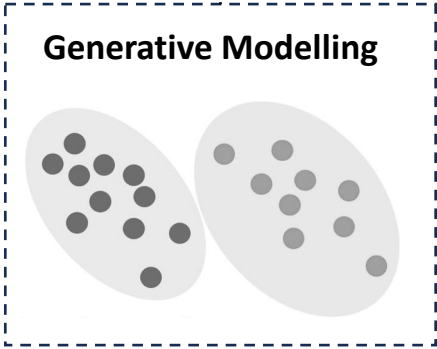


Lecture map

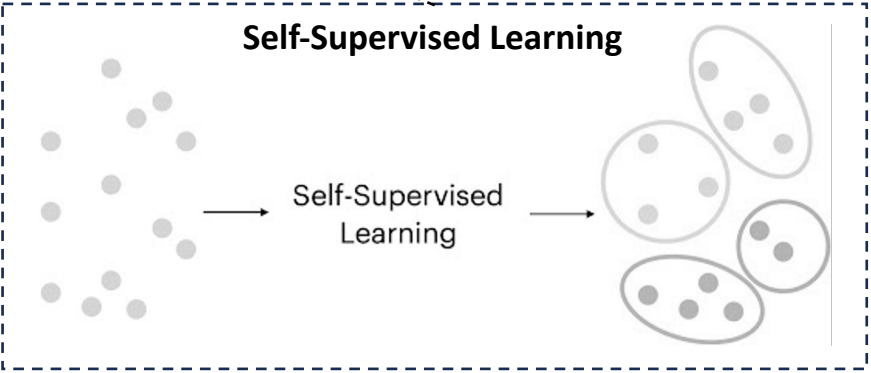
Topics



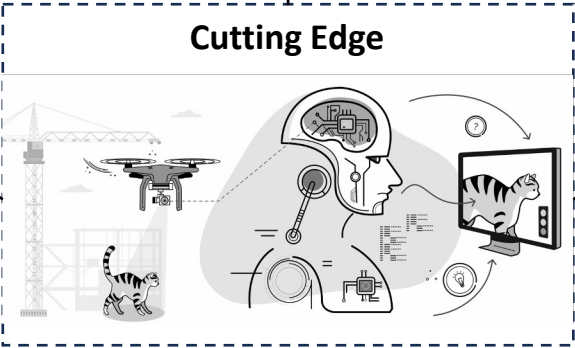
Estimated 2 lectures



Estimated 3 lectures



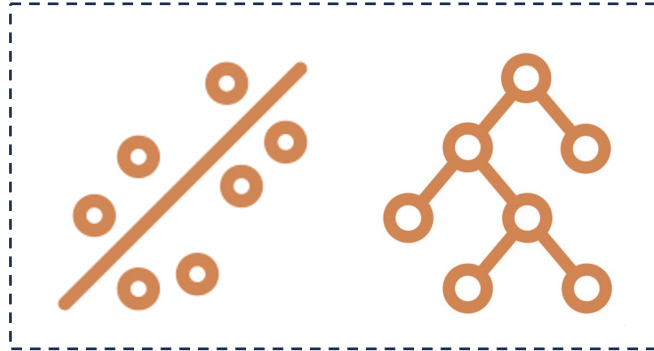
Estimated 1 lecture



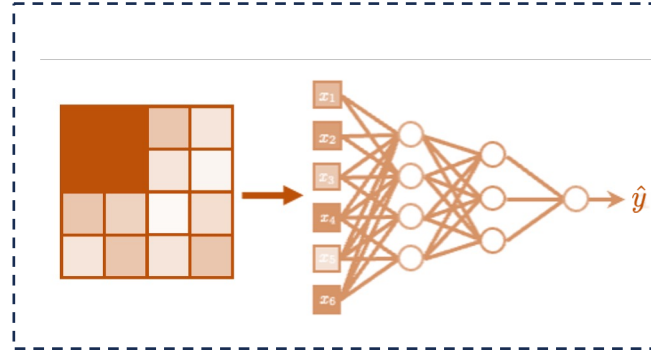
Estimated 1 lecture

Recap

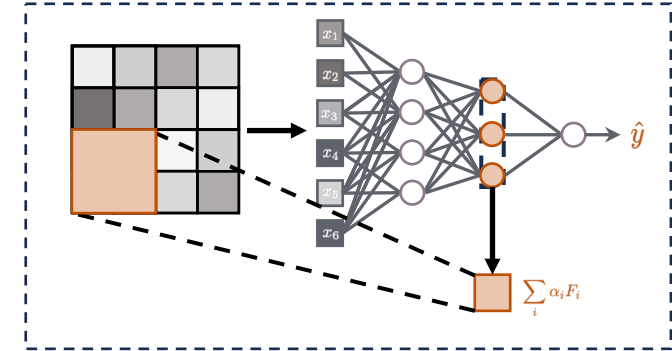
Glass box models



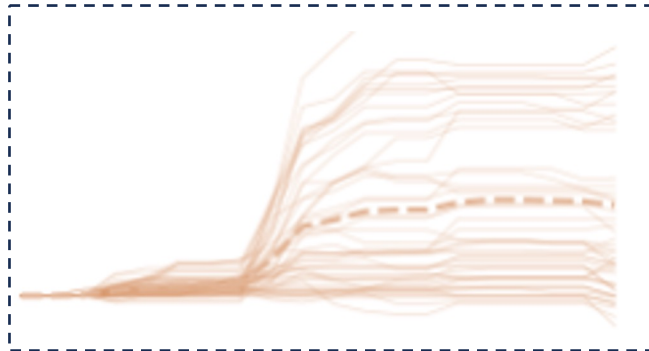
Occlusion



Class Activation Mapping (CAM)



Individual Conditional Expectation (ICPs)

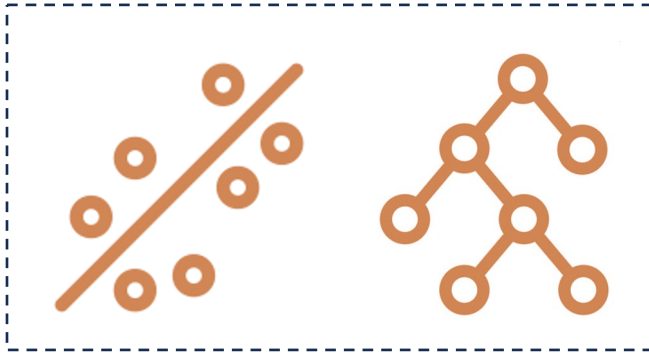


Local Interpretable Model
Agnostic Explanations (LIME)



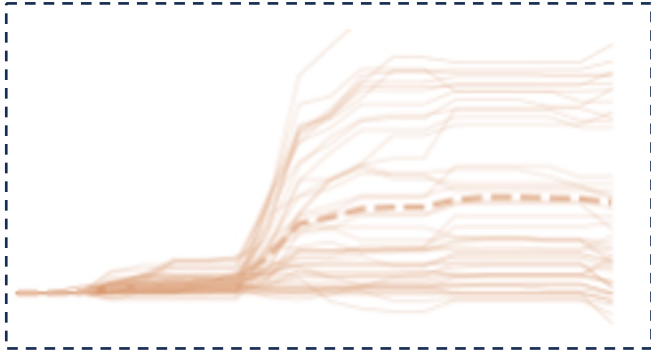
Recap

Glass box models



- How do we interpret the importance of features in linear models?
- How do we grow decision trees?
- In which two XAI deep learning methods did we encounter the use of linear models?

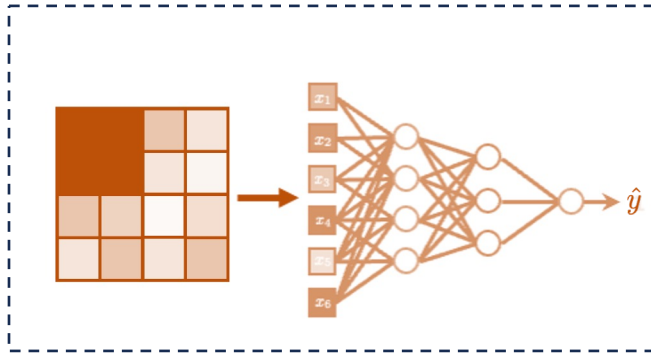
Individual Conditional Expectation (ICPs)



- Explain how to calculate an ICP curve
- What are some problems to be aware of when applying ICE and PDP plots?

Recap

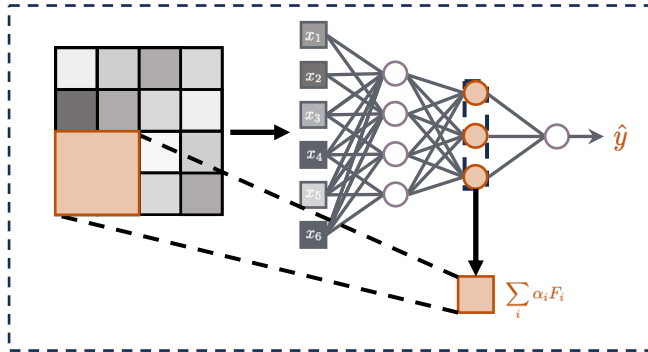
Occlusion



- How does the basic vanilla occlusion method work?
- What are the four downsides of this method?
- Is it model agnostic?

Recap

Class Activation Mapping (CAM)



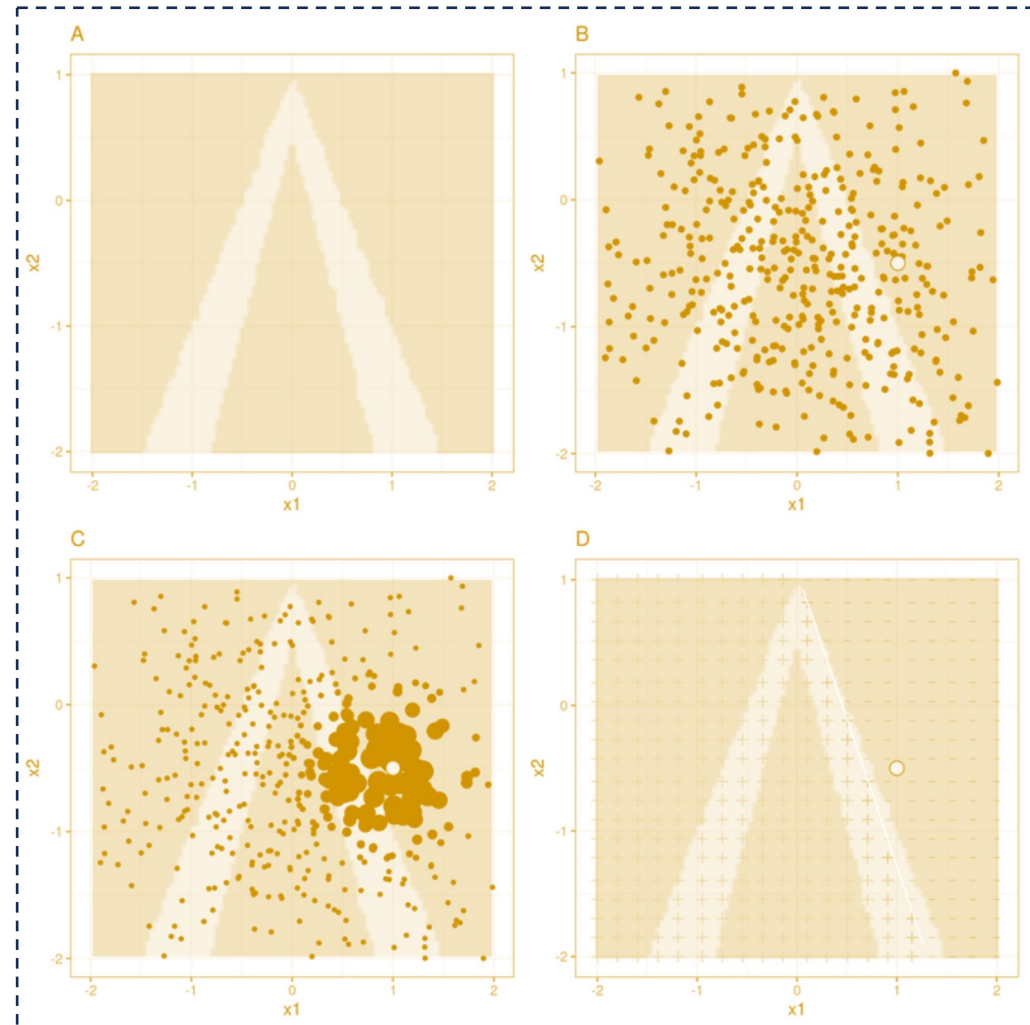
- What is the purpose of the GAP in CAM?
- What is the tension between layer depth and explanation quality?
- Is CAM model agnostic?
- How does Grad-CAM calculate the weights of feature maps?

Recap

LIME



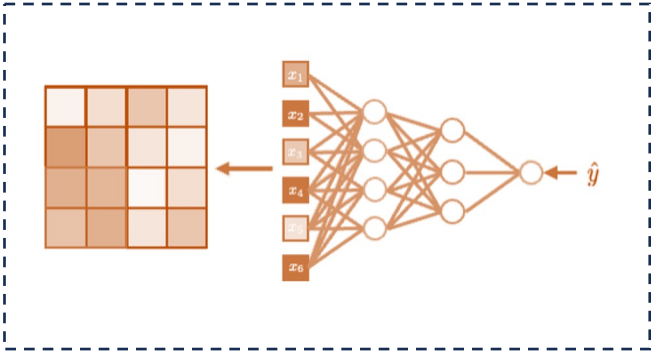
- What can make LIME fragile?
- Is LIME model agnostic?
- Is LIME data agnostic?



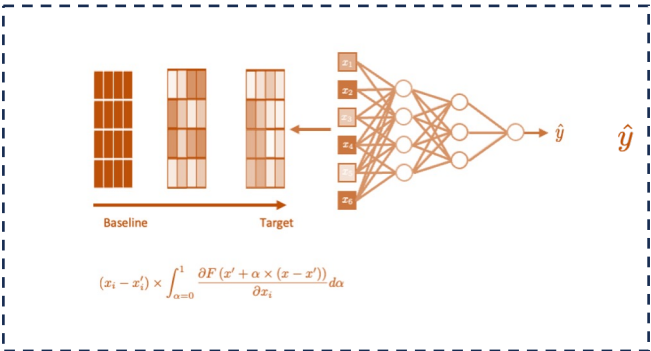
Explain the four panels

XAI Lecture 2 Topics

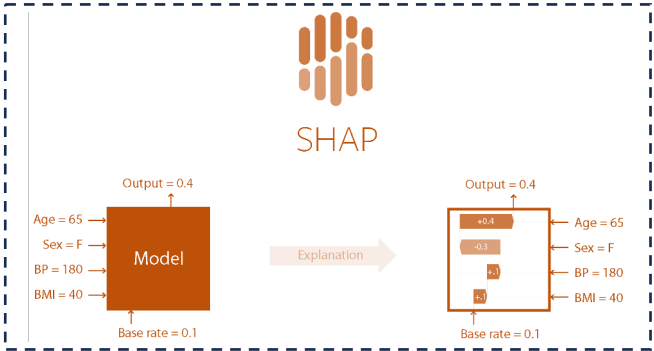
Vanila Gradients Gradient x Input



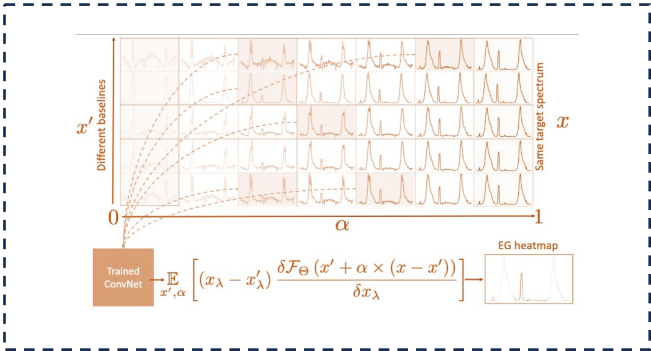
Integrated Gradients



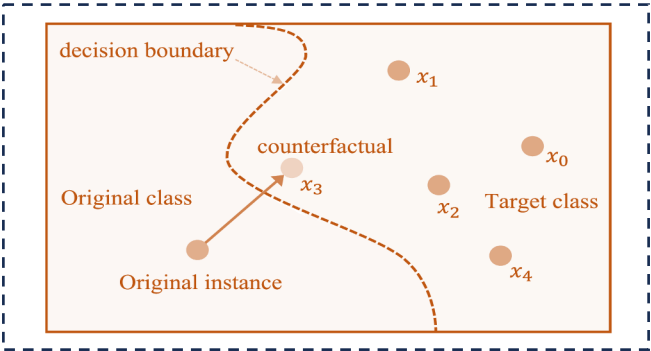
Shapley values



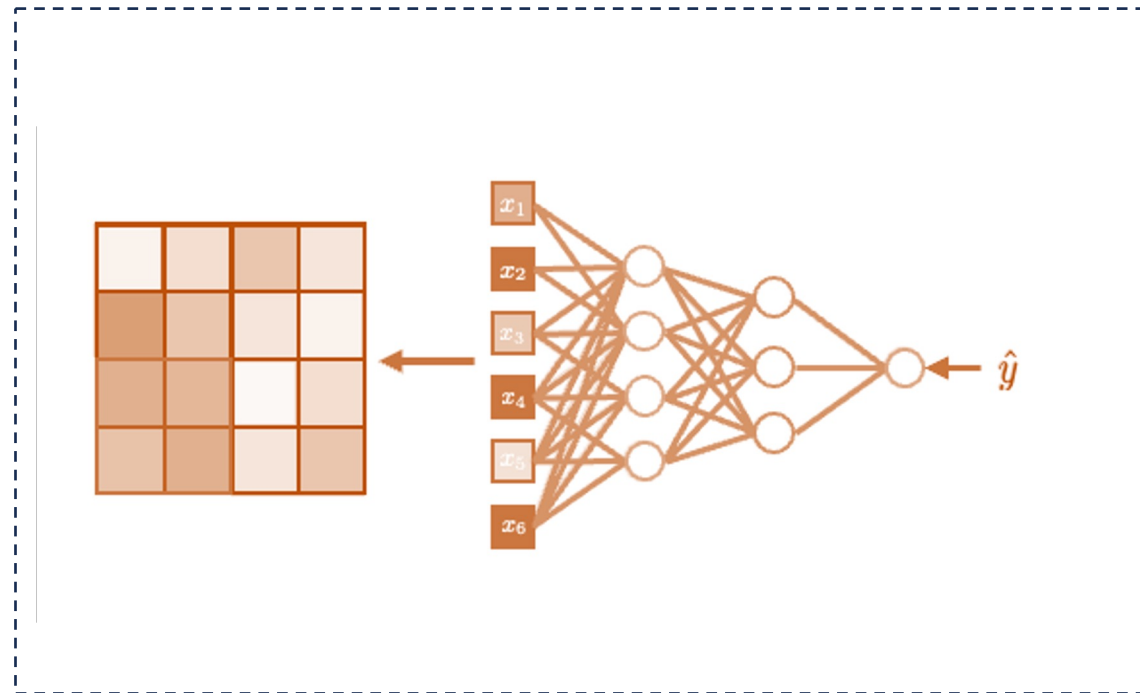
Expected Gradients



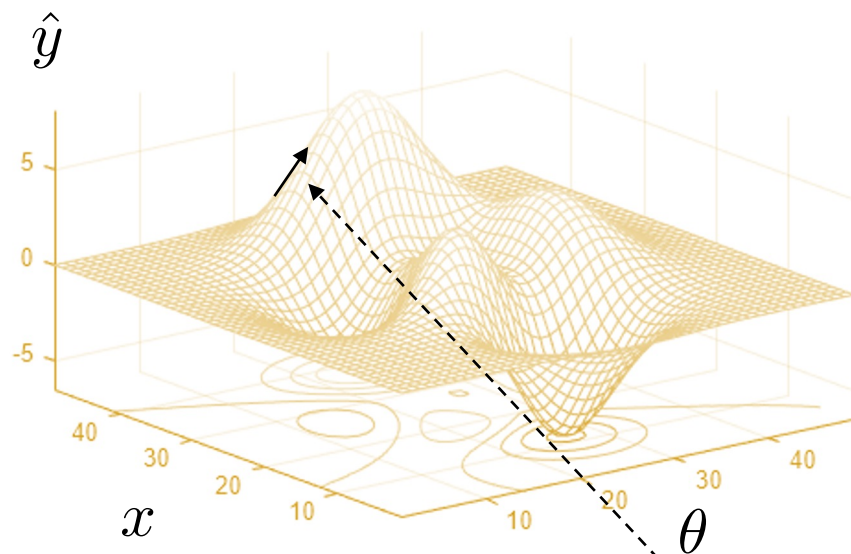
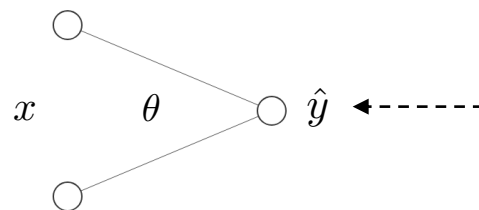
Counterfactuals



Gradient methods (Gradient X Input)



Requires model to be differentiable



When training a model, we call
backwards on $|y - \hat{y}|$ and optimize.

```
import torch
import torch.nn as nn

# Define simple data
x = torch.tensor([[1.0, 2.0]], requires_grad=True) # require gradient for input
y_true = torch.tensor([[1.0]]) # target

# Define simplest perceptron model
model = nn.Sequential(
    nn.Linear(2, 1), # y = W*x + b
    nn.Sigmoid()     # activation
)
```

```
model
✓ 0.0s

Sequential(
  (0): Linear(in_features=2, out_features=1, bias=True)
  (1): Sigmoid()
)
```

```
# Forward pass
y_pred = model(x)
print("Prediction:", y_pred.item())

# Backward pass
# Compute gradient of the prediction w.r.t. input and parameters
y_pred.backward() # d(output)/d(input,weights,bias)

# Step 5: Inspect gradients
print("\nGradients w.r.t. input x:")
print(x.grad)

for name, param in model.named_parameters():
    print(f"\nGradients for {name}:")
    print(param.grad)
```

```
✓ 0.0s

Prediction: 0.8107743859291077
```

```
Gradients w.r.t. input x:
tensor([[0.0577, 0.0954]])
```

```
Gradients for 0.weight:
tensor([[0.1534, 0.3068]])
```

```
Gradients for 0.bias:
tensor([0.1534])
```

NNs by construction
are differentiable

Setting require gradients
and calling backward()
in PyTorch allows us to
access the model's grads

This is just a vector that
points in the direction
of greatest increase of
the function

The gradient signal is a
strong signal for XAI,
however, if not careful,
it is also a noisy signal

Gradient methods (Gradient X Input)

- Simple fast method
- Saliency method
- Gradient method
- Sensitivity of prediction on the input
- Multiply the gradient by its feature value to scale the sensitivity and provide both importance and direction

```
# Gradient x Input from scratch
def gradient_x_input(model, input_tensor, target_class):
    # Enable gradients for input tensor
    input_tensor.requires_grad = True

    # Forward pass
    model.zero_grad()
    output = model(input_tensor)

    # Create one-hot vector for target class
    one_hot = torch.zeros_like(output)
    one_hot[0][target_class] = 1

    # Backward pass
    output.backward(gradient=one_hot)

    # Get gradients from input tensor
    gradients = input_tensor.grad.data.cpu().numpy()[0]

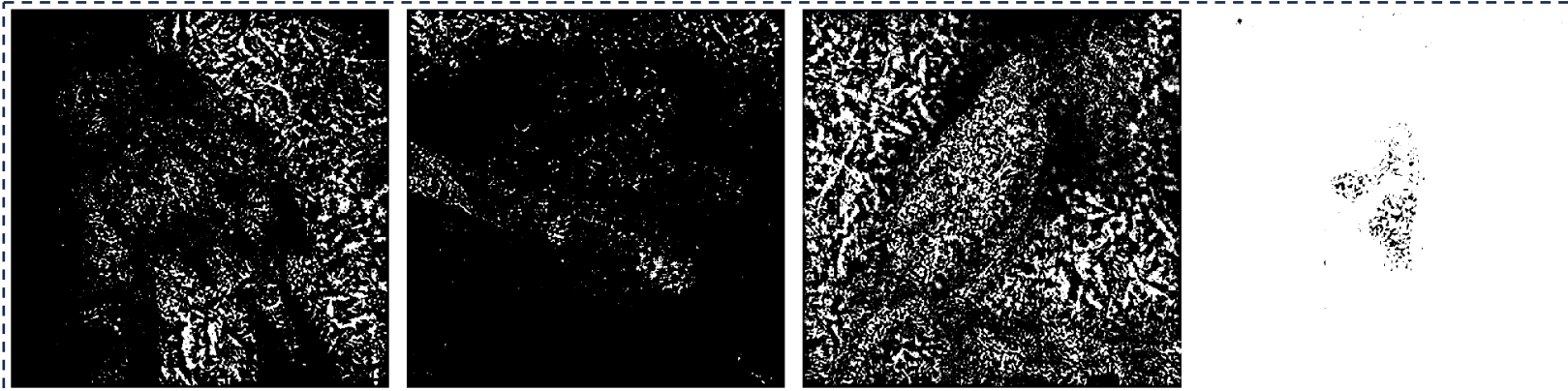
    # multiply the gradients by the input tensor
    gradient_x_input = gradients * input_tensor.detach().cpu().numpy()[0]

    return gradient_x_input
```

Get gradients of model

Gradients of input

LLM, CNN, etc.



$$M_i = x_i \cdot \frac{\partial f(x)}{\partial x_i}$$

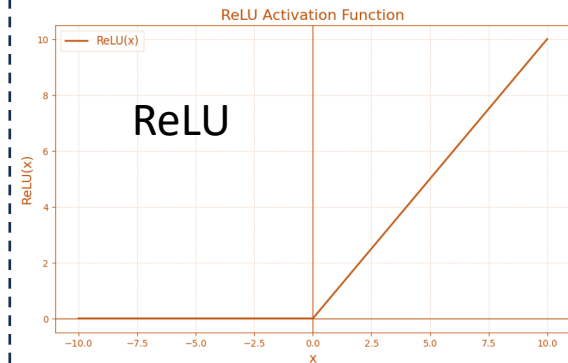
Word
Feature
Pixel value

For vanilla gradients method, don't multiply by the feature value.

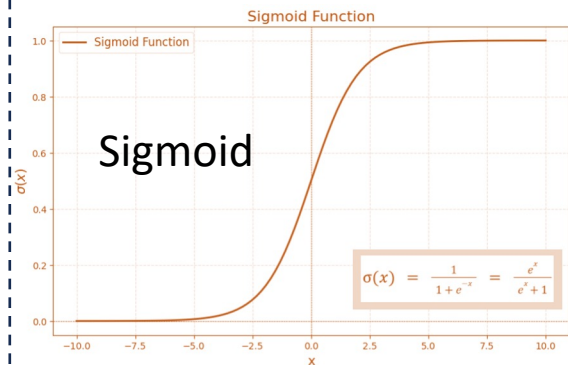
Gradient methods (**Gradient X Input**)

- Vanilla gradients method generates **noisy** saliency maps, often highlighting pixels that are not semantically important for a model's prediction. Conversely, the method can fail to highlight pixels that are crucial.

Saturation problem

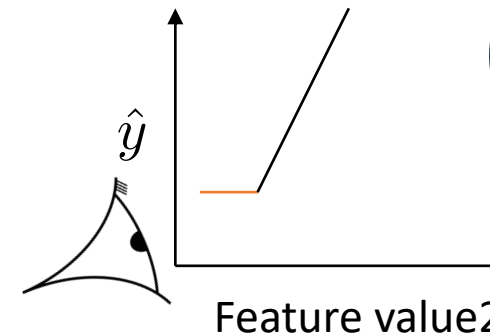
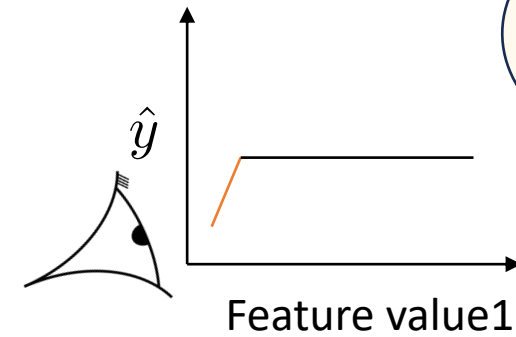


Gradient doesn't capture magnitude of a neurons activation, i.e., in put of large or small x has a gradient of 1



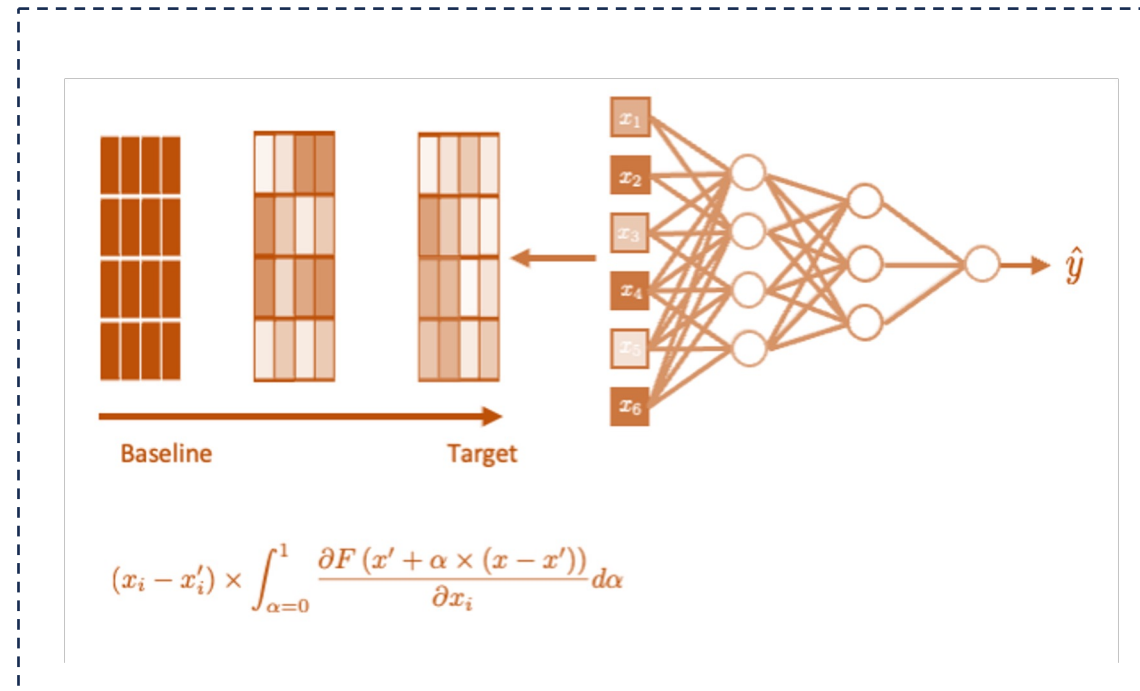
If the feature is extremely strong, the output ~ 1 and the gradient = 0. Even though the model relies on the feature, it will be ranked as unimportant.

Local linear assumption



A raw input-gradient is a local, first-order, high-frequency sensitivity signal taken on a piecewise-linear, non-smooth function

Gradient methods (Integrated Gradients)



Visualizing the Impact of Feature Attribution Baselines

Excellent resource for path integral methods

To gain an intuition please engage with the following interactive website

<https://distill.pub/2020/attribution-baselines/>

Gradient methods (Integrated Gradients)

Intuition why we require a different gradient method:

Once a model is confident of its prediction, changing a few pixel values is likely to not change the model's prediction. This is due to how contextually dependent (correlated) image data is. When a model learns how a value of a specific pixel affects the model's prediction, the gradient for that pixel will become smaller and smaller until it is eventually zero. This is called saturation.

We therefore require a trick that allows us to **access the gradient information of each pixel when the model is more sensitive to its change.**

Gradient methods (Integrated Gradients)

Attribution for single pixel (i) given a function and a baseline

Linearly interpolate between baseline and target

$$\phi_i^{IG}(f, x, x') = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\delta f(x' + \alpha(x - x'))}{\delta x_i} d\alpha$$

Integrate along a path in image space

Diagram illustrating the components of the Integrated Gradients formula:

- f points to **NN** (Neural Network).
- x points to **Target**.
- x' points to **Baseline**.
- The integral term points to **Integrate along a path in image space**.
- The entire formula points to the title: **Attribution for single pixel (i) given a function and a bassline**.

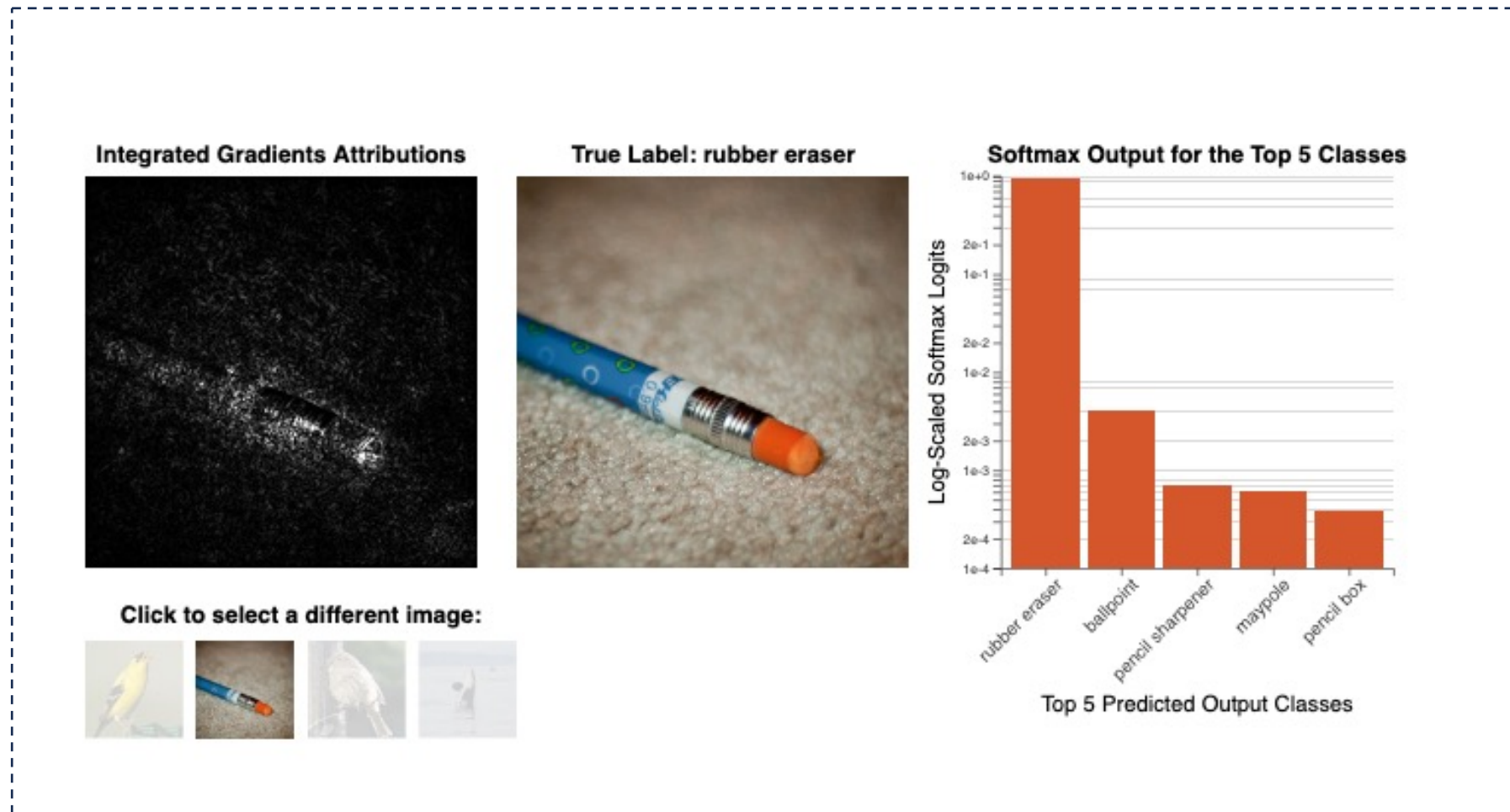
Gradient methods (Integrated Gradients)

Simple extension of Gradient X Input allowing

$$M_i = x_i \cdot \frac{\partial f(x)}{\partial x_i}$$

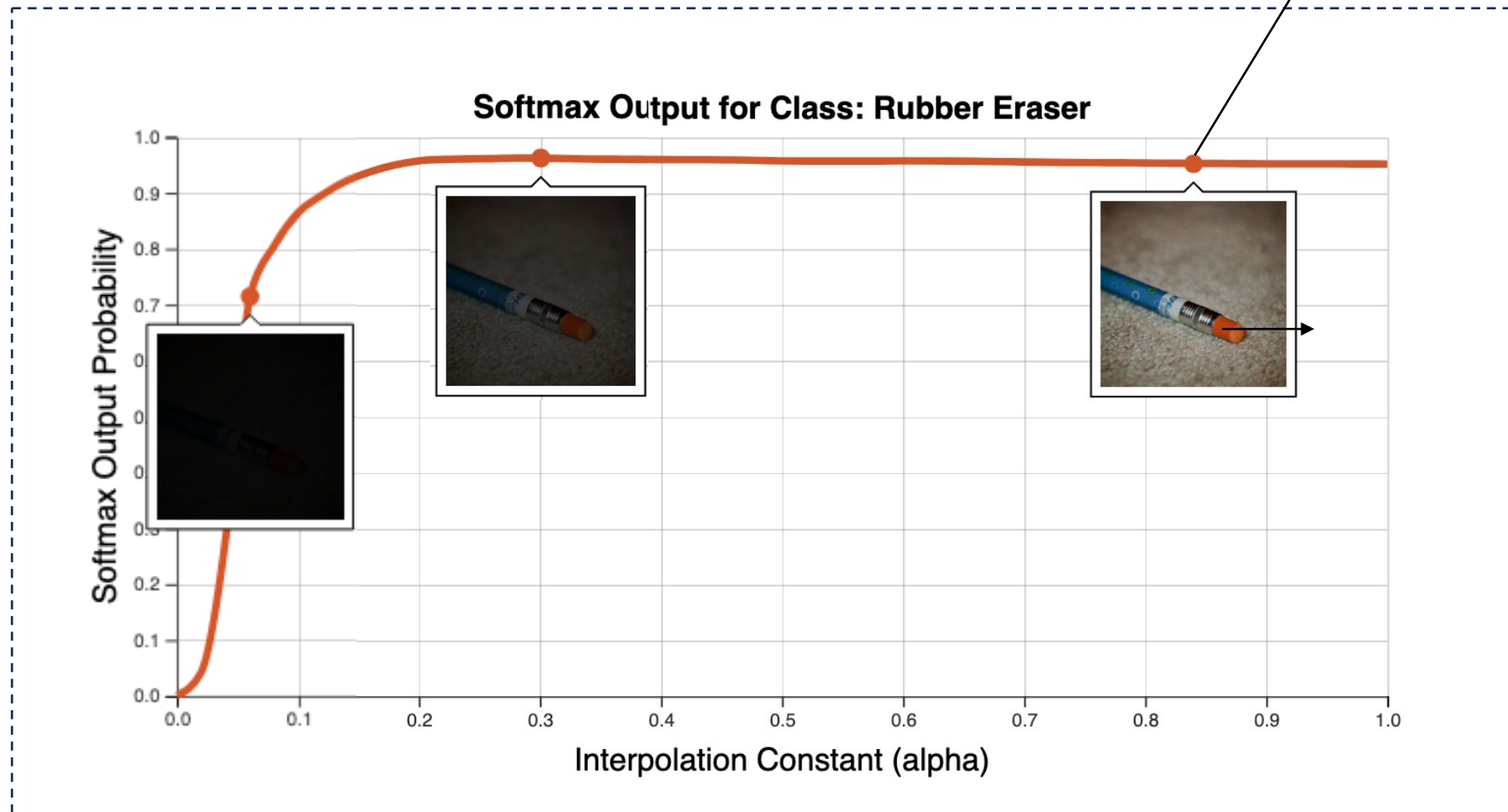
$$\phi_i^{IG}(f, x, x') = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\delta f(x' + \alpha(x - x'))}{\delta x_i} d\alpha$$

Gradient methods (Integrated Gradients)



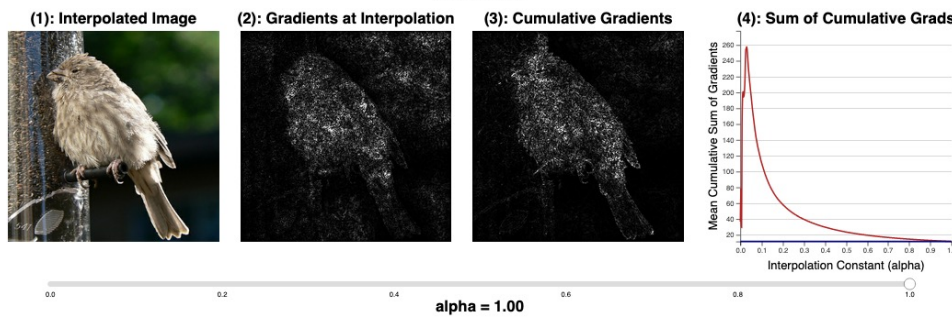
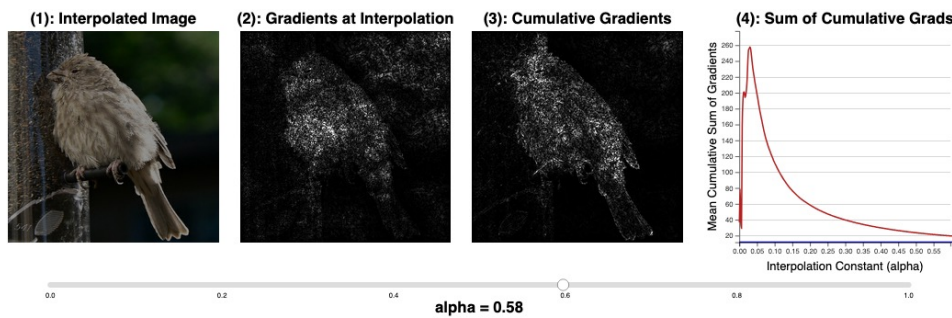
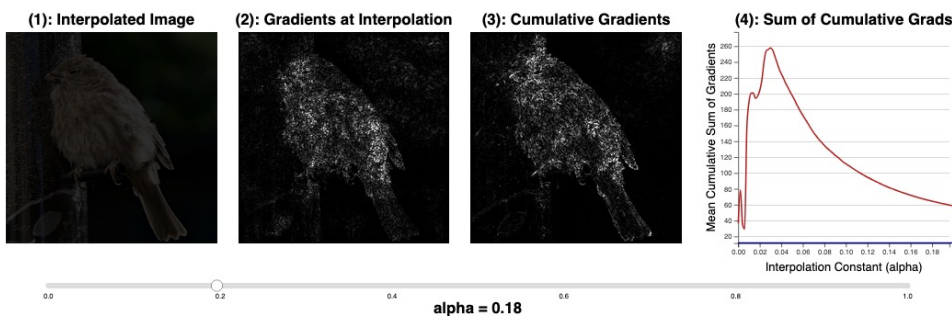
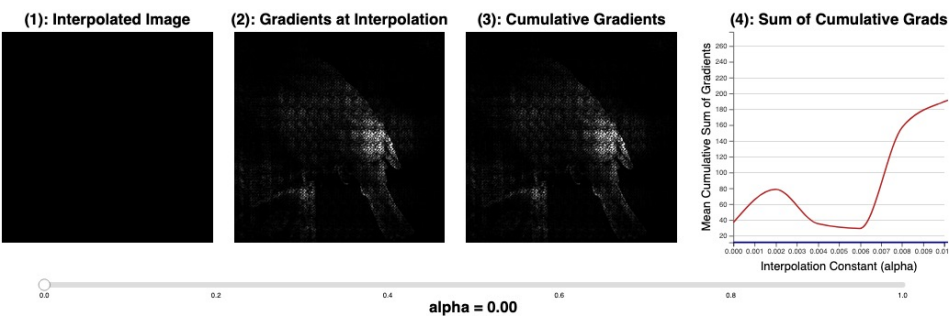
Gradient methods (Integrated Gradients)

This is where Gradient X Input operates



The networks prediction saturates early during the interpolation

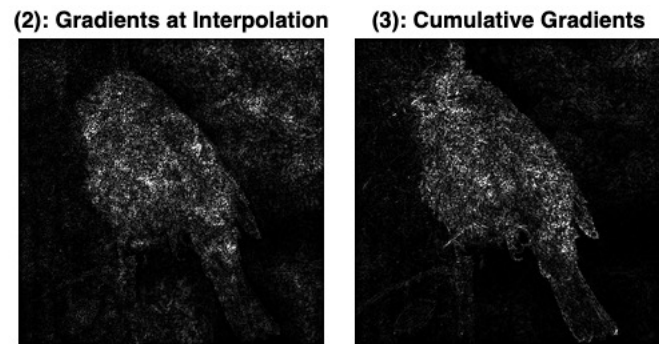
Gradient methods (Integrated Gradients)



$$\int_{\alpha'=0}^{\alpha} \underbrace{(x_i - x'_i) \times \frac{\delta f(\overbrace{x' + \alpha'(x - x')}^{(1): \text{Interpolated Image}})}{\delta x_i}}_{(2): \text{Gradients at Interpolation}} d\alpha' = \overbrace{\phi_i^{IG}(f, x, x'; \alpha)}^{(3): \text{Cumulative Gradients up to } \alpha}$$

→ Largest contribution to gradients

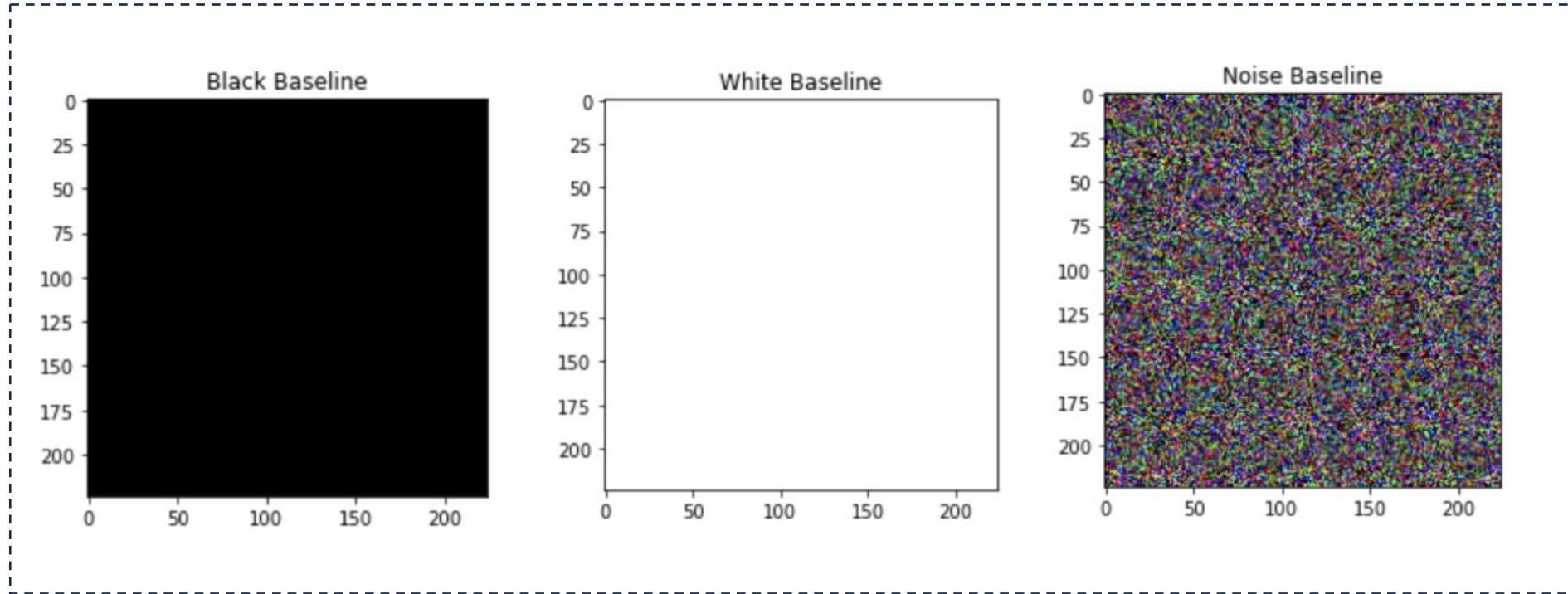
→ Gradients are saturated



Left image: Equivalent to Gradient X Input shifted by the baseline

Gradient methods (Integrated Gradients)

$$\phi_i^{IG}(f, x, \mathbf{x}') = (x_i - \mathbf{x}'_i) \int_{\alpha=0}^1 \frac{\delta f(\mathbf{x}' + \alpha(x - \mathbf{x}'))}{\delta x_i} d\alpha$$

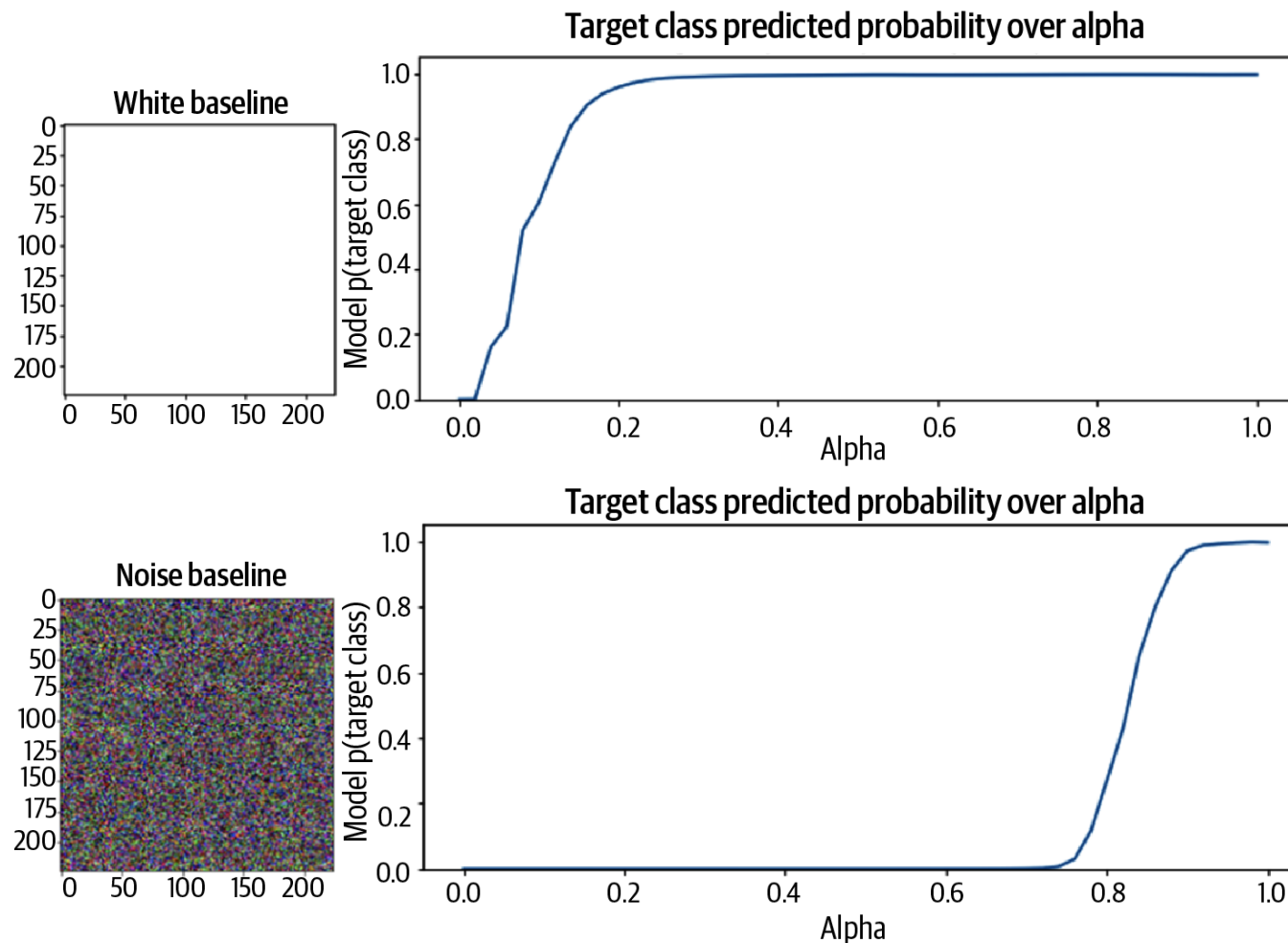


How do we select the correct baseline

Gradient methods (Integrated Gradients)

$$\phi_i^{IG}(f, x, \mathbf{x}') = (x_i - \mathbf{x}'_i) \int_{\alpha=0}^1 \frac{\delta f(\mathbf{x}' + \alpha(x - \mathbf{x}'))}{\delta x_i} d\alpha$$

Which gradient
will saturate first?



The images topology
(contextual information)
is mediatly available
when increasing alpha
from the white bassline,
all that remains is the
intensity information.

The noise bassline
obscures both the
pictures context
and intensity information

Gradient methods (Integrated Gradients)

$$\phi_i^{IG}(f, x, \boldsymbol{x}') = (x_i - \boldsymbol{x}'_i) \int_{\alpha=0}^1 \frac{\delta f(\boldsymbol{x}' + \alpha(x - \boldsymbol{x}'))}{\delta x_i} d\alpha$$

max dist.

blur

uniform

gaussian



Click to select a different image:

Baseline Image

Original Image

Pixel Attributions

max dist.

blur

uniform

gaussian



Click to select a different image:

Baseline Image

Original Image

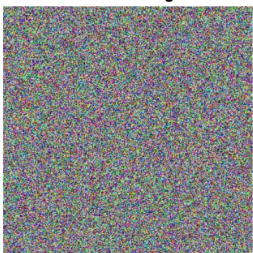
Pixel Attributions

max dist.

blur

uniform

gaussian



Click to select a different image:

Baseline Image

Original Image

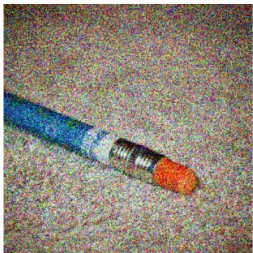
Pixel Attributions

max dist.

blur

uniform

gaussian

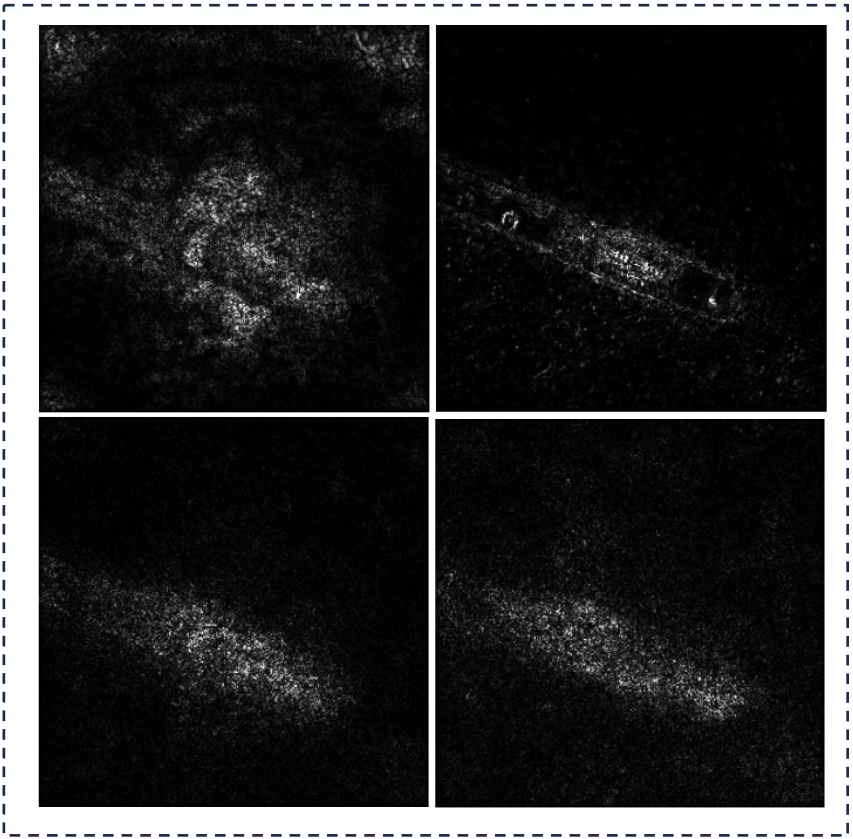


Click to select a different image:

Baseline Image

Original Image

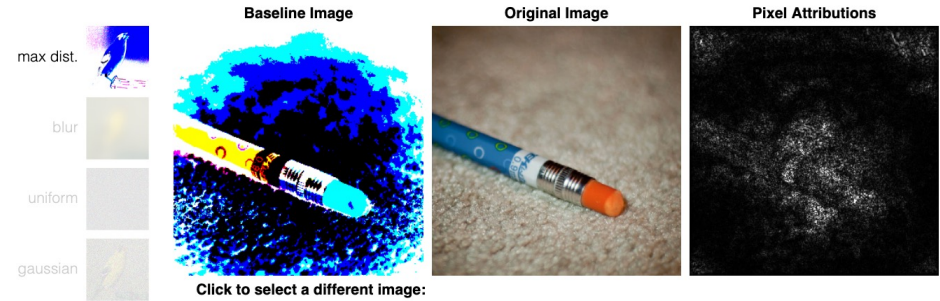
Pixel Attributions



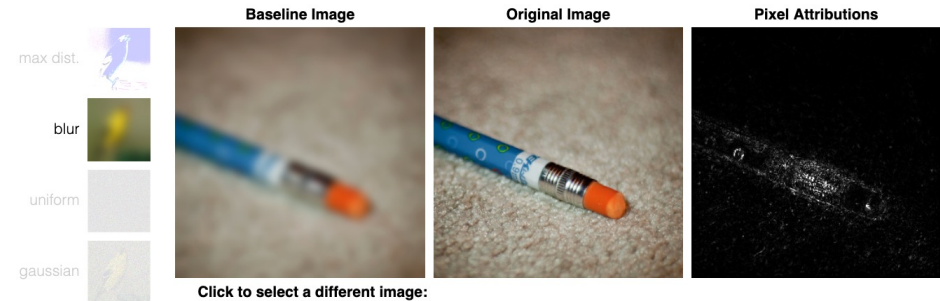
The attributions are strongly Influences by the baseline

Gradient methods (Integrated Gradients)

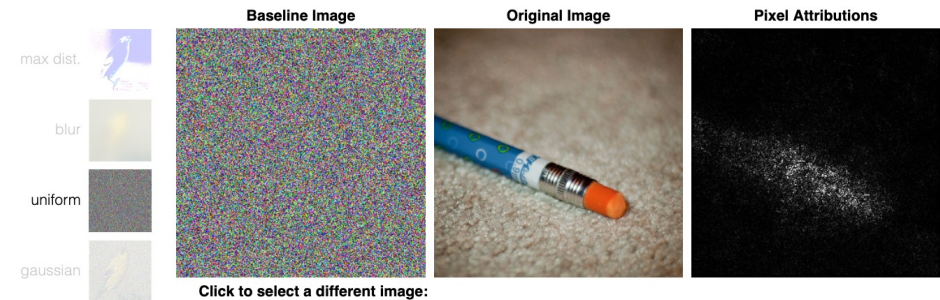
$$\phi_i^{IG}(f, x, x') = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\delta f(x' + \alpha(x - x'))}{\delta x_i} d\alpha$$



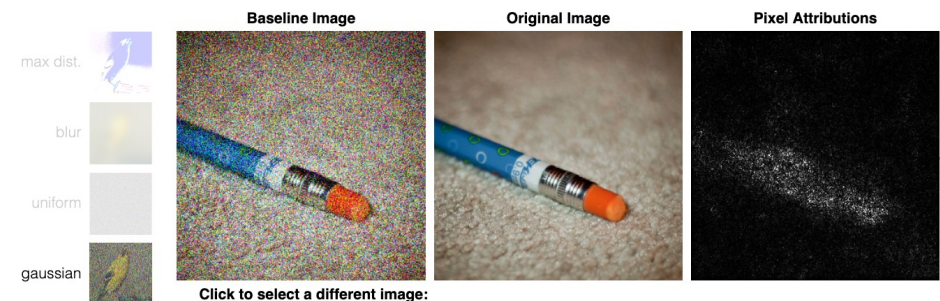
Max distance: Fixes the issue with distance from baseline pre-factor
Doesn't represent missingness, has with as much information as image



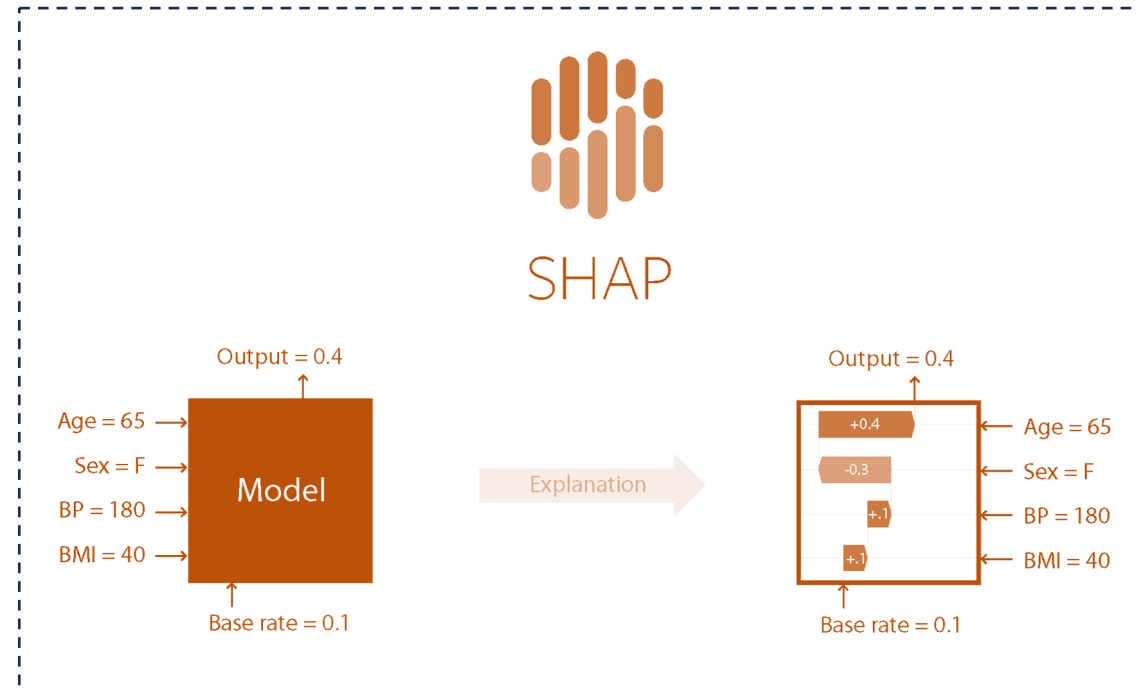
Blur: Intuitive since the image is missing information, however biased to highlight high frequency data, i.e., pixels close together and very different will result in an average that intern generates a larger perfector an pathlength to integrate along.



Uniform: Sample random image within the allowed pixel range
Assumes the pixels are uncorrelated which is not true

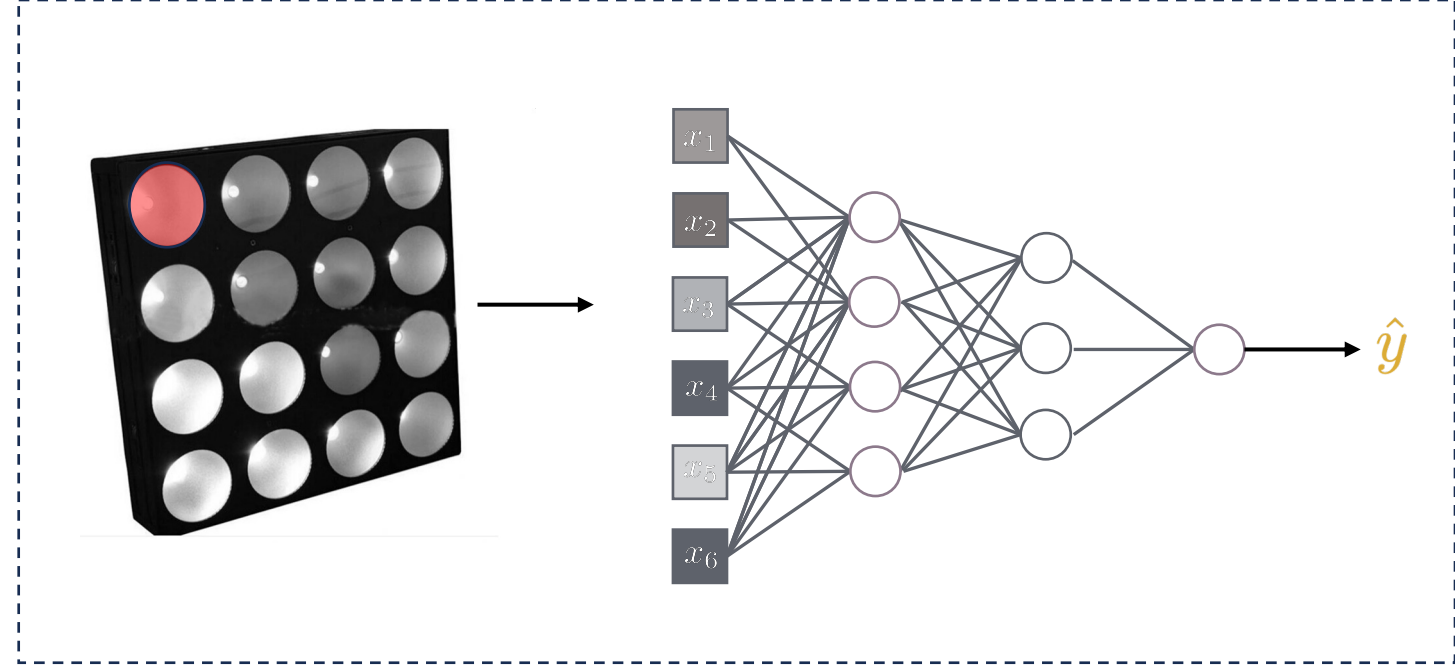


Gaussian: Same as above but with more control



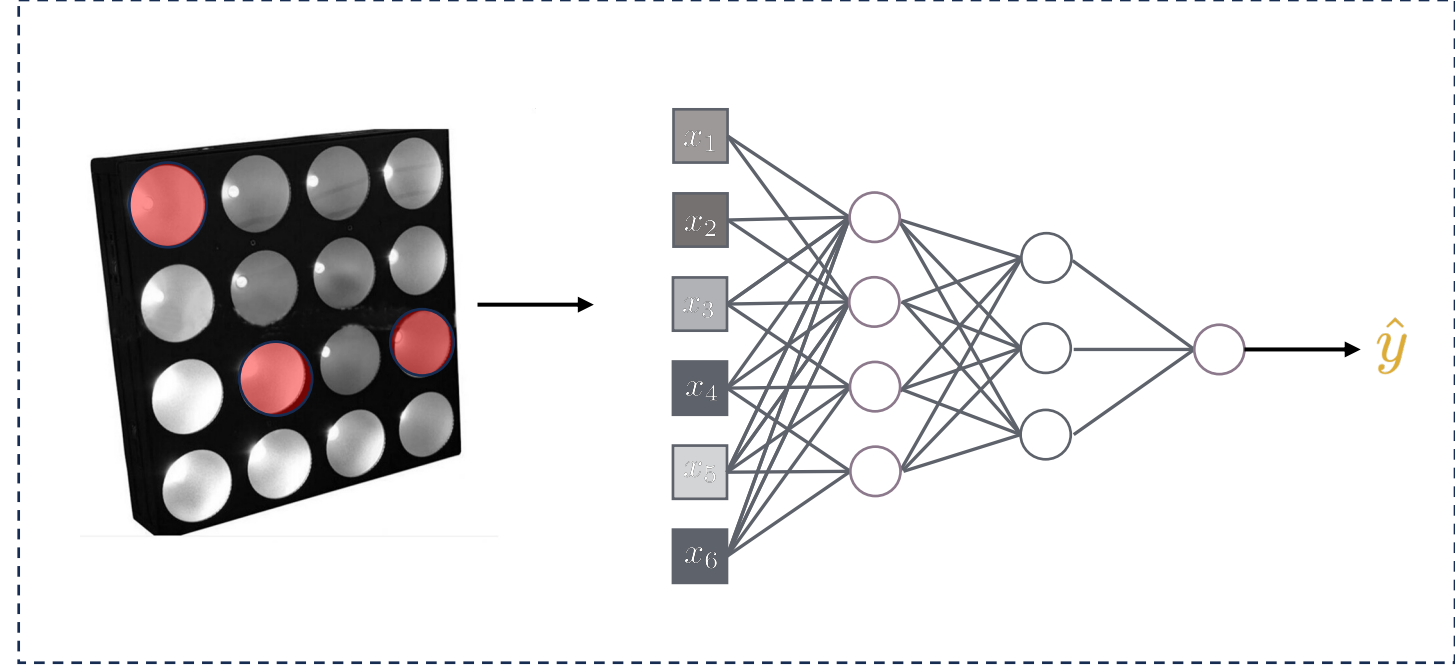
Shapley Values (The problem)

- If two pixels carry the same information and this information is critical for the model's prediction.
- Removing one pixel will not change the result because the load will be carried by the remaining pixel transmitting the same information
- The same is true for the other pixel
- Therefore, removing a single pixel at a time could result in the two most important features being ranked as least important! This is due to the coupling of pixel information, i.e., how do features interact and correlate?
- A mathematically exact way for calculating the contribution of each pixel/feature to the model's prediction is to sum over all possible subsets of pixels....
- This means 2^n combinations (each is on or off and n pixels), for a small colour image 2^{786432} unique subsets....



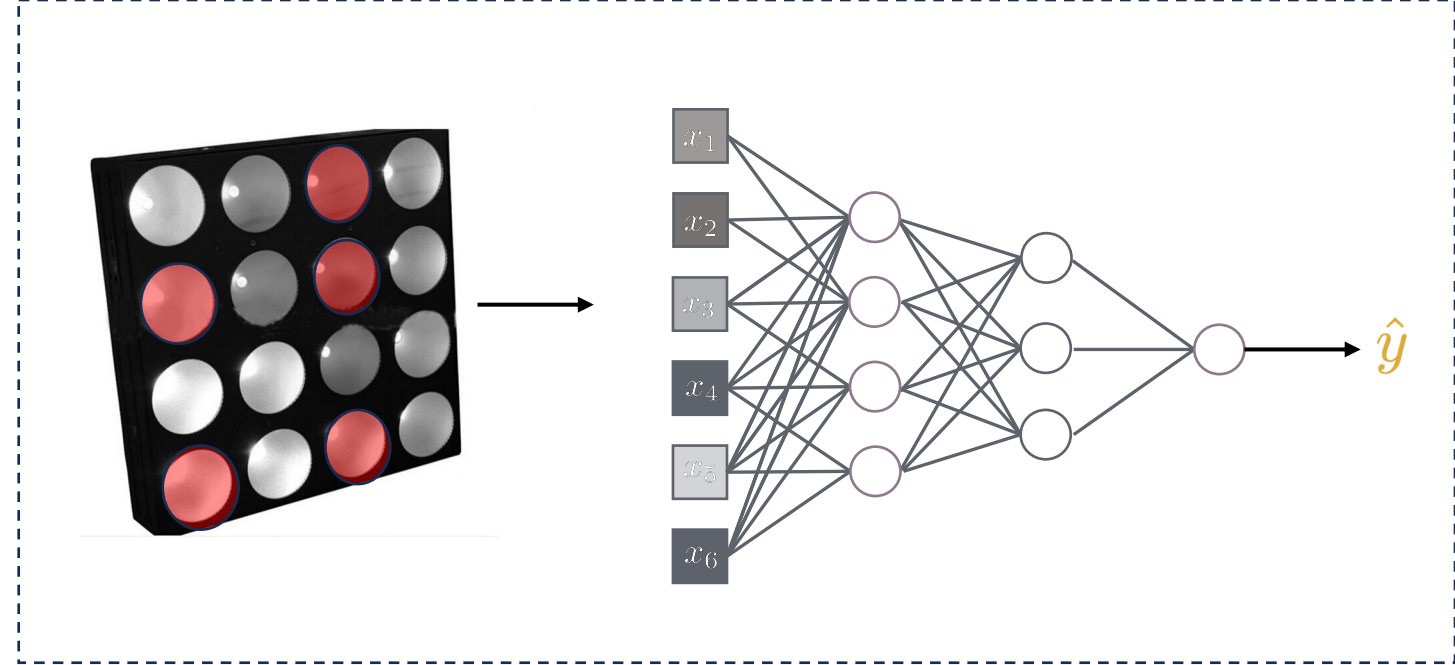
Shapley Values (The problem)

- If two pixels carry the same information and this information is critical for the model's prediction.
- Removing one pixel will not change the result because the load will be carried by the remaining pixel transmitting the same information
- The same is true for the other pixel
- Therefore, removing a single pixel at a time could result in the two most important features being ranked as least important! This is due to the coupling of pixel information, i.e., how do features interact and correlate?
- A mathematically exact way for calculating the contribution of each pixel/feature to the model's prediction is to sum over all possible subsets of pixels....
- This means 2^n combinations (each is on or off and n pixels), for a small colour image 2^{786432} unique subsets....



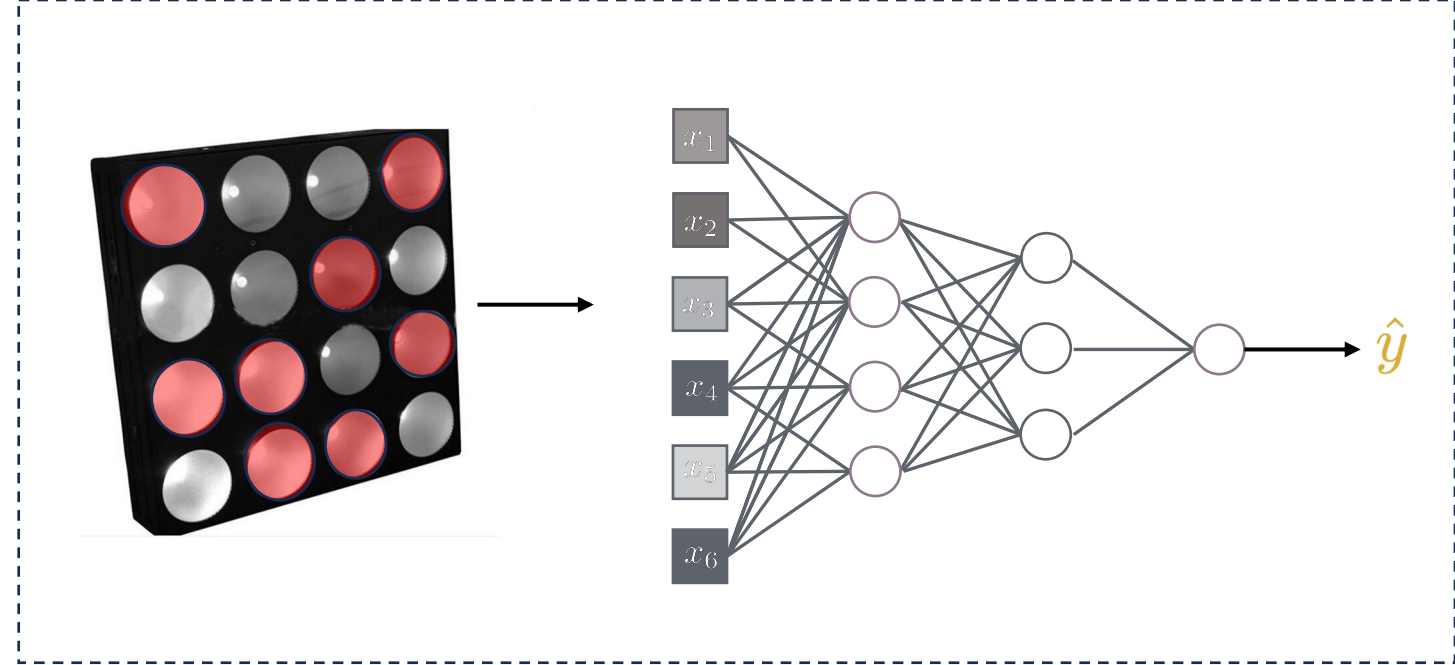
Shapley Values (The problem)

- If two pixels carry the same information and this information is critical for the model's prediction.
- Removing one pixel will not change the result because the load will be carried by the remaining pixel transmitting the same information
- The same is true for the other pixel
- Therefore, removing a single pixel at a time could result in the two most important features being ranked as least important. This is due to the coupling of pixel information, i.e., how do features interact and correlate?
- A mathematically exact way for calculating the contribution of each pixel/feature to the model's prediction is to sum over all possible subsets of pixels....
- This means 2^n combinations (each is on or off and n pixels), for a small colour image 2^{786432} unique subsets....



Shapley Values (The problem)

- If two pixels carry the same information and this information is critical for the model's prediction.
- Removing one pixel will not change the result because the load will be carried by the remaining pixel transmitting the same information
- The same is true for the other pixel
- Therefore, removing a single pixel at a time could result in the two most important features being ranked as least important. This is due to the coupling of pixel information, i.e., how do features interact and correlate?
- A mathematically exact way for calculating the contribution of each pixel/feature to the model's prediction is to sum over all possible subsets of pixels....
- This means 2^n combinations (each is on or off and n pixels), for a small colour image 2^{786432} unique subsets....



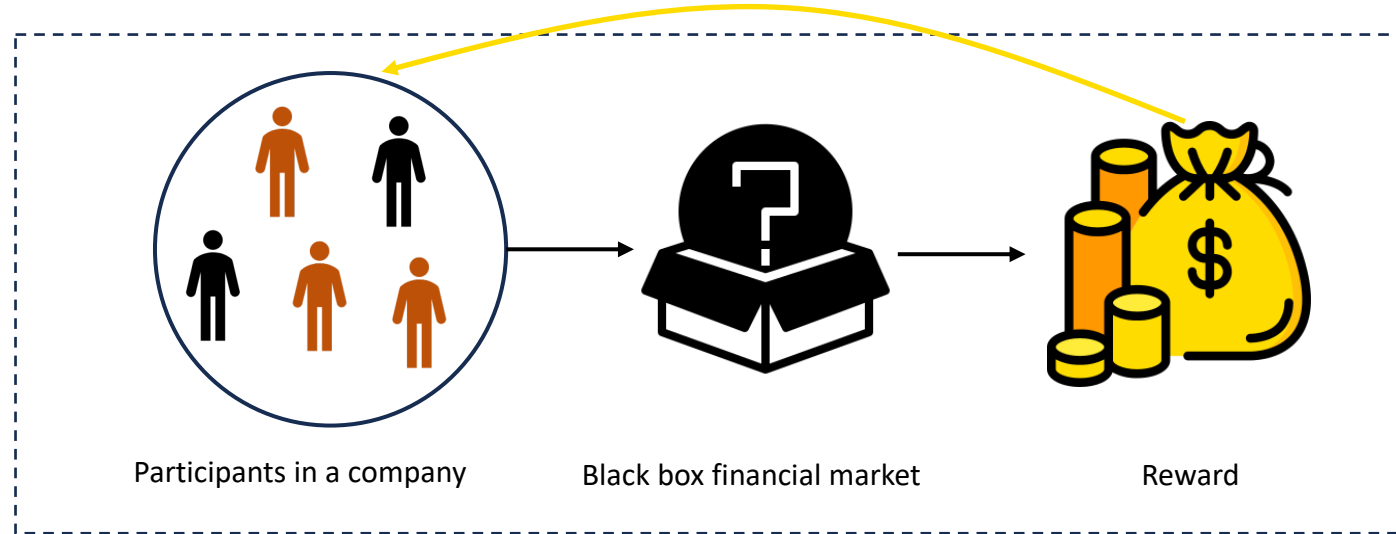
Shapley Values (The optimal solution)



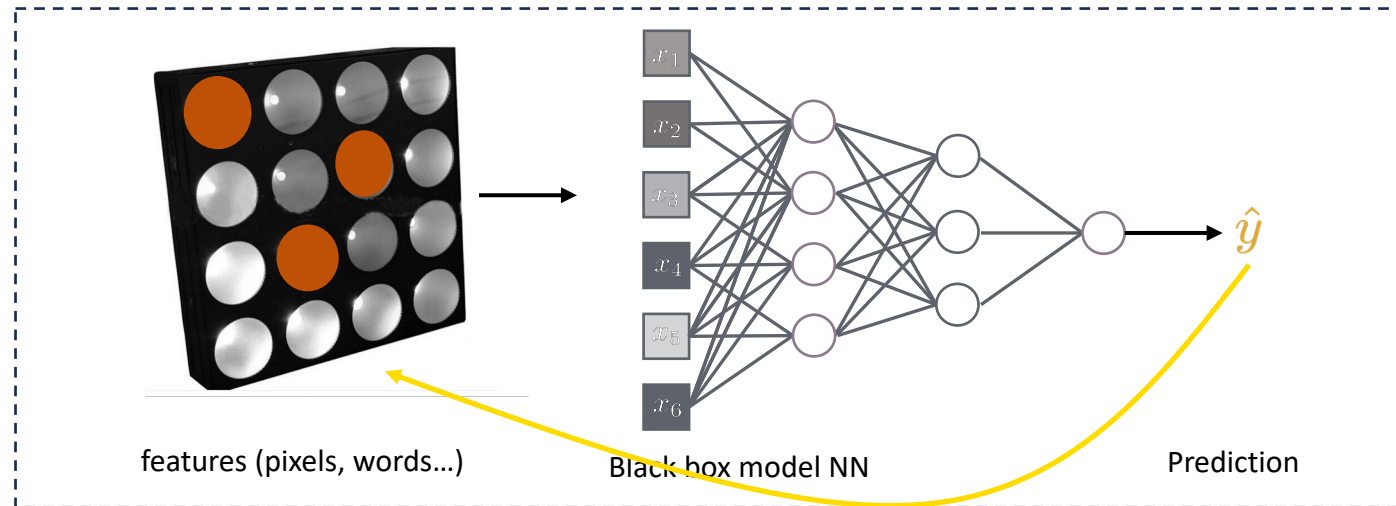
The problem has been solved in cooperative game theory

Shapley Values (The optimal solution)

Coalitions of players



Coalitions of features



Shapley Values (The maths)

$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \times (|N| - |S| - 1)!}{|N|!} (f_\theta(S \cup \{i\}) - f_\theta(S))$$

The reward/importance of the i'th person/feature is the average marginal contribution of feature i across all possible subsets, weighted by the probability of each subset occurring. This is called the features Shapley Value.

Probability of obtaining each subset S when randomly selecting features

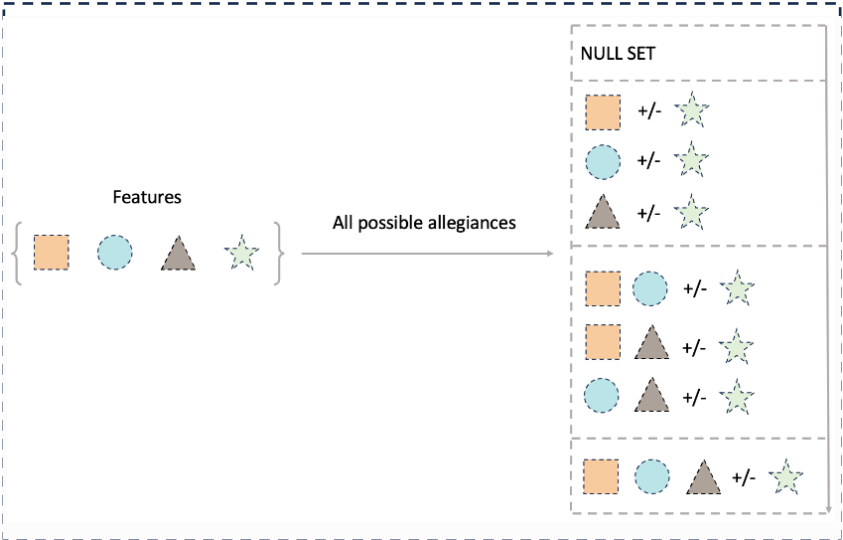
|S| = size of the subset

|N| = total number of features

Sum over all subsets of total N features

(|N| - |S| - 1) = Number of features not in S excluding i

difference between model prediction with and without feature i



Normalization factor is important otherwise sets that are small or close to N would contribute less since their are fewer subsets of these sizes

Shapley Values (The maths)

- **Efficiency** (or Additivity): The sum of all Shapley values equals the total prediction of the model when all features are included
- **Symmetry**: If two features contribute equally to all possible coalitions, their Shapley values should be equal
- **Dummy** (or Null Player): If a feature adds no marginal value to any coalition, its Shapley value should be zero
- **Linearity**: The Shapley value of a sum of two games should be the sum of the Shapley values of the individual games. This allows for decomposition of complex games into simpler ones

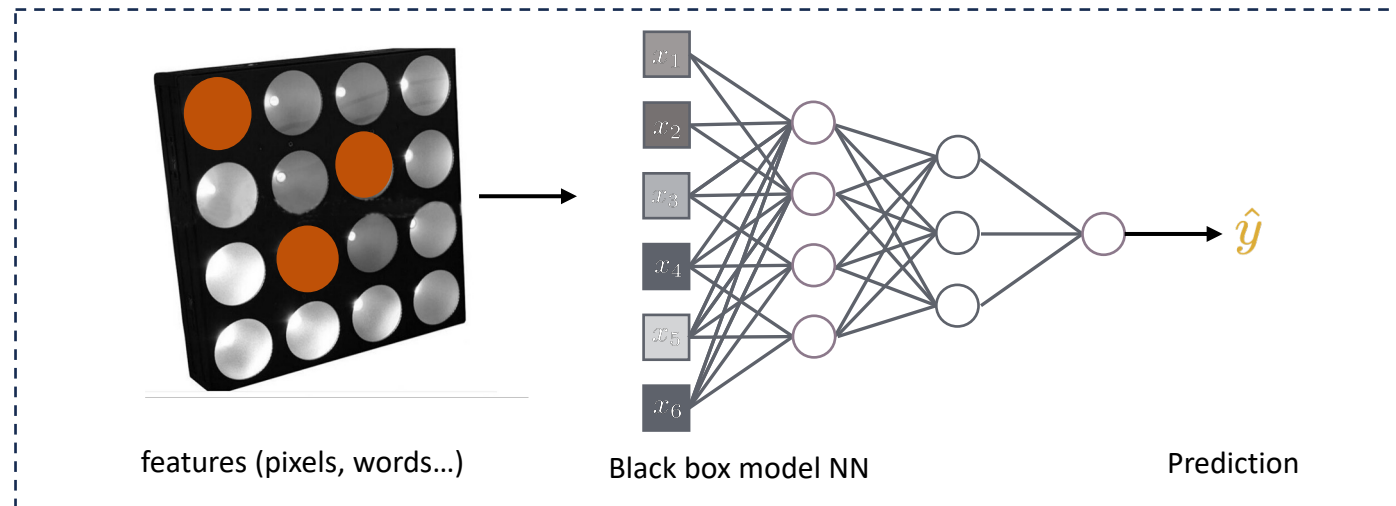
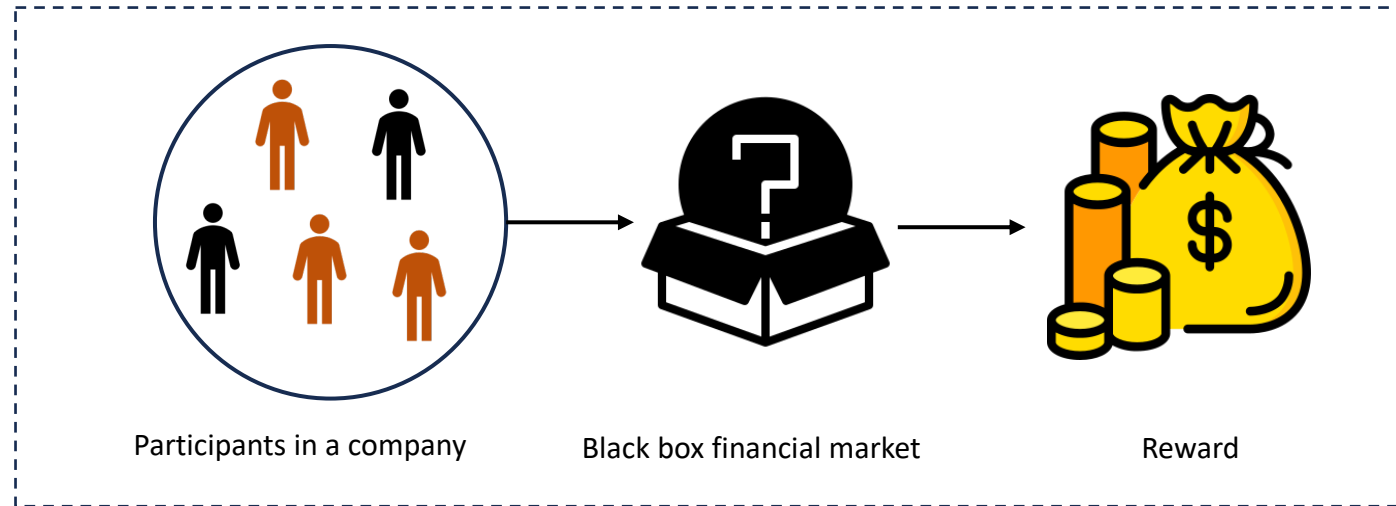
$$\sum_{i=1}^n \phi_i(v) = f(N)$$

$$\text{If } f(S \cup i) = f(S \cup j) \quad \text{Then } \phi_i(f) = \phi_j(f)$$

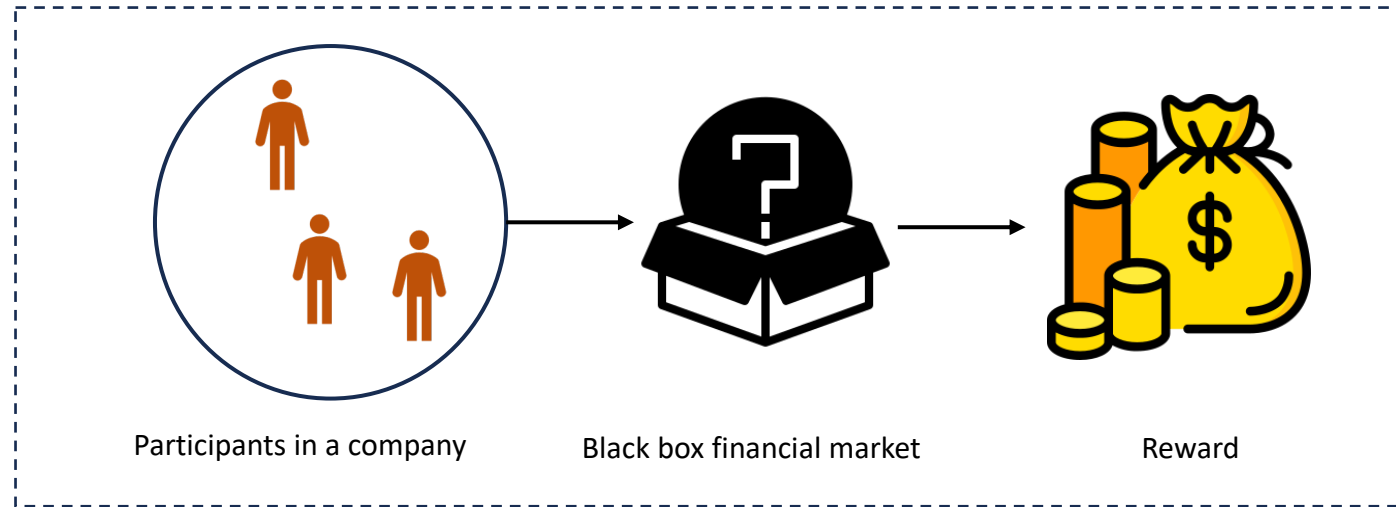
$$\text{If } f(S \cup i) = f(S) \quad \text{Then } \phi_i(f) = 0$$

$$\phi_i(f + g) = \phi_i(f) + \phi_i(g)$$

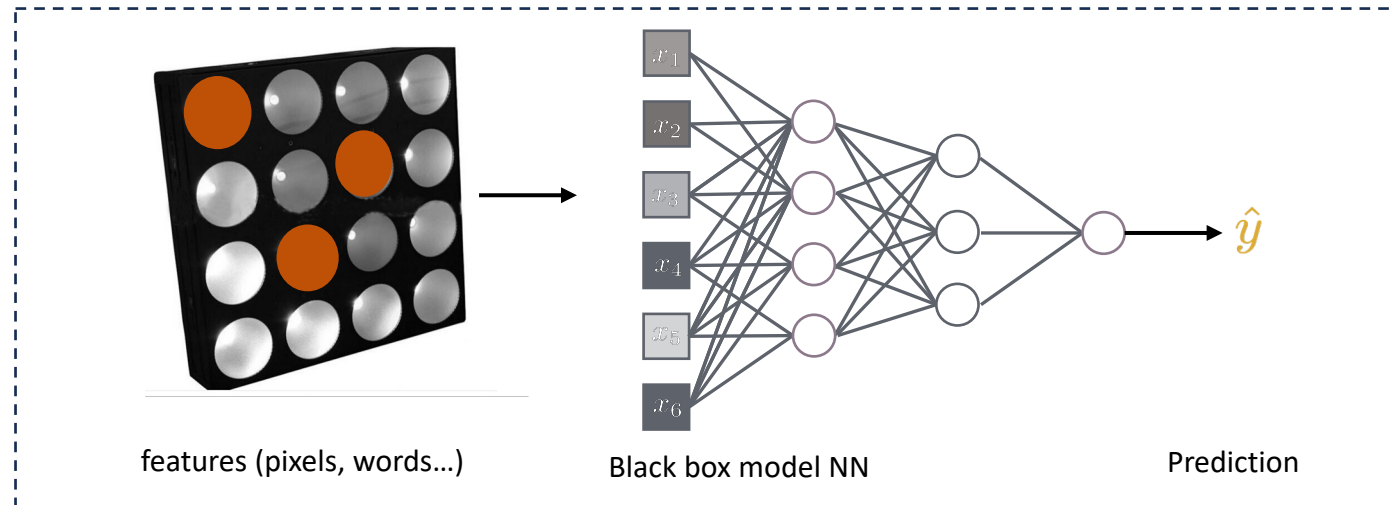
Shapley Values (The problem)



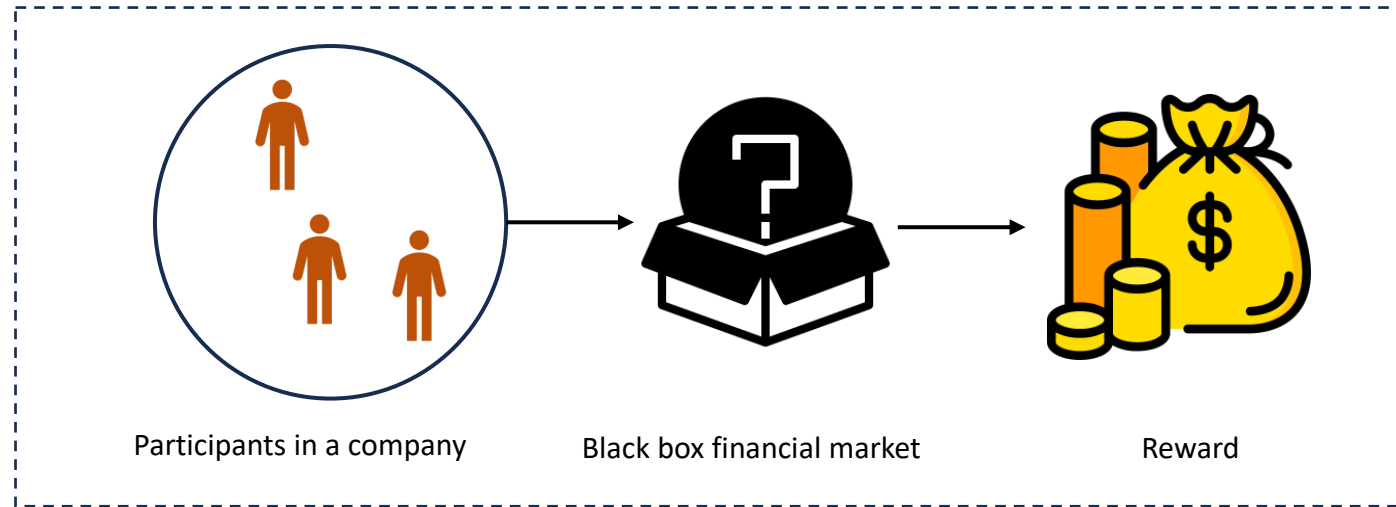
Shapley Values (The problem)



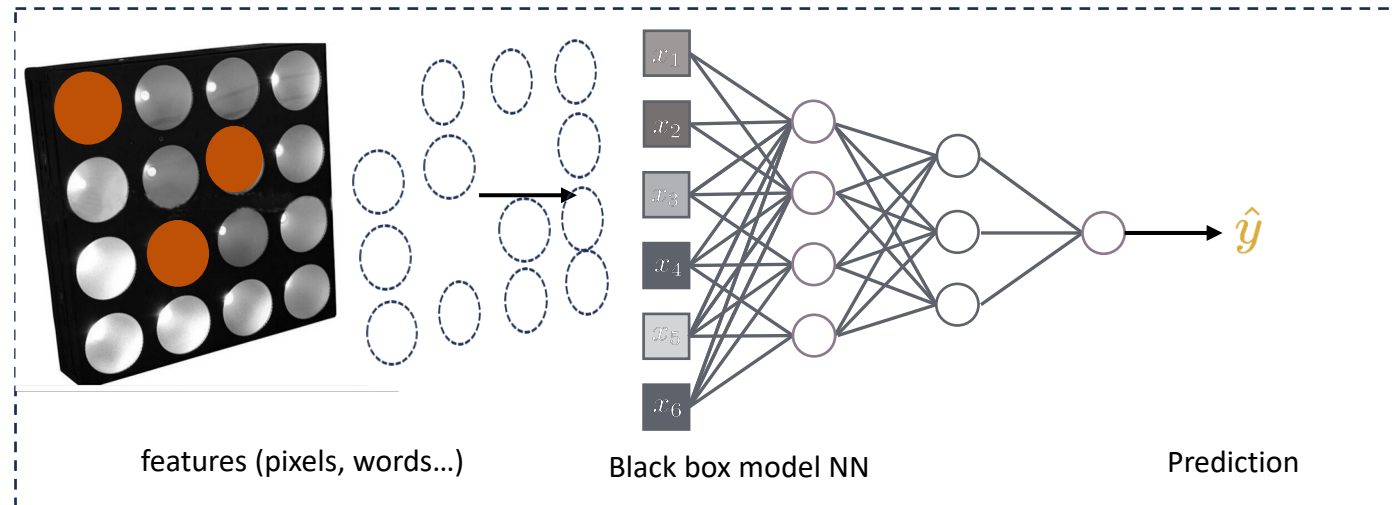
Can remove people
and rerun the experiment



Shapley Values (The problem)



Can remove people and rerun the experiment



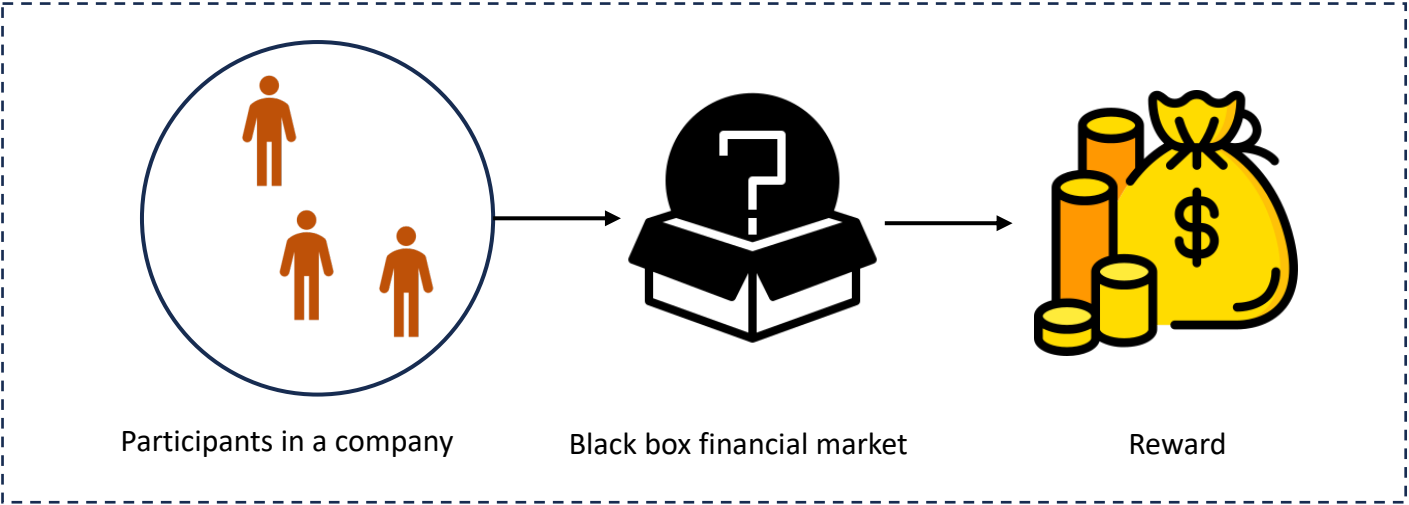
Network requires information to run through, need a good baseline to represent missingness

Shapley Values (The problem)

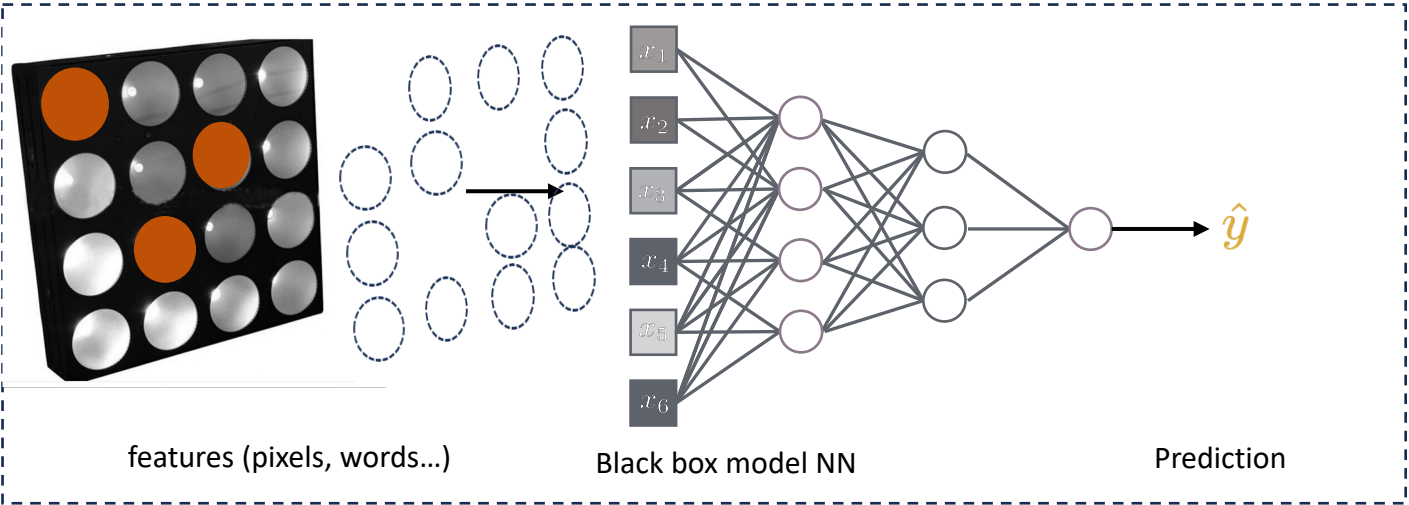
$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \times (|N| - |S| - 1)!}{|N|!} (f_\theta(S \cup \{i\}) - f_\theta(S))$$

which is what the formula requires

2^{786432} forward passes are not feasible



Can remove people and rerun the experiment

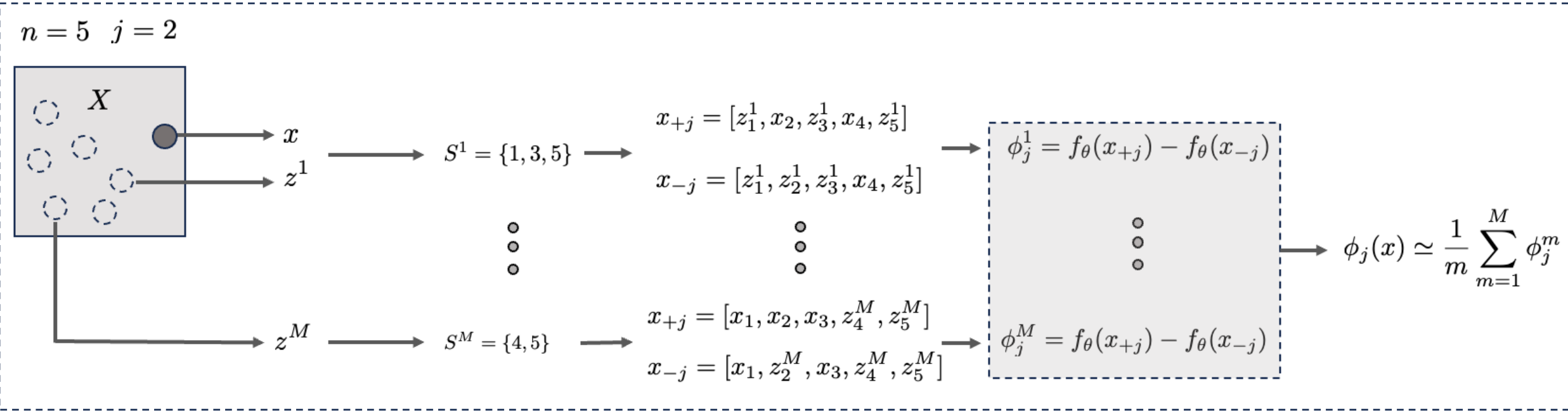


Network requires information to run through, need a good baseline to represent missingness

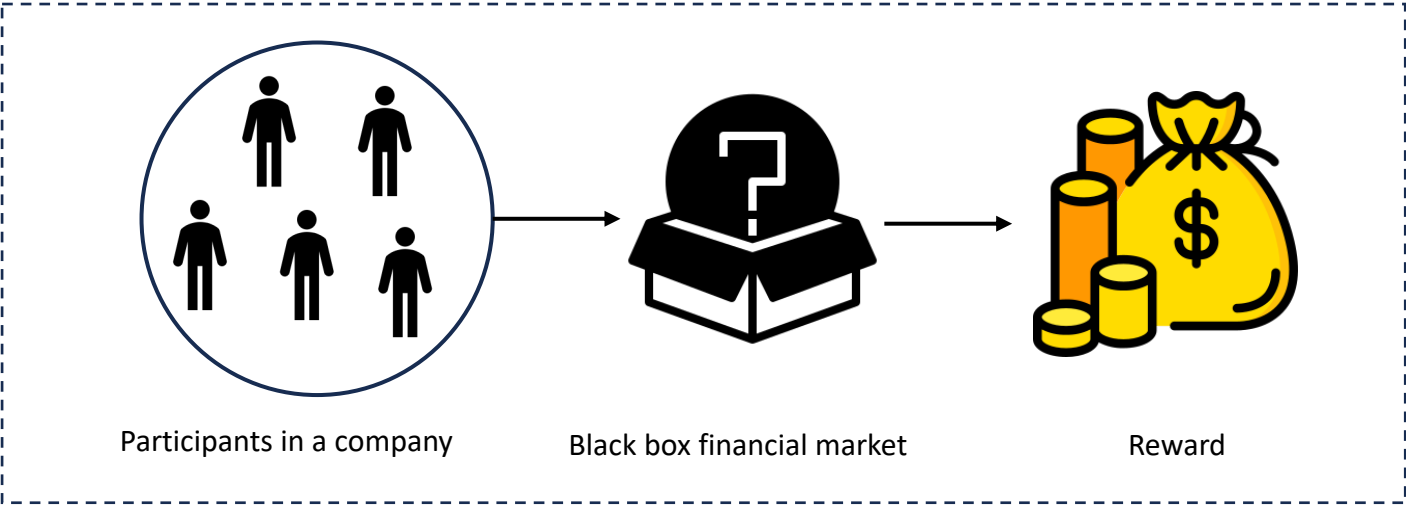
Shapley Values (Monte Carlo approximation | Tabular data)

	Fet 1	Fet 2	Fet 3	Fet 4	Fet 5
1	1	7.8	3.9	3.2	4.6
x	4.5	1.6	2.1	9.4	3.1
3	2.1	5.3	2	3.3	1.1
4	1.9	3.3	3.2	9.1	2.8
5	4.1	7.2	7.4	3.2	1.4

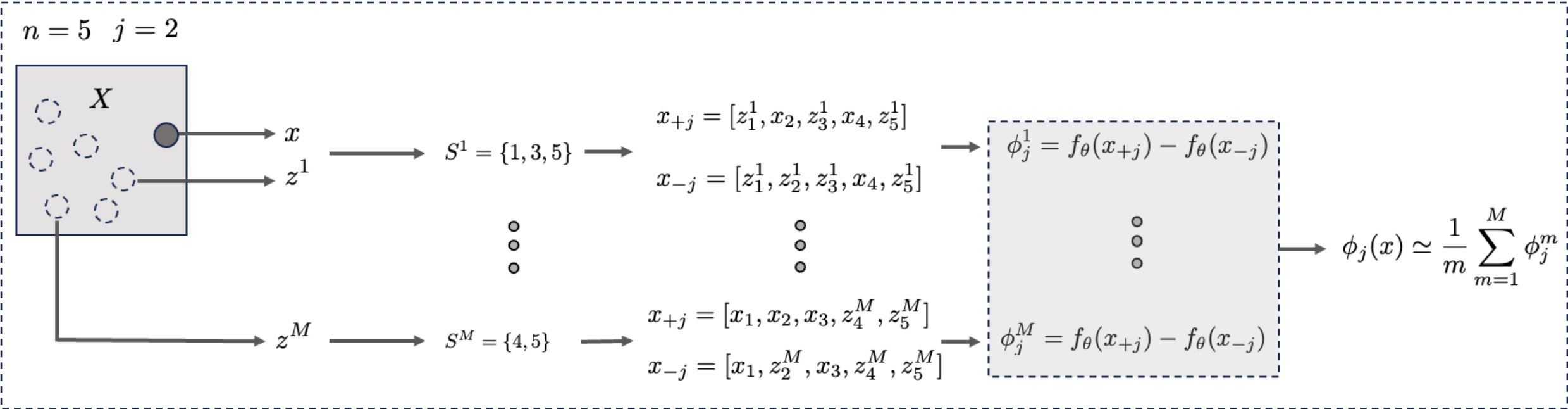
Calculate $\phi_2(x)$



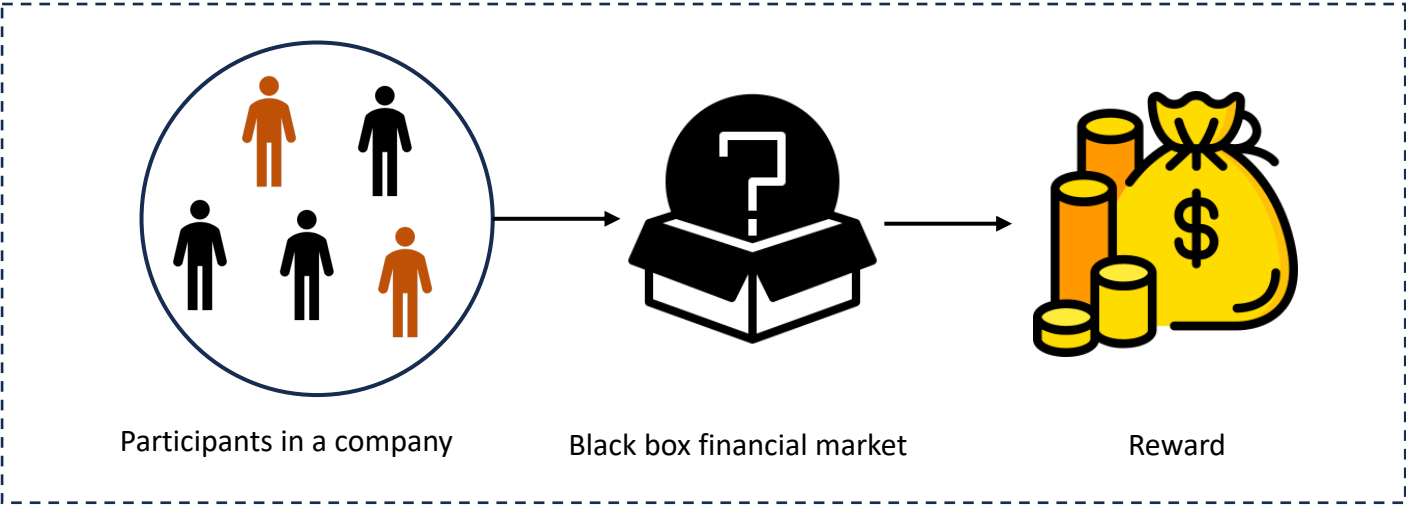
Shapley Values (Monte Carlo approximation | Tabular data)



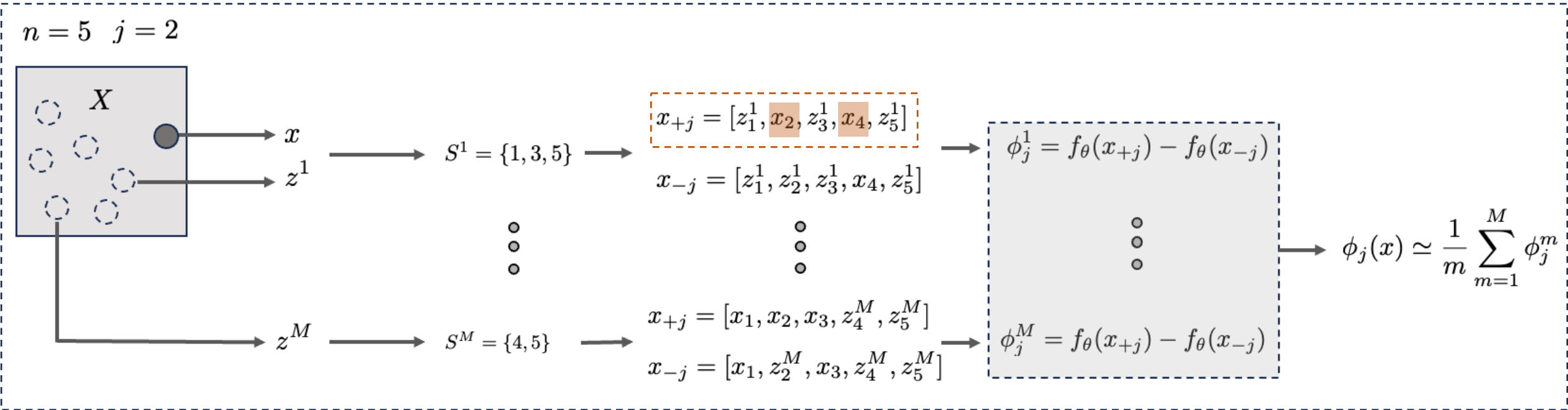
We substitute the data from other instances (rows) in the dataset



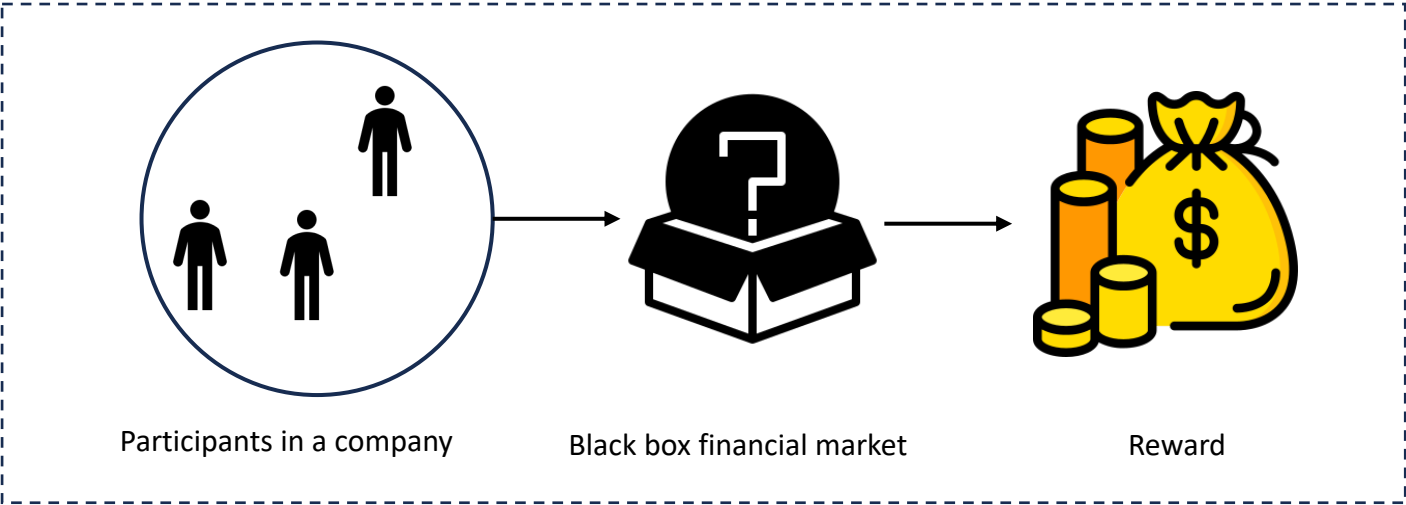
Shapley Values (Monte Carlo approximation | Tabular data)



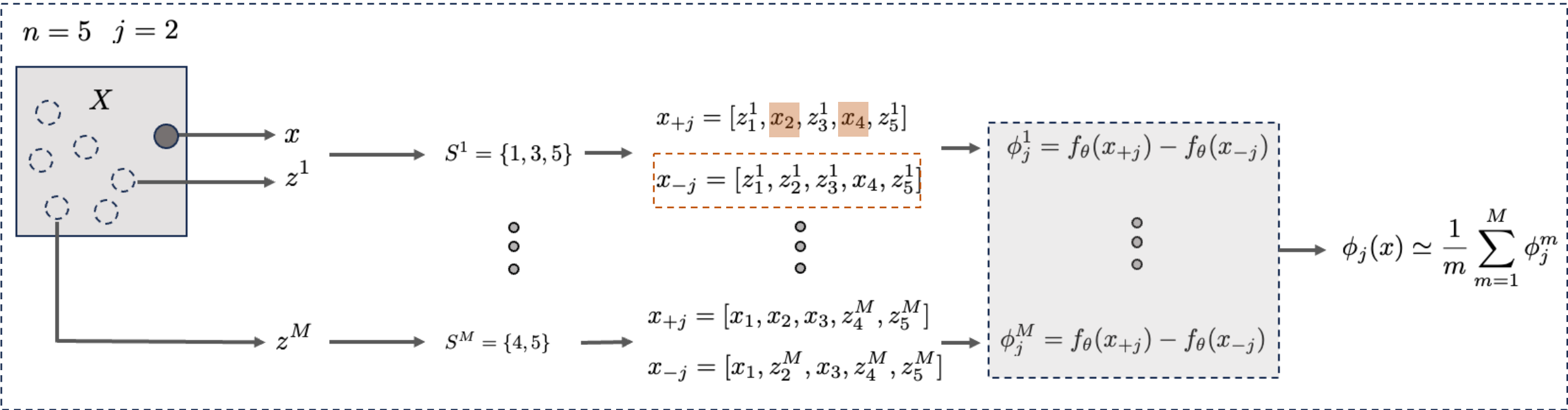
Calculate with



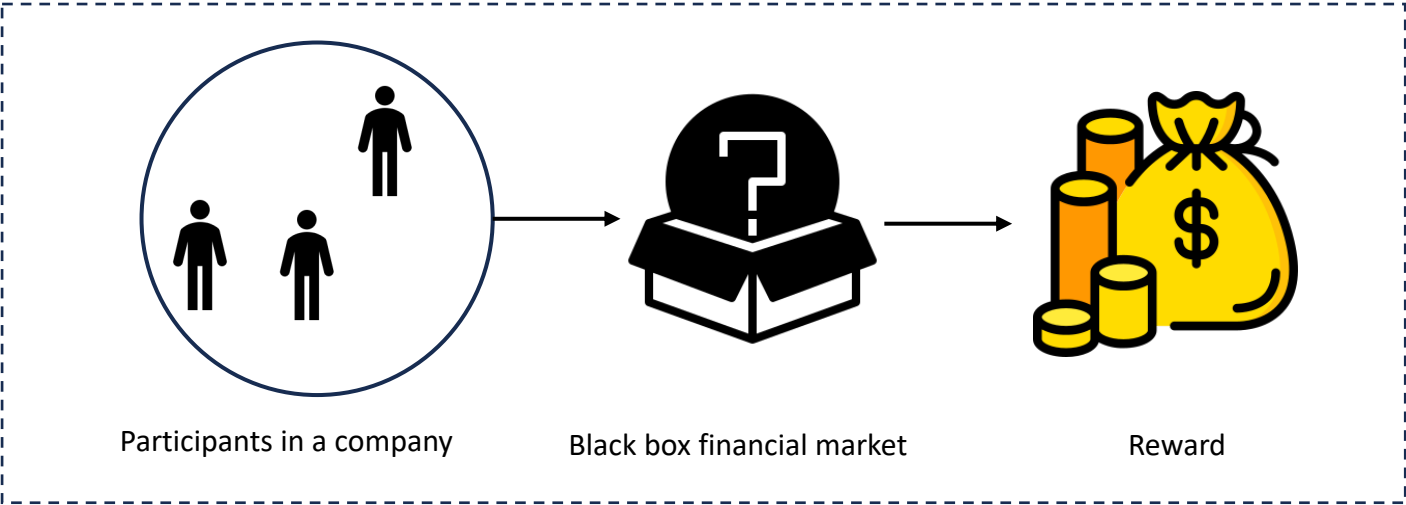
Shapley Values (Monte Carlo approximation | Tabular data)



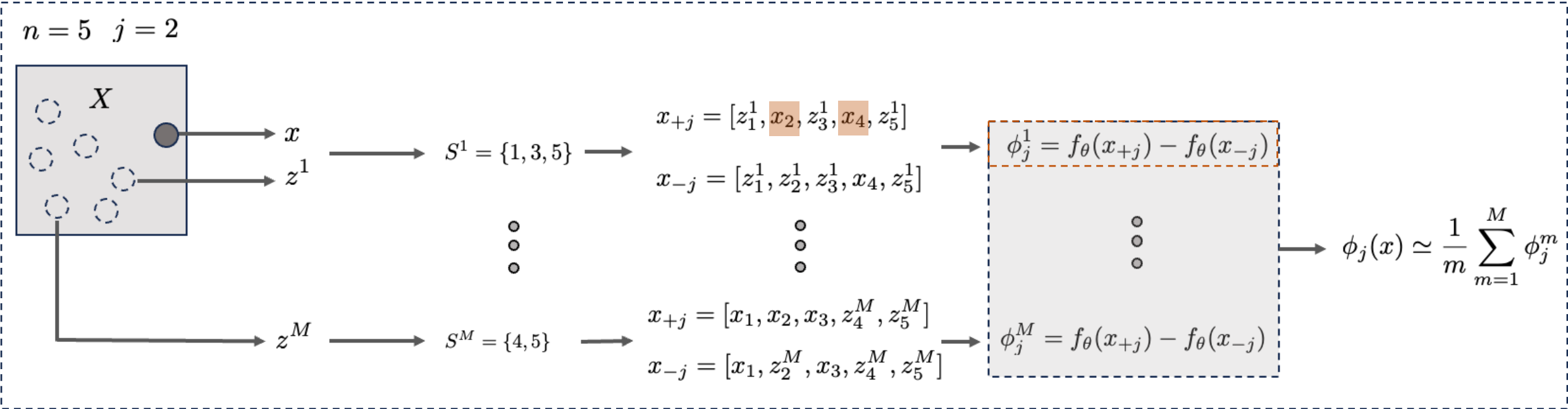
Calculate without



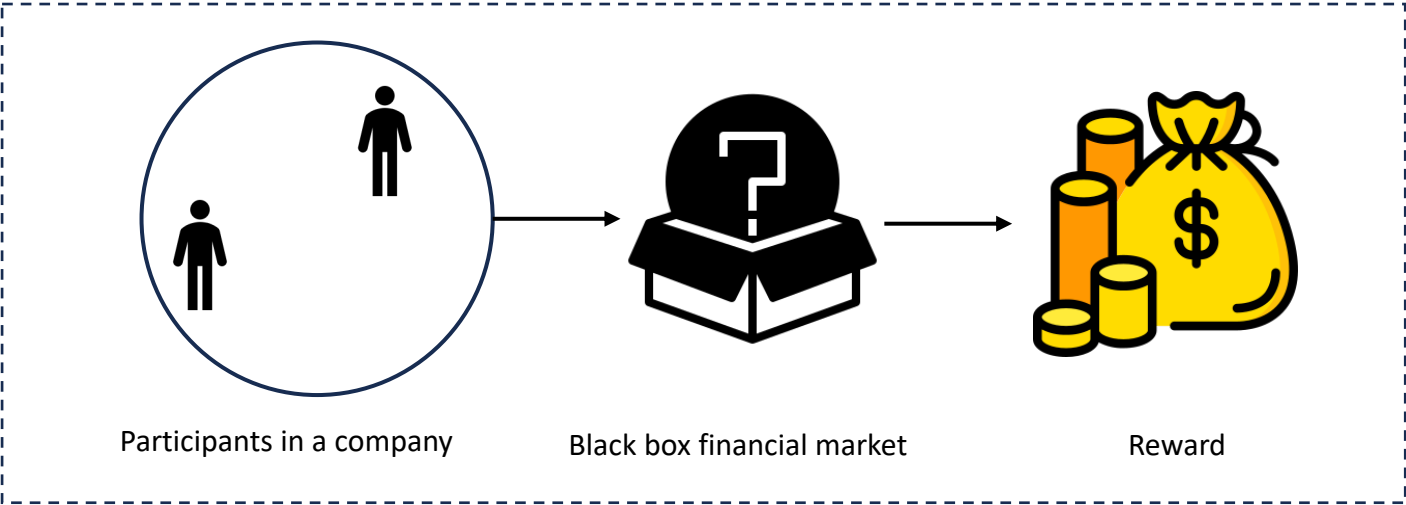
Shapley Values (Monte Carlo approximation | Tabular data)



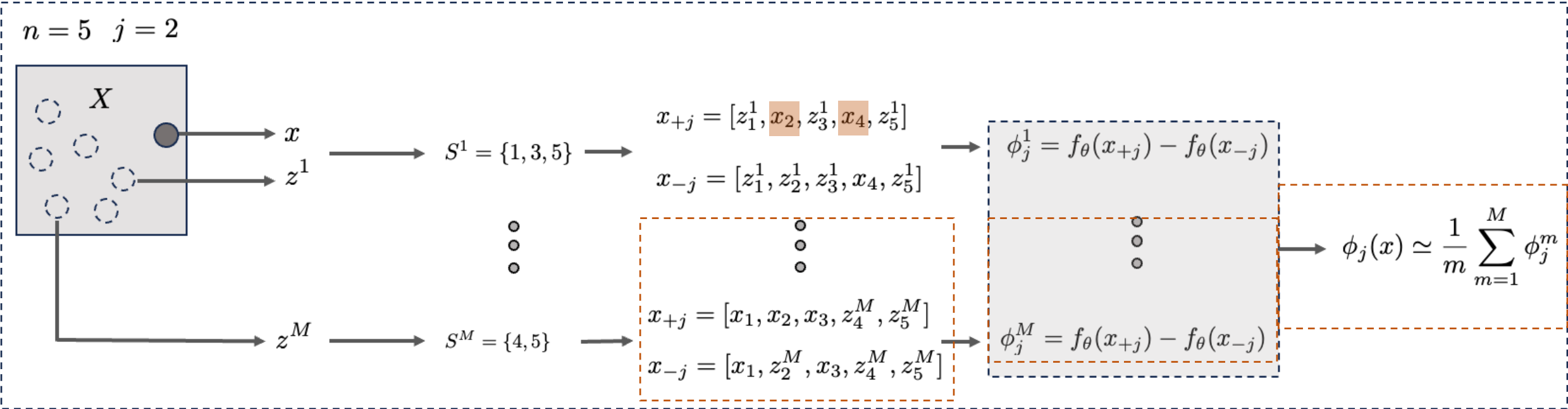
Difference with and without for one subset



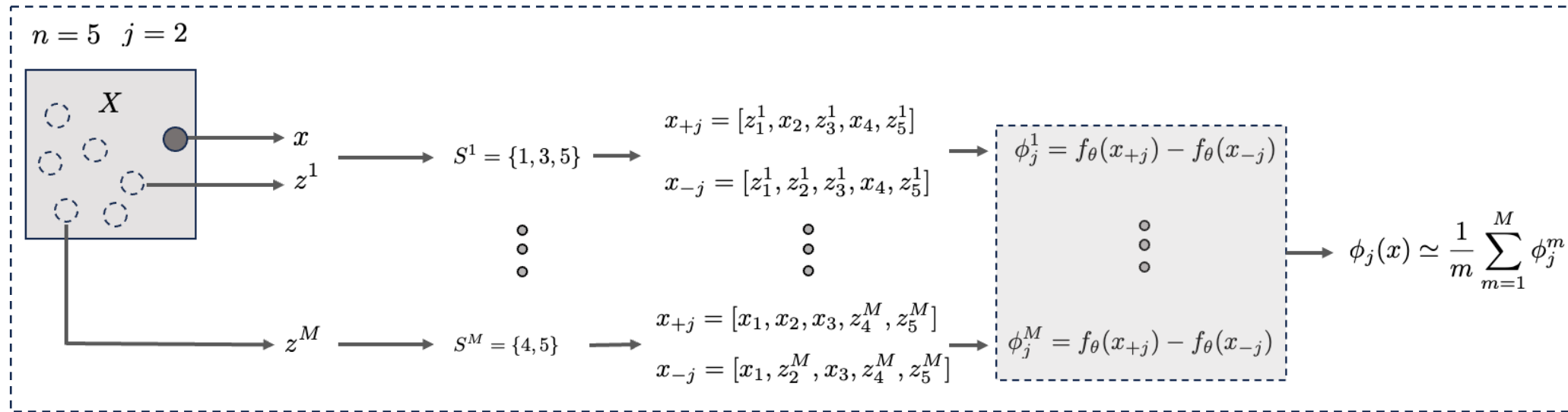
Shapley Values (Monte Carlo approximation | Tabular data)



Repeated for different subsets and take av



Shapley Values (Monte Carlo approximation)



- Approximate missingness by using random features from random samples as background
- Value with and without
- Normalization simpler because sample from a flat distribution, i.e., every combination is equally probable

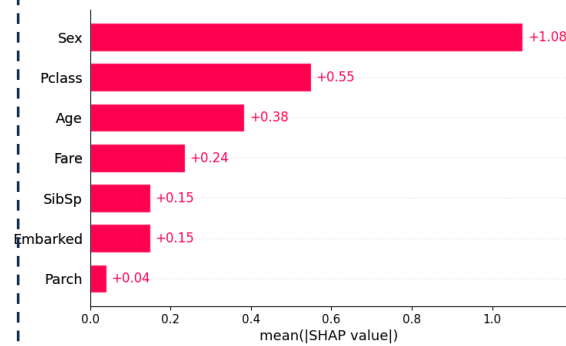
Why do we only use the background from a single instance of a time and don't mix the z 's from different instances?

To reduce the production of unrealistic data

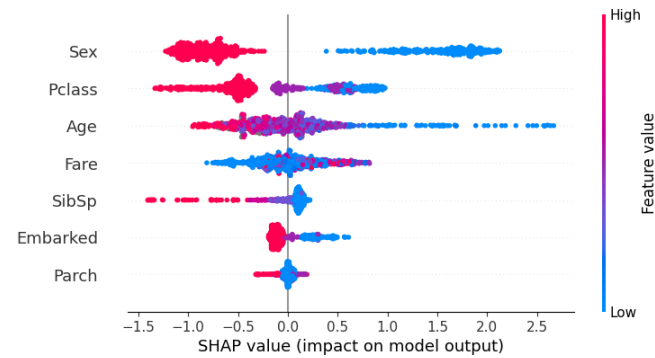
Shapley Values (Titanic | Tabular)

Target plots and interpretations

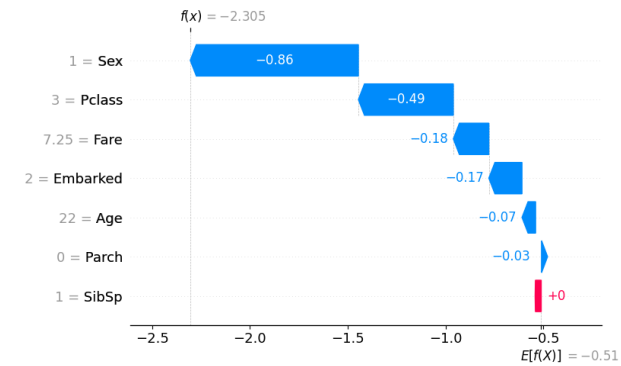
Bar



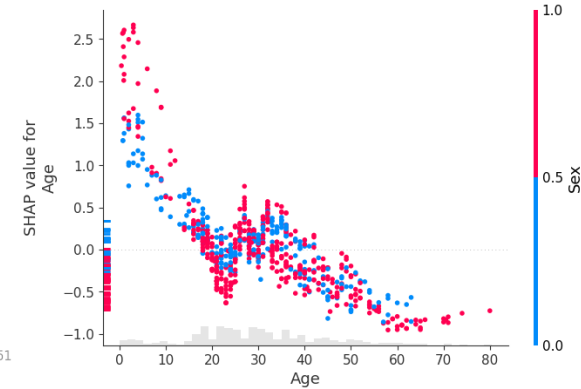
beeswarm



waterfall



Dependence



Shapley Values (Expected Gradients | Image data)

Our formula for integrated gradients $\phi_i^{IG}(f, x, x') = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\delta f(x' + \alpha(x - x'))}{\delta x_i} d\alpha$

Problem → each baseline comes with a bias



$$\begin{aligned}\phi_i^{\text{EG}}(f, x, x') &= \int_{x'} ((x_i - x'_i) \times \\ &\quad \times \int_{\alpha=0}^1 \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha(x - x'))}{\delta x_i} d\alpha) p_D(x') dx' \\ &\simeq \mathbb{E}_{x' \sim D, \alpha \sim U(0,1)} \left[(x_i - x'_i) \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha \times (x - x'))}{\delta x_i} \right]\end{aligned}$$

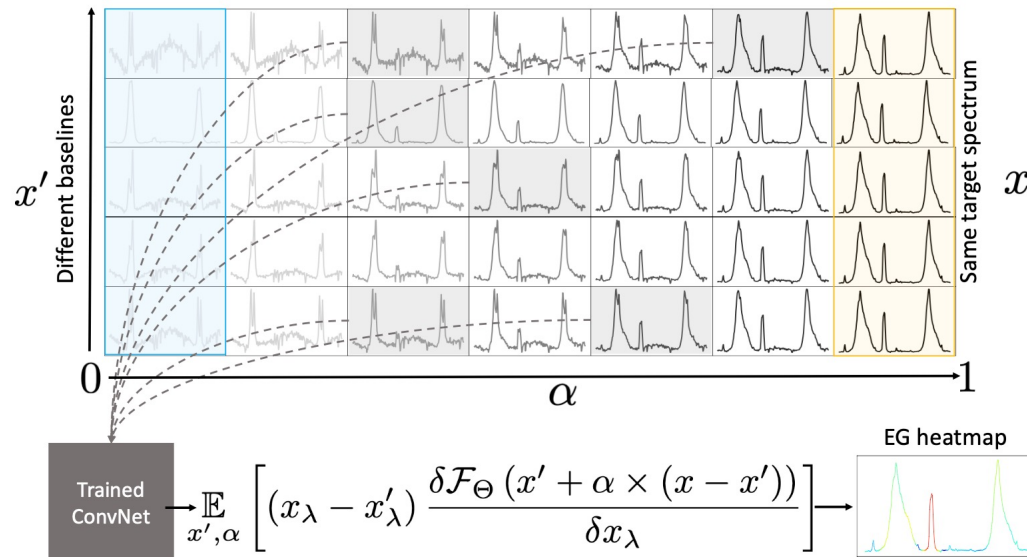
Shapley value treatment in image space

Shapley Values (Expected Gradients | Image data)

$$\begin{aligned}\phi_i^{\text{EG}}(f, x, x') &= \int_{x'} ((x_i - x'_i) \times \\ &\times \int_{\alpha=0}^1 \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha(x - x'))}{\delta x_i} d\alpha) p_D(x') dx' \\ &\simeq \mathbb{E}_{x' \sim D, \alpha \sim U(0,1)} \left[(x_i - x'_i) \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha \times (x - x'))}{\delta x_i} \right]\end{aligned}$$

- An additional integral over multiple image paths
- We sample from the dataset
- We take the expectation because the integral is intractable

Example



1. Start from different baselines (blue)
2. Linearly Interpolate to target, i.e., different paths to same (orange)
3. Calculate accumulated grads along each path for each wavelength
4. Expectation value instead of intractable integral
5. Create heatmap from pixel/wavelength attributions
6. Require a background dataset (500 randomly sampled spectra)

Shapley Values (Expected Gradients | Image data)

$$\begin{aligned}\phi_i^{\text{EG}}(f, x, x') &= \int_{x'} ((x_i - x'_i) \times \\ &\quad \times \int_{\alpha=0}^1 \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha(x - x'))}{\delta x_i} d\alpha) p_D(x') dx' \\ &\simeq \mathbb{E}_{x' \sim D, \alpha \sim U(0,1)} \left[(x_i - x'_i) \frac{\delta \mathcal{F}_{\Theta}(x' + \alpha \times (x - x'))}{\delta x_i} \right]\end{aligned}$$

- An additional integral over multiple image paths
- We sample from the dataset
- We take the expectation because the integral is intractable

We don't want to see the effects of our baseline on the model's output

All the other baselines biased the results (black, white, blur), even the gaussian noise assumes pixels are uncorrelated

By using the background dataset as a baseline, we ensure that we respect the dependencies between pixels

Additionally, integrating along many image paths against this background ensures that in the limit, the baseline leaves the output stationary with respect to the model's average prediction, since some feature will fight to increase the model's prediction, while others will try to decrease it

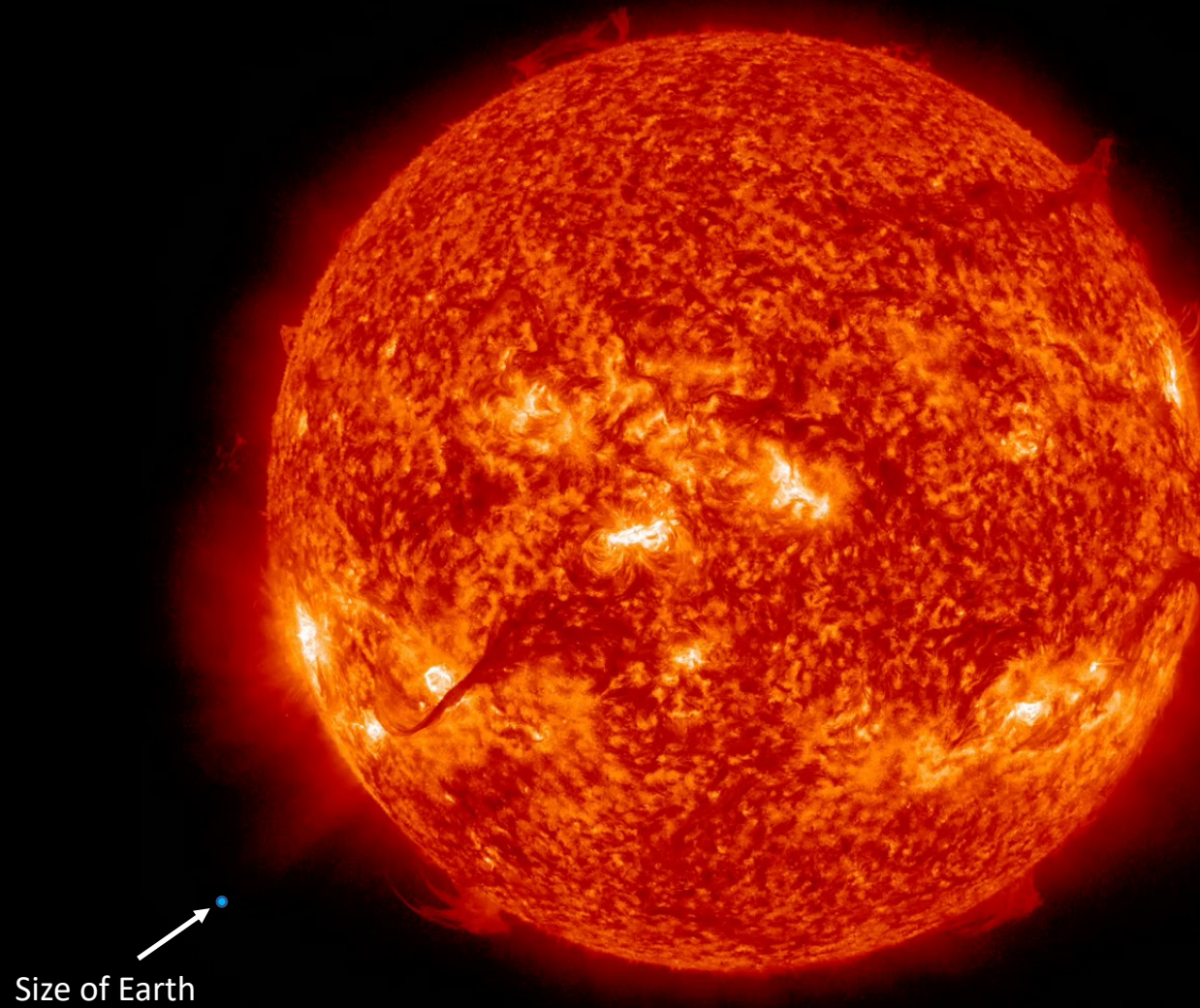
This implies that the additional integration over multiple image paths is an effective way to take pseudo subsets, i.e., the additional integral allows us to effectively turn features on and off without introducing bias into our explanation.

$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \times (|N| - |S| - 1)!}{|N|!} (f_{\theta}(S \cup \{i\}) - f_{\theta}(S))$$

Shapley formula

Shapley Values (Expected Gradients | Example application)

Solar flares are the most energetic events in our solar system

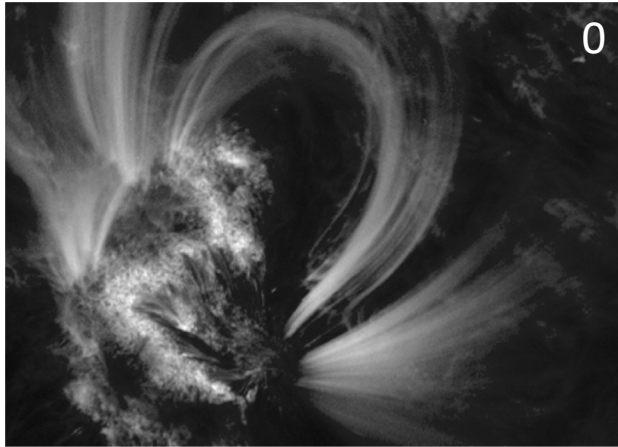


Size of Earth

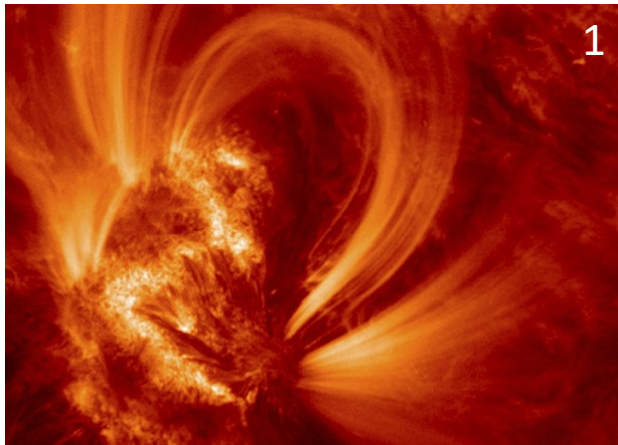
AIA 304 - 2012/08/31 - 18:26:31Z

Shapley Values (Expected Gradients | Example application)

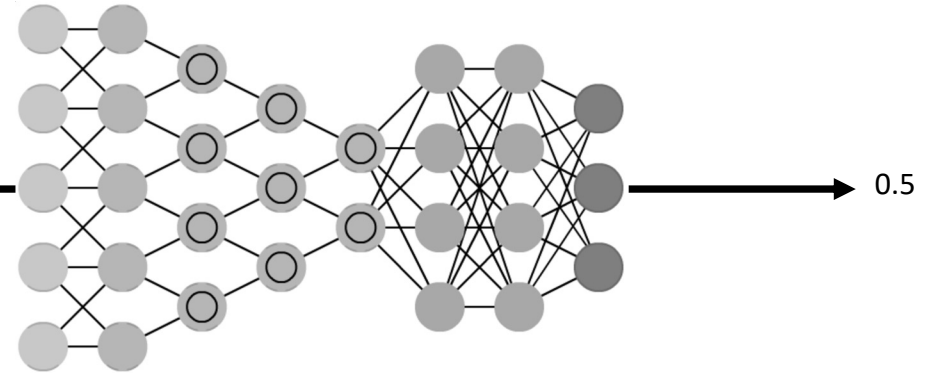
Active Region Dataset



Pre-Flare Dataset



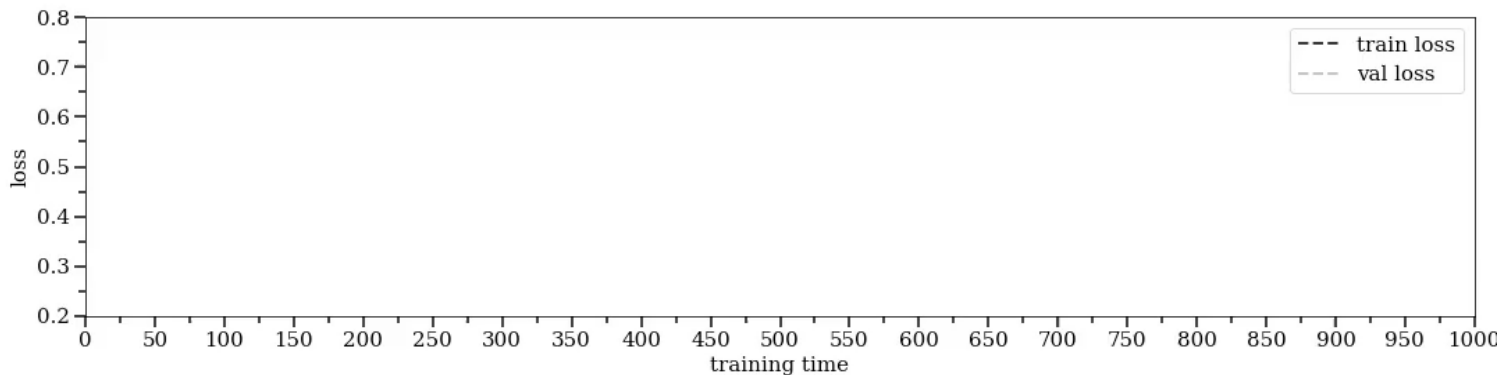
Trying to isolate pre flare signals



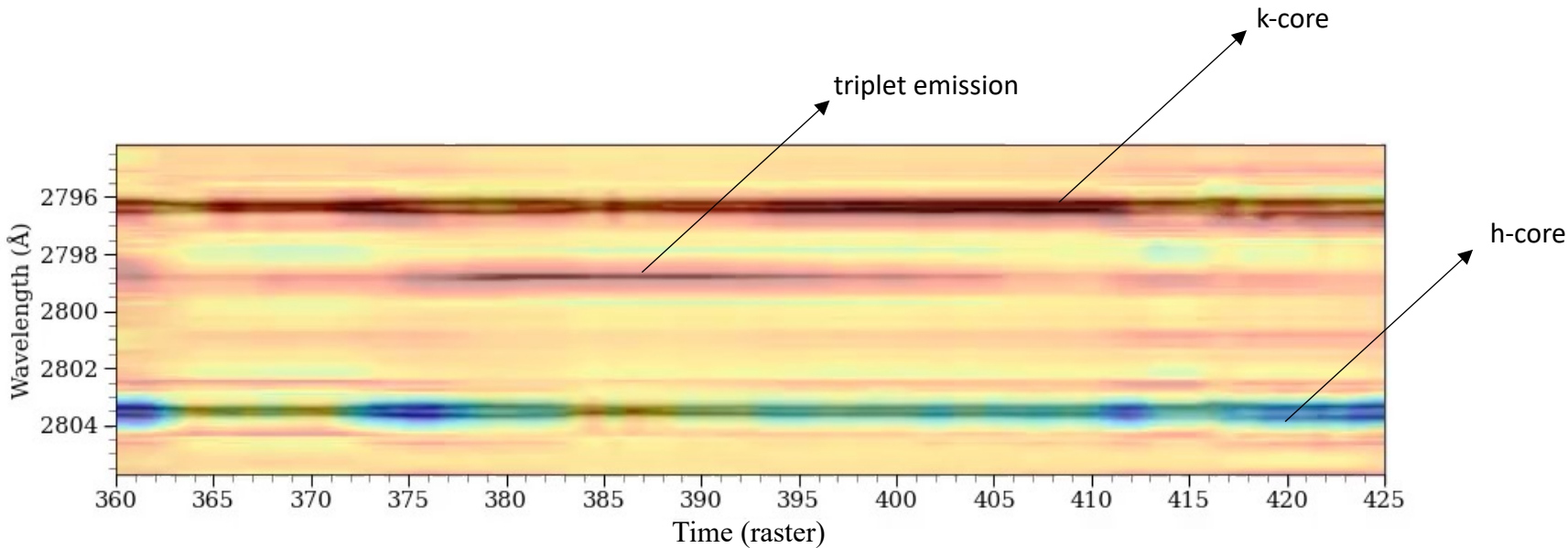
Trained a model to distinguish between spectra that lead to a flare and spectra that did not lead to a flare

Shapley Values (Expected Gradients | Example application)

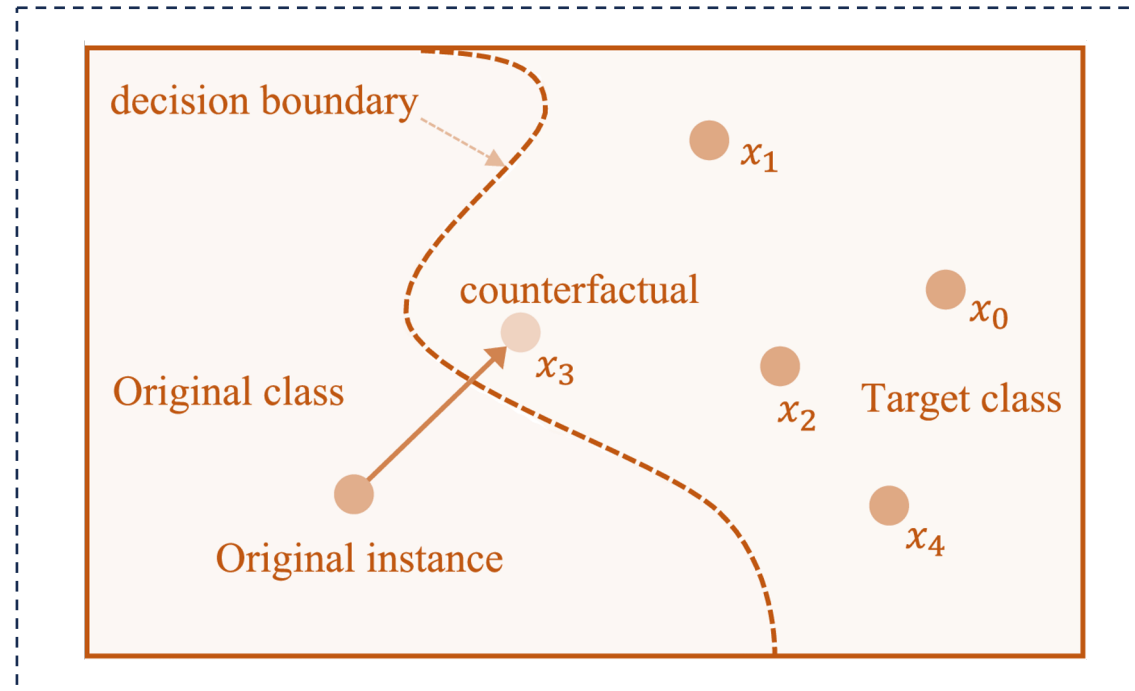
The model improves when the grey curve decreases



Spectrogram
Time along the
x-axis, wavelength
along the y-axis.



Model's attention during training



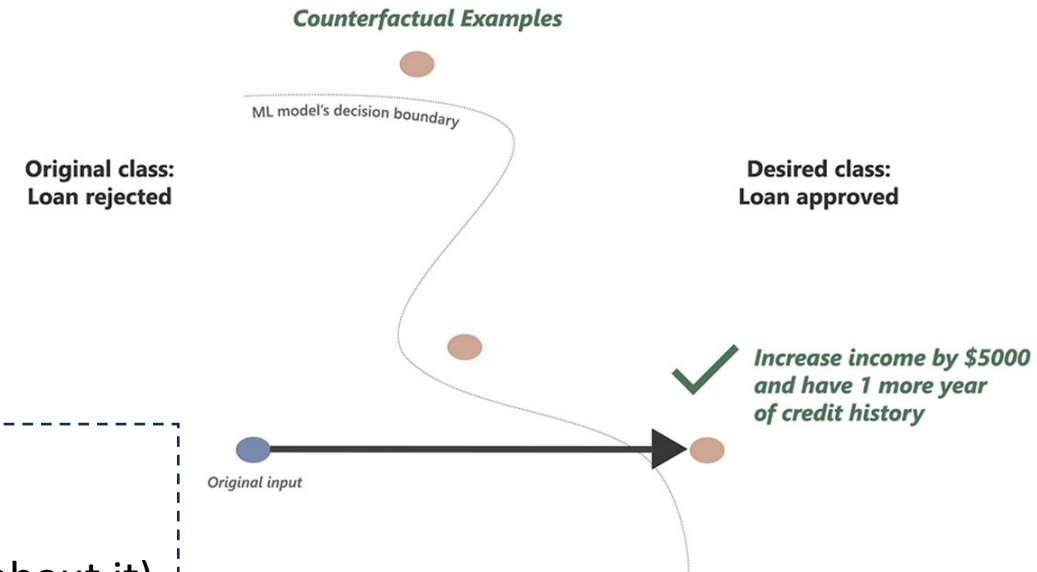
Counterfactual Explanations (**Idea**)

A counterfactual explanation of a prediction describes the smallest change to the feature values that changes the prediction to a predefined output.

Counterfactuals are **human-friendly** explanations because they are **contrastive to the current instance** and because they are **selective**, meaning they usually focus on a small number of feature changes.

What characteristics do we want from our counterfactuals?

- Should **reach the target output** (probabilistically within a small region about it)
- Should be as **similar as possible to the instance** regarding feature values
- Should also **change as few features** as possible.
- Should produce **multiple diverse** counterfactual explanations
- Should have **feature values** that are **likely**.



Let's analyze the utility of these desirable characteristics for two different problems, a classification and regression problem

Counterfactual Explanations (Cases studies)

Case study 1 (classification)

Peter applies for a loan and gets rejected by the (machine learning-powered) banking software. He wonders why his application was rejected and how he might improve his chances to get a loan. The question of “why” can be formulated as a counterfactual: What’s the smallest change to the features (income, number of credit cards, age, ...) that would change the prediction from rejected to approved? One possible answer could be: If Peter would earn 10,000 more per year, he would get the loan. Or if Peter had fewer credit cards and had not defaulted on a loan five years ago, he would get the loan. Peter will never know the reasons for the rejection, as the bank has no interest in transparency, but that is another story.



Case study 2 (regression)

In our second example, we want to explain a model that predicts a continuous outcome with counterfactual explanations. Anna wants to rent out her apartment, but she is not sure how much to charge for it, so she decides to train a machine learning model to predict the rent. Of course, since Anna is a data scientist, that is how she solves her problems. After entering all the details about size, location, whether pets are allowed, and so on, the model tells her that she can charge 900 EUR. She expected 1000 EUR or more, but she trusts her model and decides to play with the feature values of the apartment to see how she can improve the value of the apartment. She finds out that the apartment could be rented out for over 1000 EUR if it were 15 m² larger. Interesting, but non-actionable knowledge, because she cannot enlarge her apartment. Finally, by tweaking only the feature values under her control (built-in kitchen yes/no, pets allowed yes/no, type of floor, etc.), she finds out that if she allows pets and installs windows with better insulation, she can charge 1000 EUR. Anna has intuitively worked with counterfactuals to change the outcome. Note that Anna worked with the rent prediction model and wasn't necessarily interested in whether these factors are truly causal for higher rent in the “real world.”



Counterfactual Explanations (Wachter et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Should be close to
desired target

Loan ✓

Rent ~ 1000 CHF

Should be as similar to the original
instances as possible

A higher value of Lambda means that we prefer counterfactuals with predictions close to the desired outcome, (get the loan, and achieve a rent of 1000 CHF) while a lower value means that we prefer counterfactuals that are very similar to the original instance.

What happens if Lambda is very large?

What happens in the case of Lambda being very small?

$$\arg \min_{\mathbf{x}'} \max_{\lambda} L(\mathbf{x}, \mathbf{x}', y', \lambda).$$

Tricky to select the best Lambda

Counterfactual Explanations (Wachter et al.)

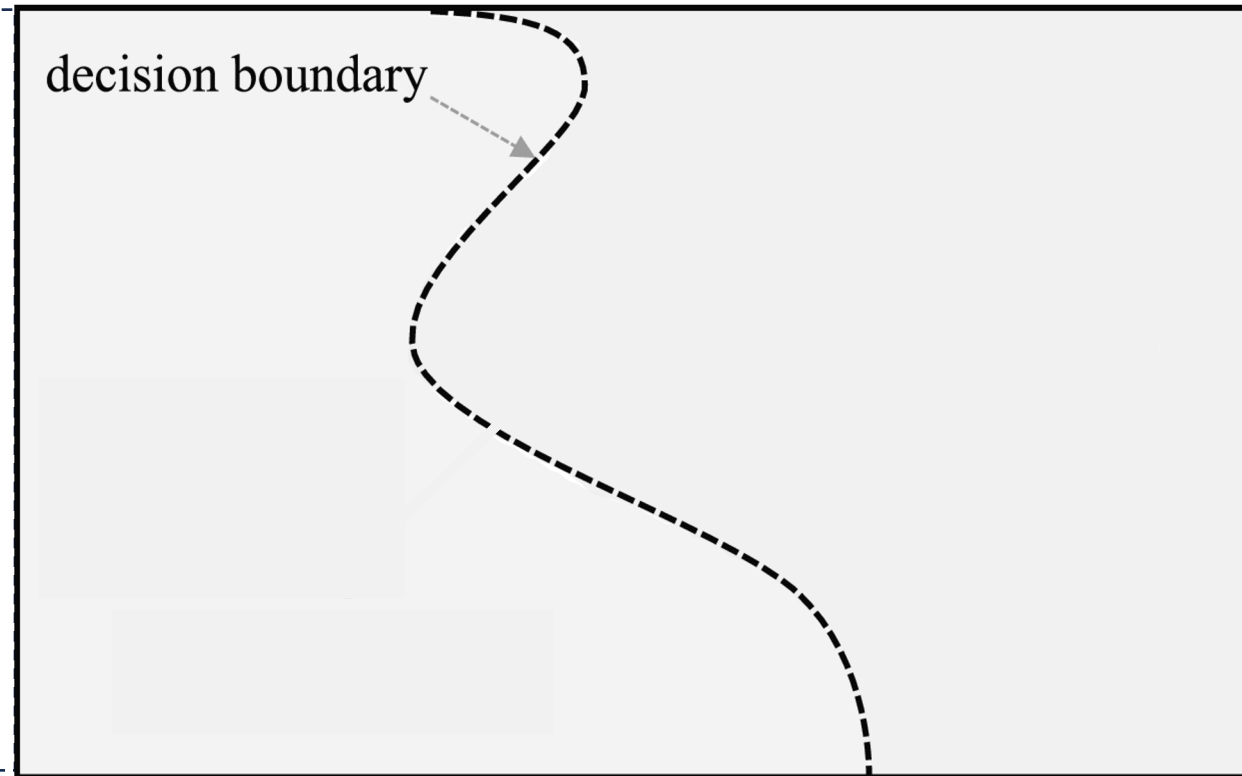
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

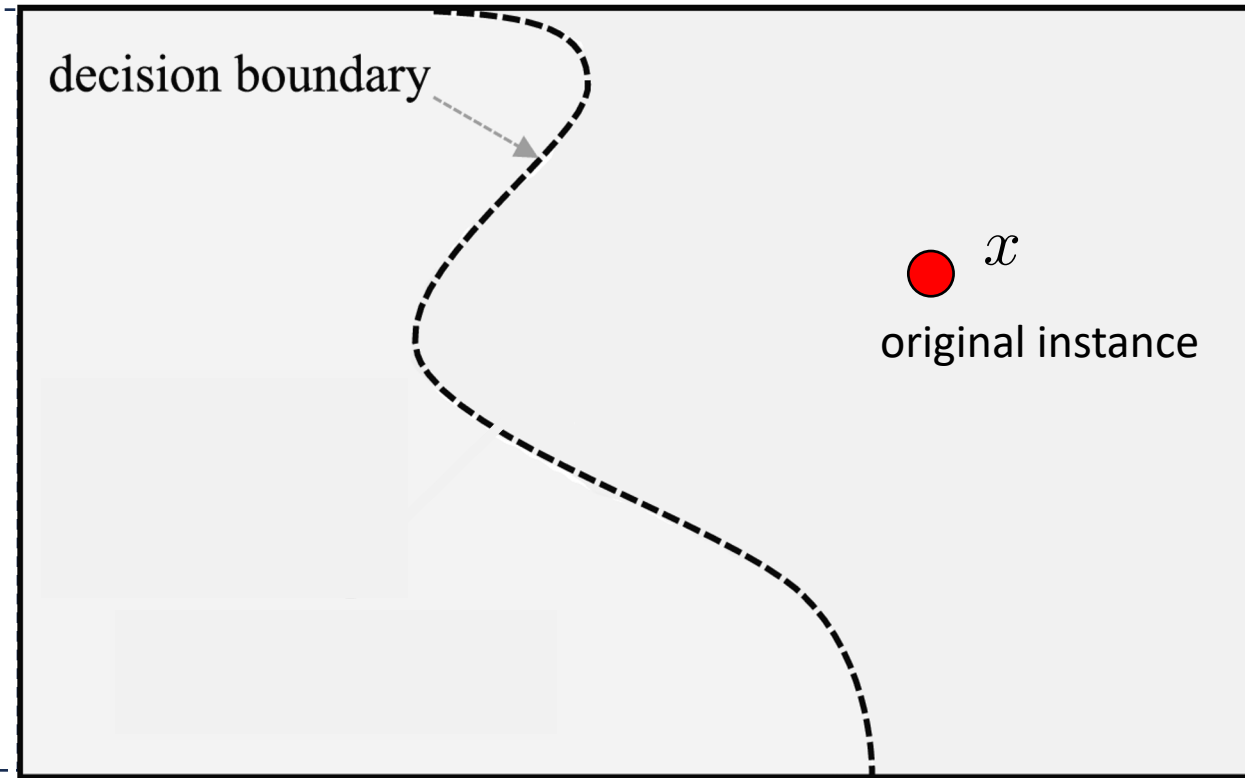
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

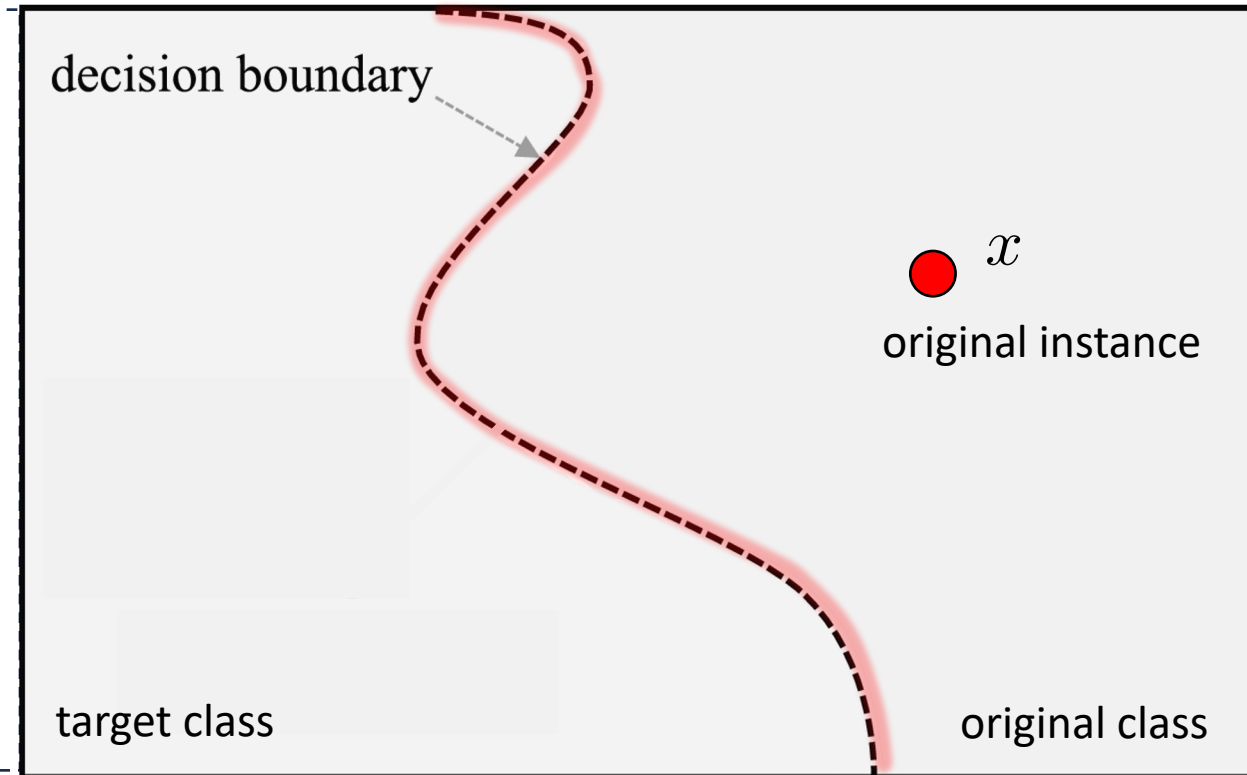
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

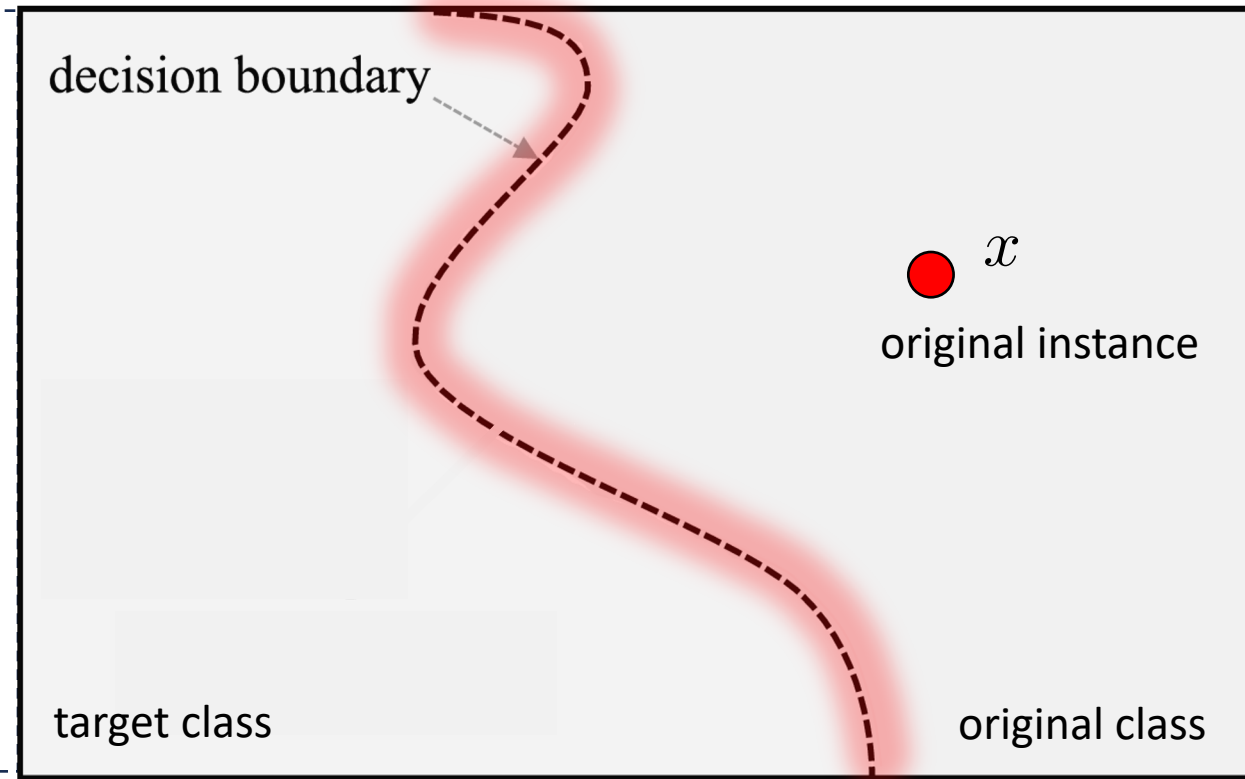
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

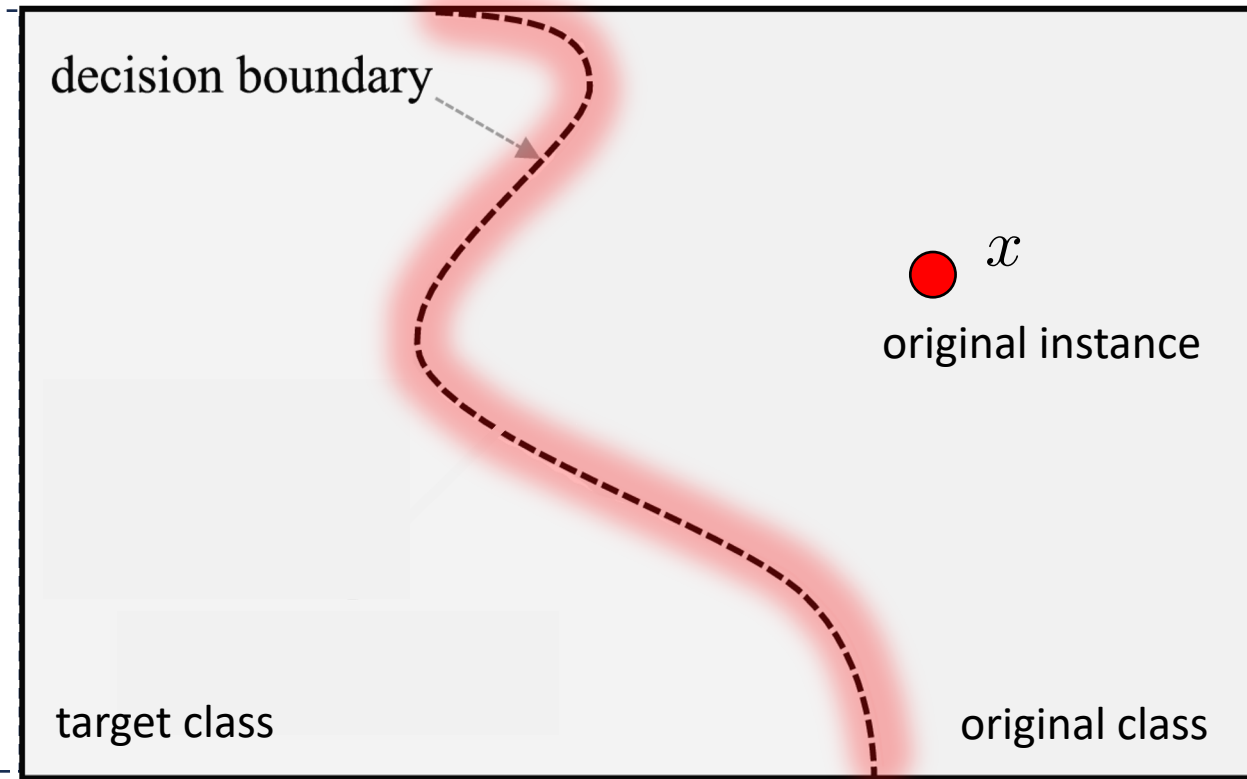
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

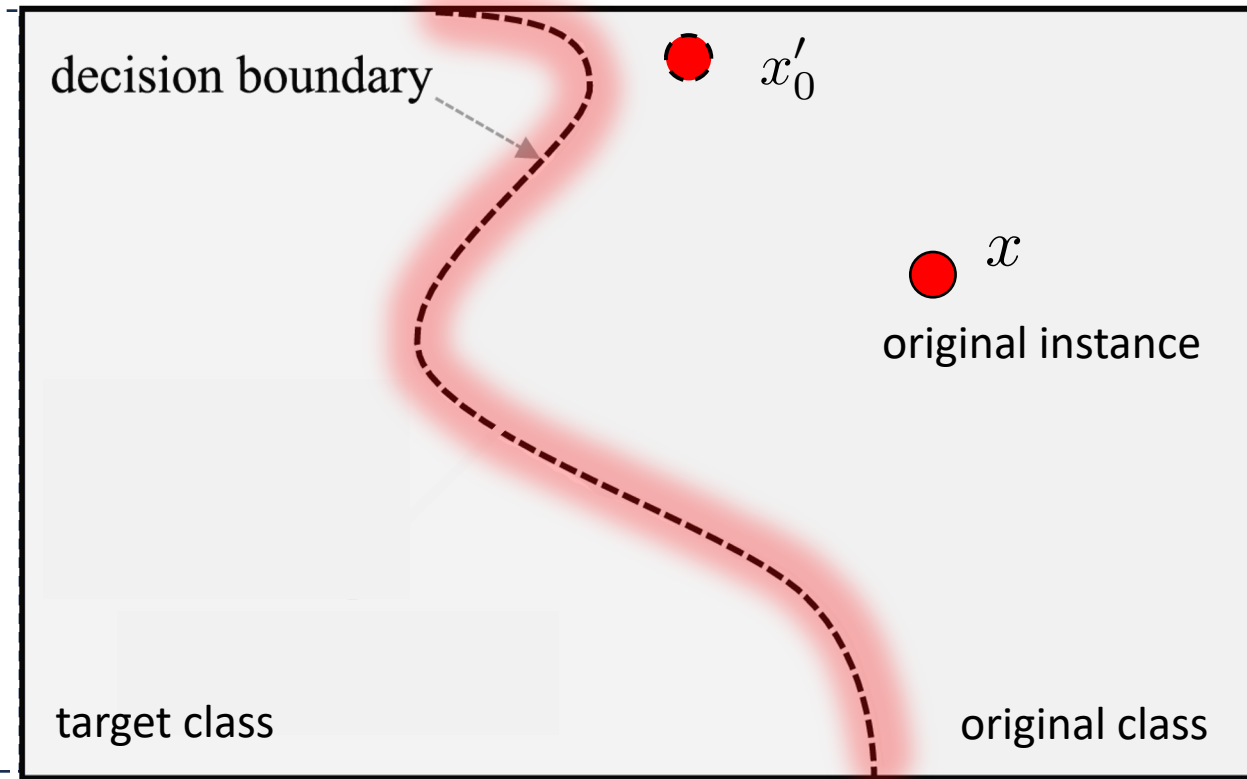
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

Counterfactual Explanations (Wachter et al.)

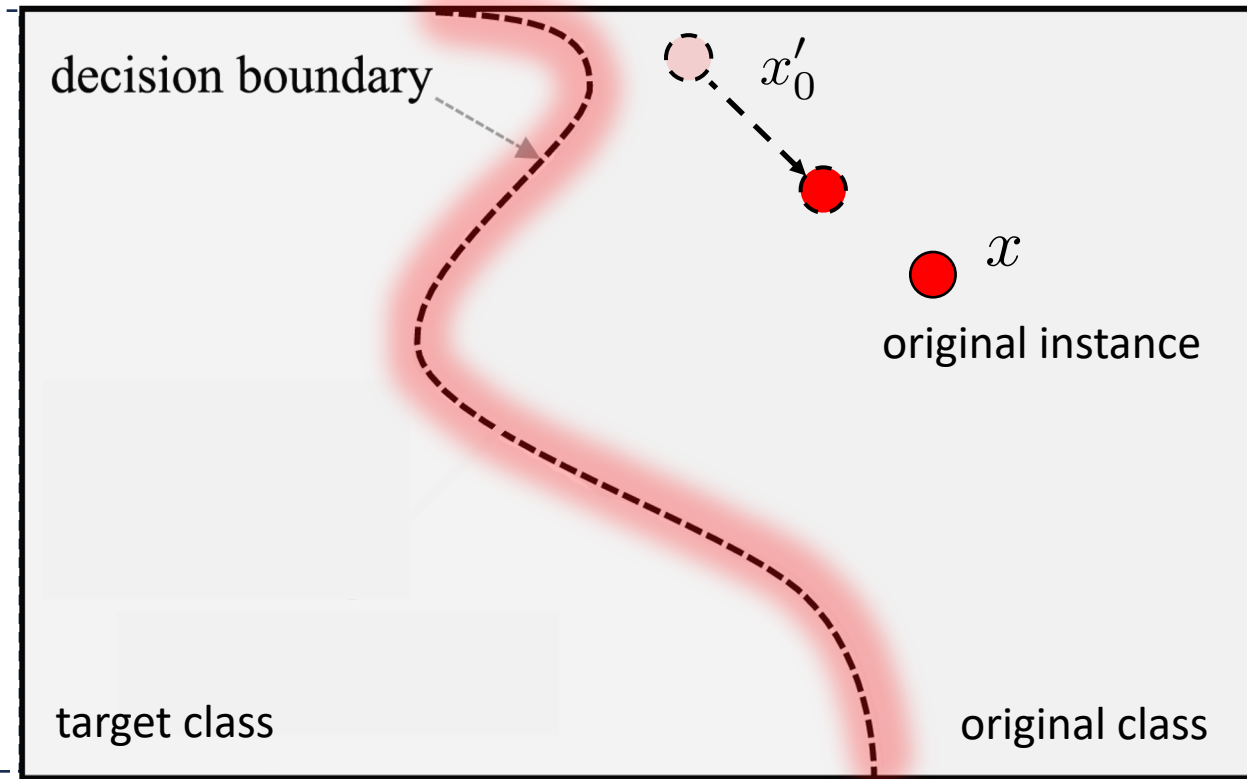
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



In this step because Lambda is small the point is pulled towards the original instance more than the decision boundary.

[]

Counterfactual Explanations (Wachter et al.)

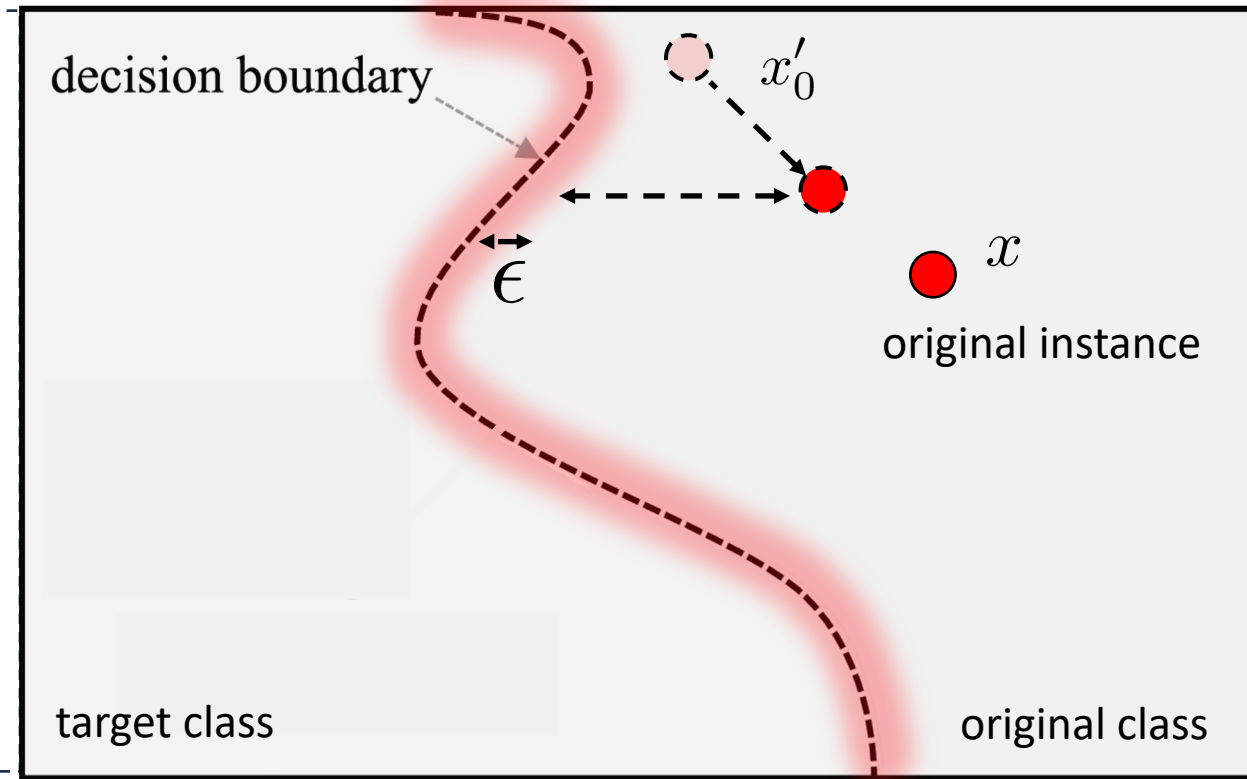
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



[]

When the distance to the target boundary is too far, then enter the loop.

Counterfactual Explanations (Wachter et al.)

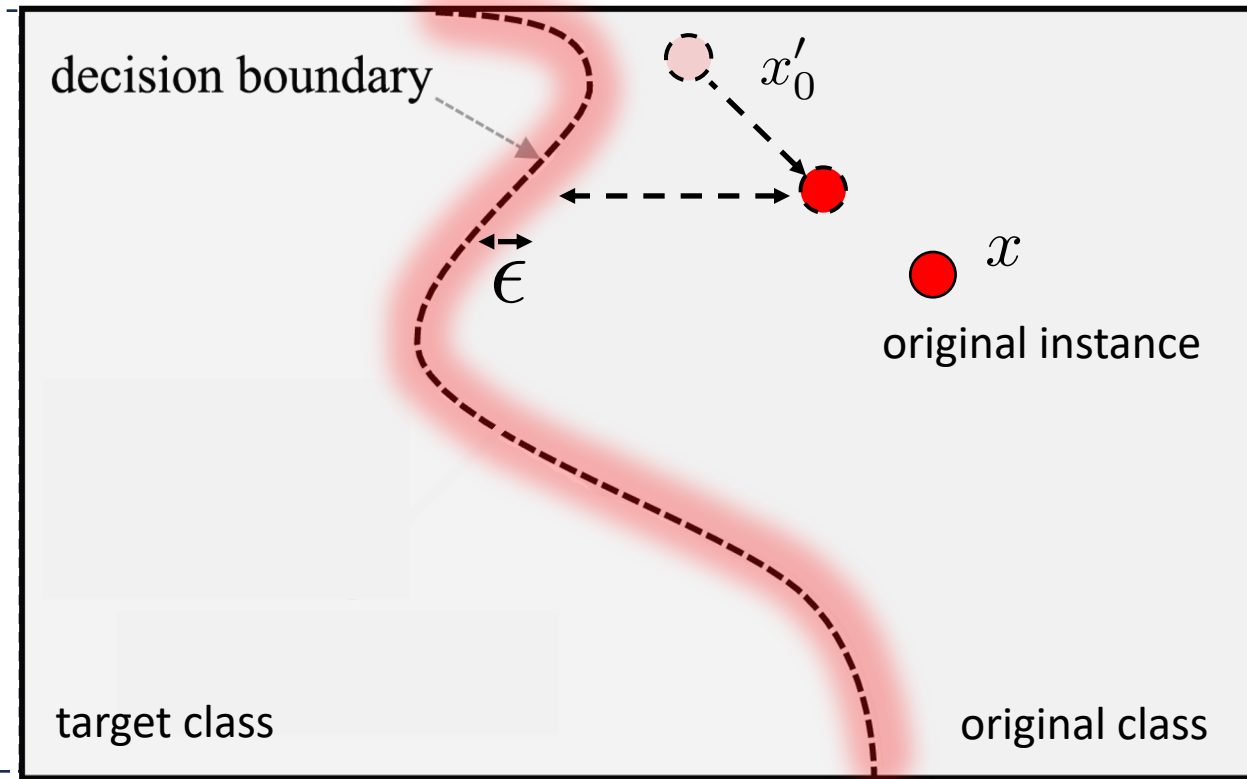
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



We increase Lambda, leading to a reprioritization of distance to the original instance and the target.

[]

Counterfactual Explanations (Wachter et al.)

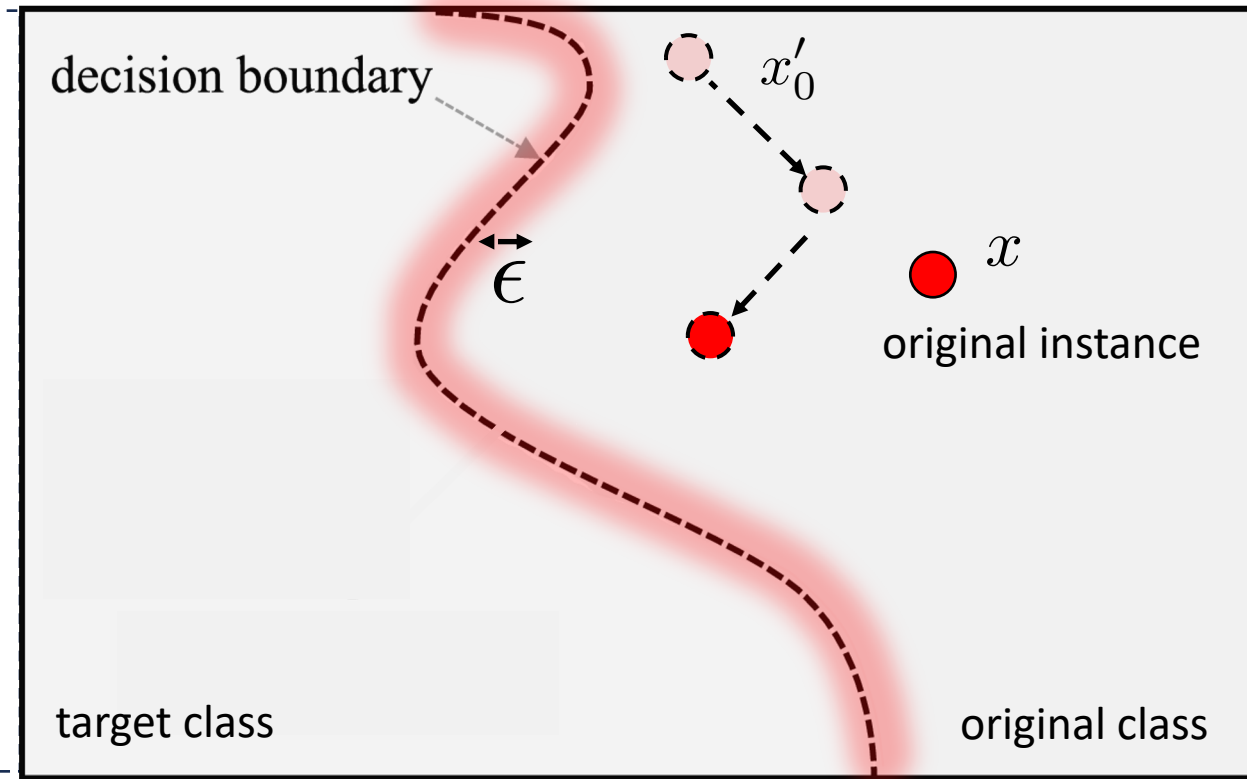
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



In this step The counterfactual is now slowly untethered from the original in instance and heads towards the target decision.

[]

Counterfactual Explanations (Wachter et al.)

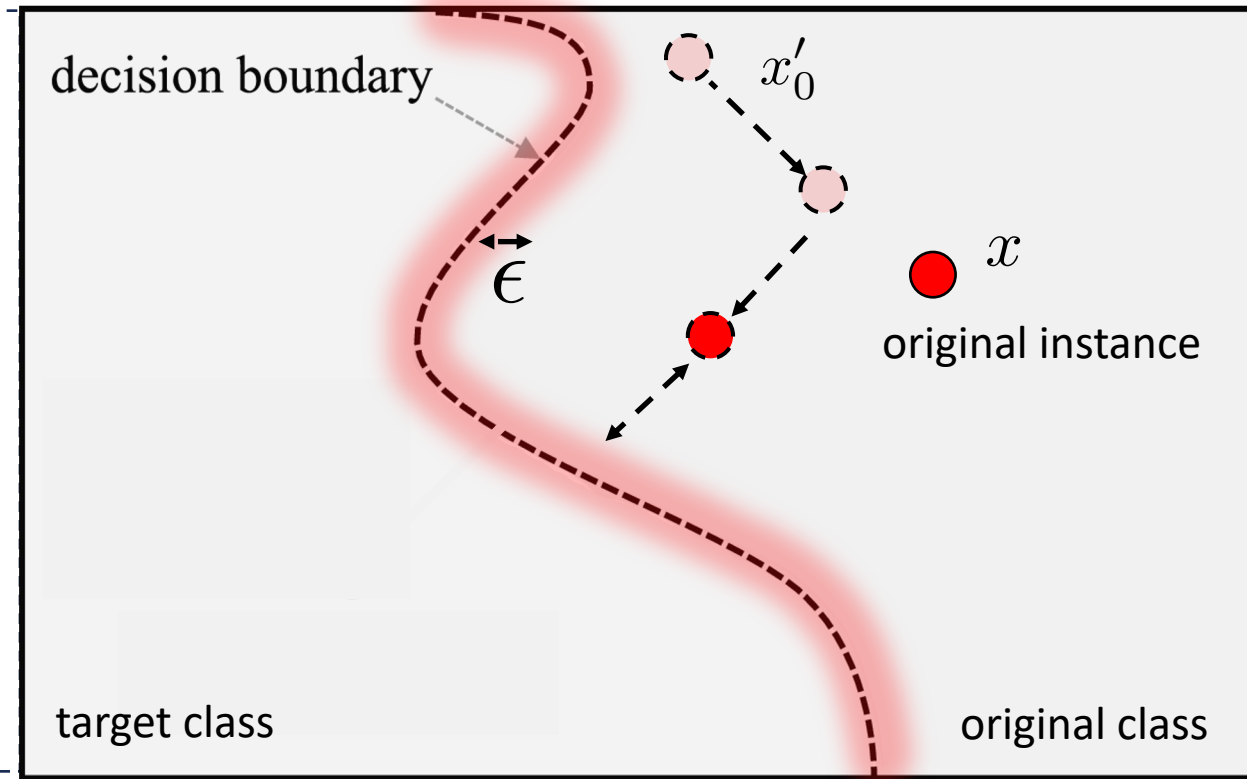
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



If it is still to far away from the targe value then repeat.

[]

Counterfactual Explanations (Wachter et al.)

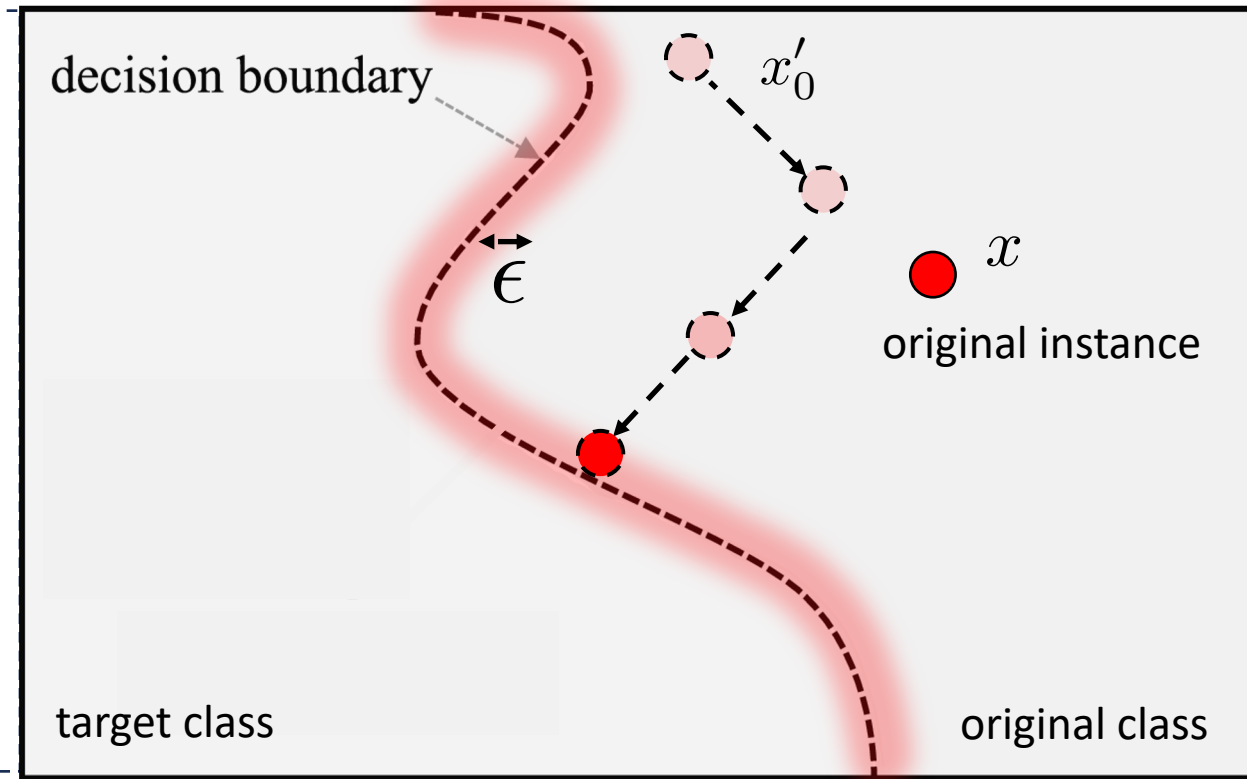
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



Increase Lambda again placing more emphasis on reaching the target rather than being similar to the original instance.

[]

Counterfactual Explanations (Wachter et al.)

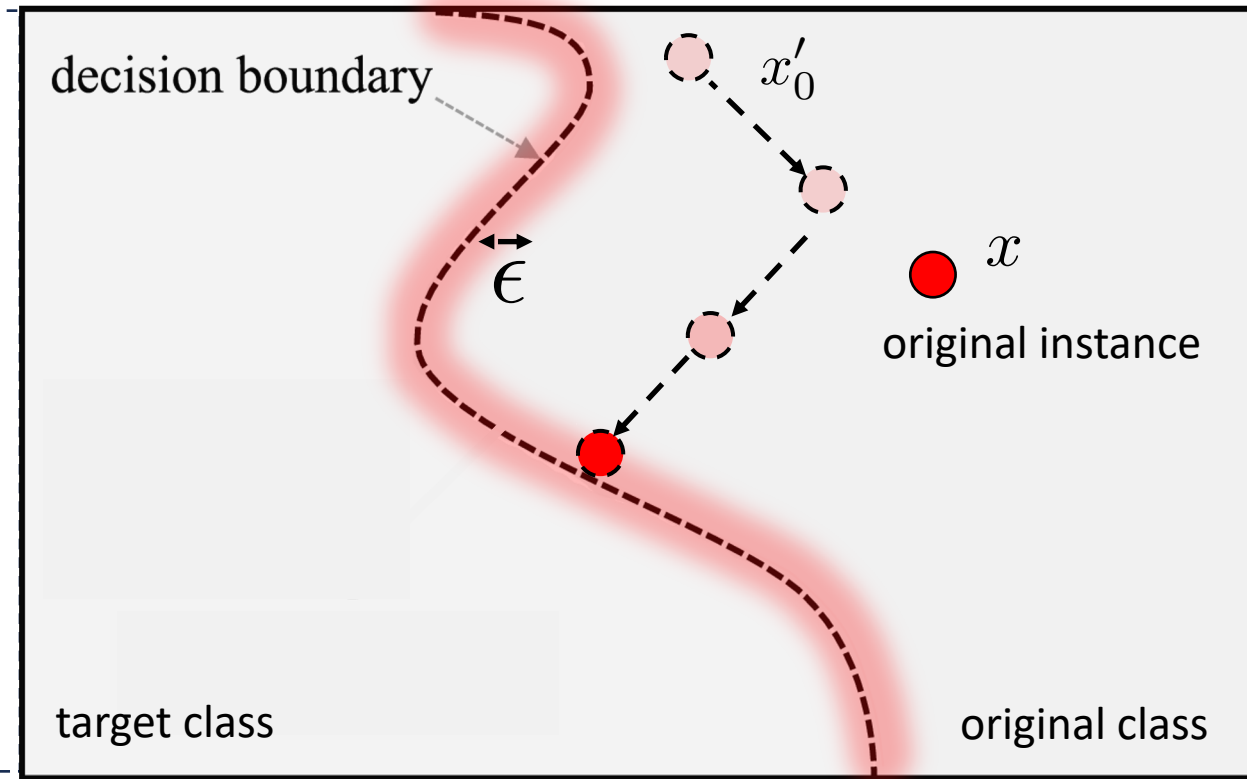
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



Once the target is achieved then store the counterfactual.

Counterfactual Explanations (Wachter et al.)

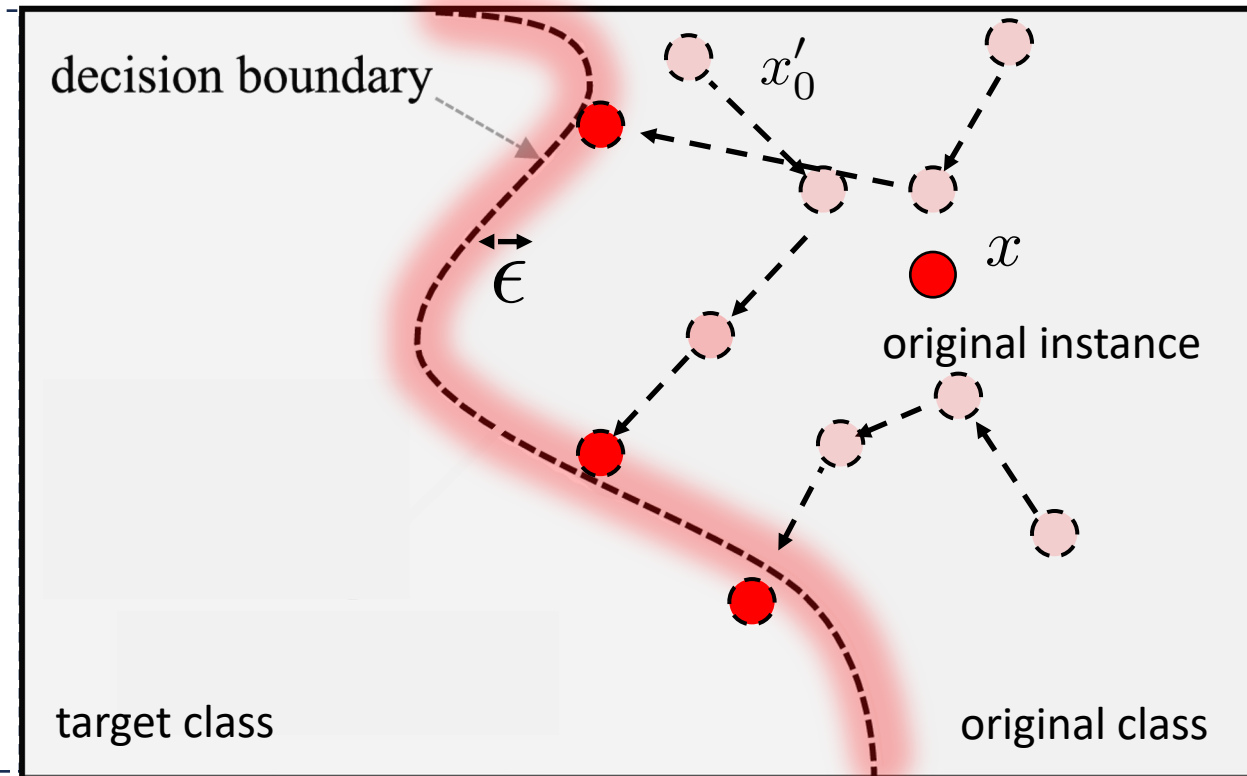
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



Counterfactual Explanations (Wachter et al.)

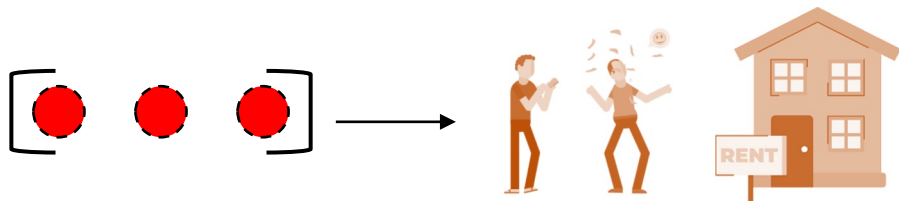
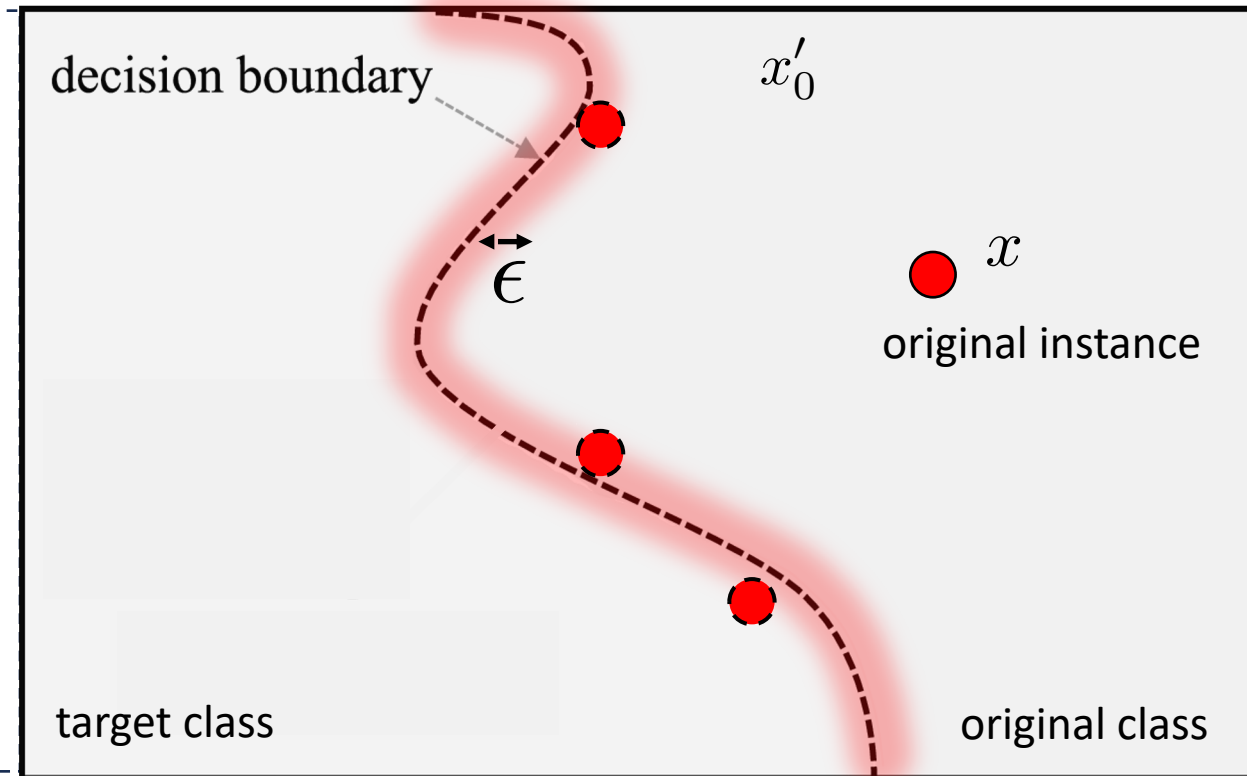
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



Counterfactual Explanations (Wachter et al.)

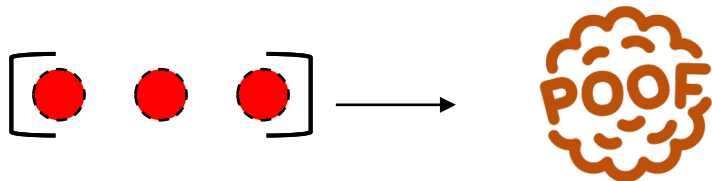
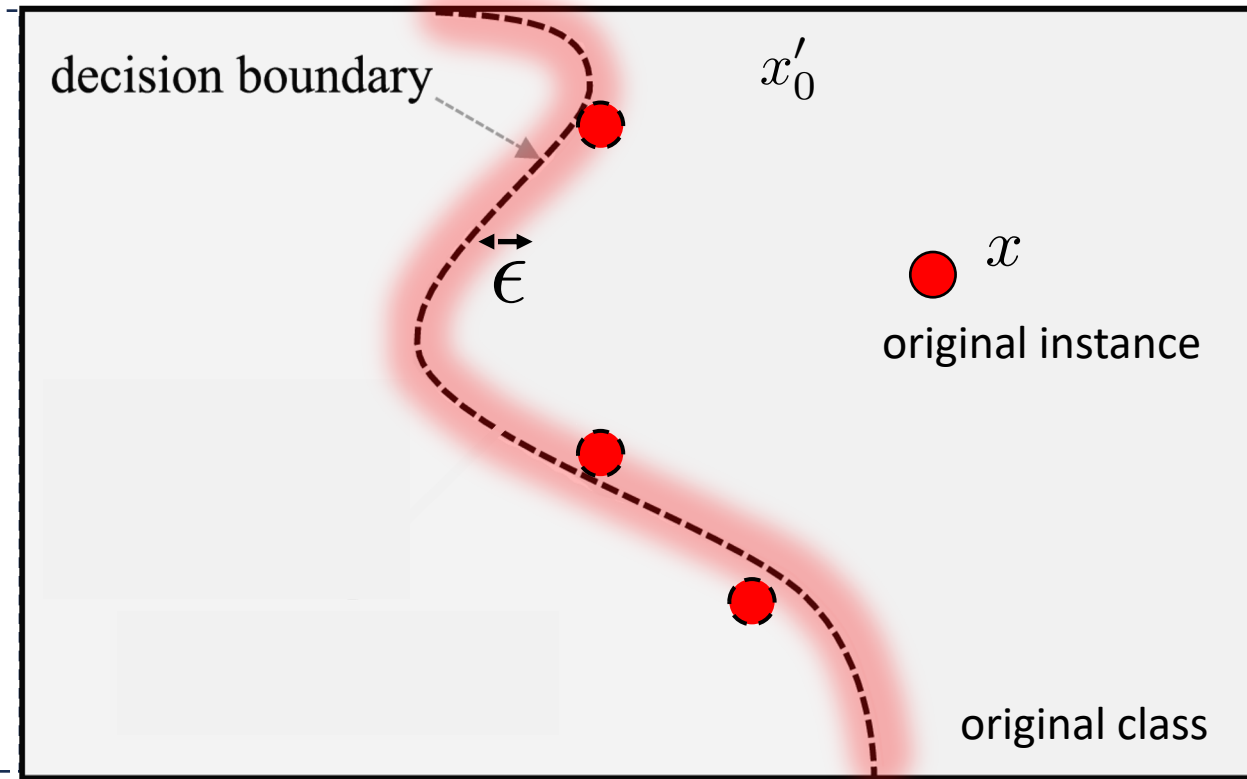
$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

Algorithm: Counterfactual Generation

Input: Instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , initial λ_0

Output: Counterfactual $\mathbf{x}'$ or list of counterfactuals

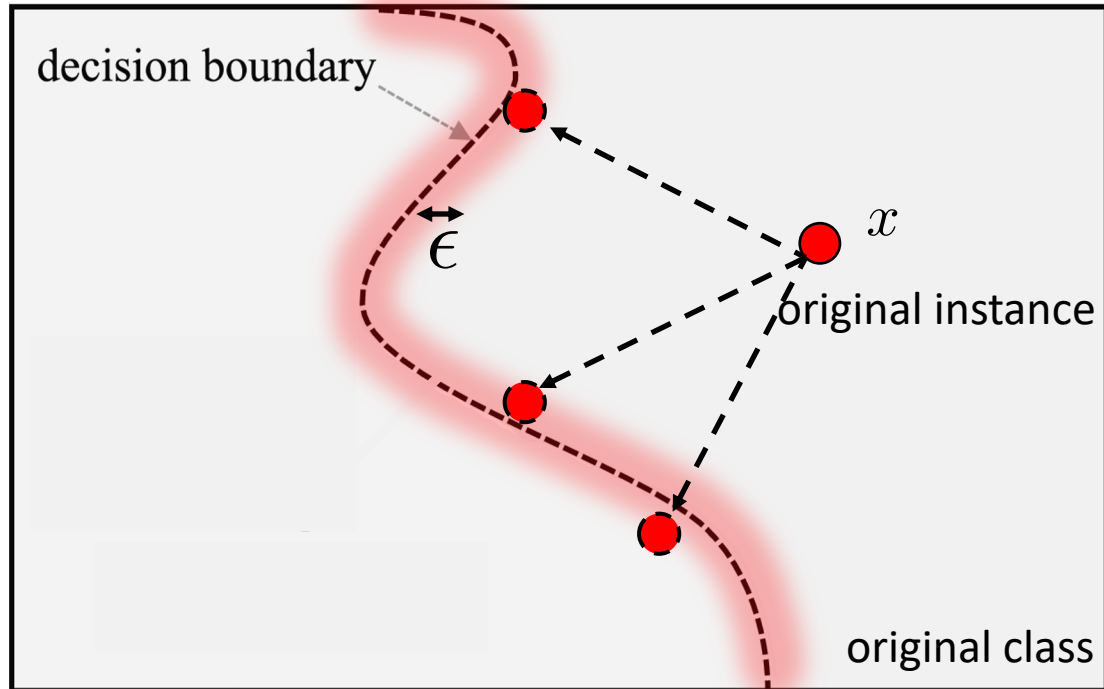
1. Select instance $\mathbf{x}$, desired outcome y' , tolerance ϵ , and initial $\lambda \leftarrow \lambda_0$
2. Sample random instance $\mathbf{x}'_0$ as initial counterfactual
3. Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from $\mathbf{x}'_0$
4. While $|\hat{f}(\mathbf{x}') - y'| > \epsilon$:
 - a) Increase λ (e.g., $\lambda \leftarrow 2\lambda$)
 - b) Optimize: $\mathbf{x}' \leftarrow \arg \min_{\mathbf{x}'} L(\mathbf{x}, \mathbf{x}', y', \lambda)$ starting from current $\mathbf{x}'$
 - c) Store $\mathbf{x}'$ that minimizes $L(\mathbf{x}, \mathbf{x}', y', \lambda)$
5. Repeat steps 2-4 for multiple random initializations
6. Return list of counterfactuals or the one that minimizes the loss



Only takes the first and second criteria into account, not the last two (“produce counterfactuals with only a **few feature changes** and **likely feature values**”). **does not prefer sparse solutions** since increasing 10 features by 1 will give the same distance to as increasing one feature by 10. Unrealistic feature combinations are not penalized.

Counterfactual Explanations (Wachter et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

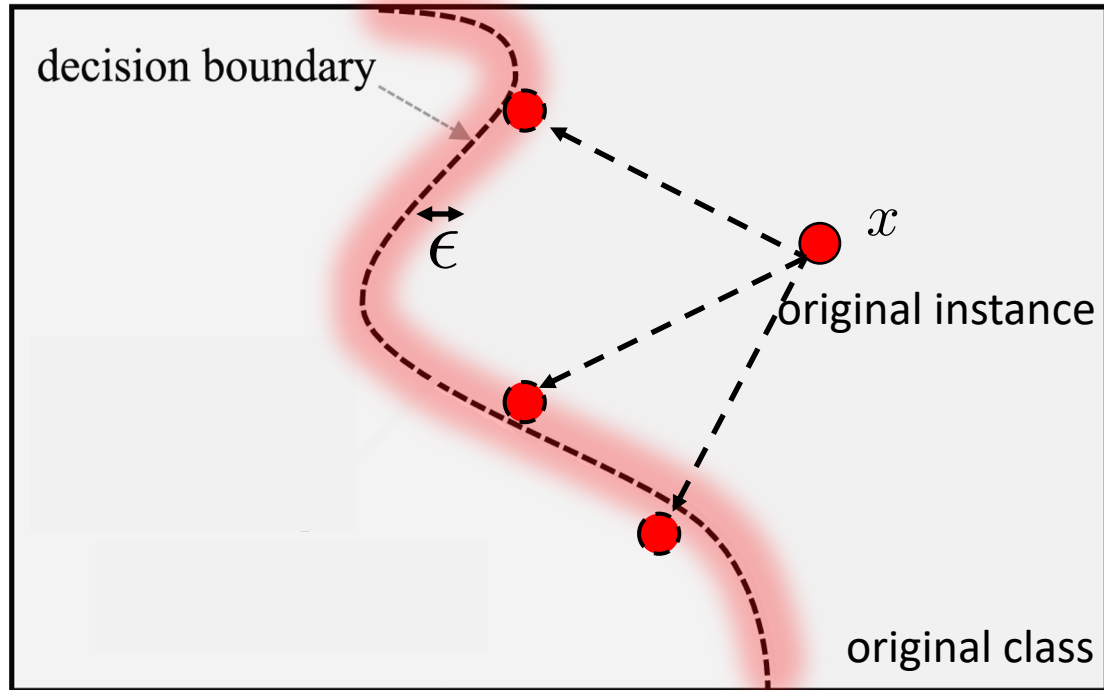


Only takes the first and second criteria into account

- Should **reach the target output** (probabilistically within a small region about it)
- Should be as **similar as possible to the instance** regarding feature values
- Should also **change as few features** as possible.
- Should produce **multiple diverse** counterfactual explanations
- Should have **feature values** that are **likely**.

Counterfactual Explanations (Wachter et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$

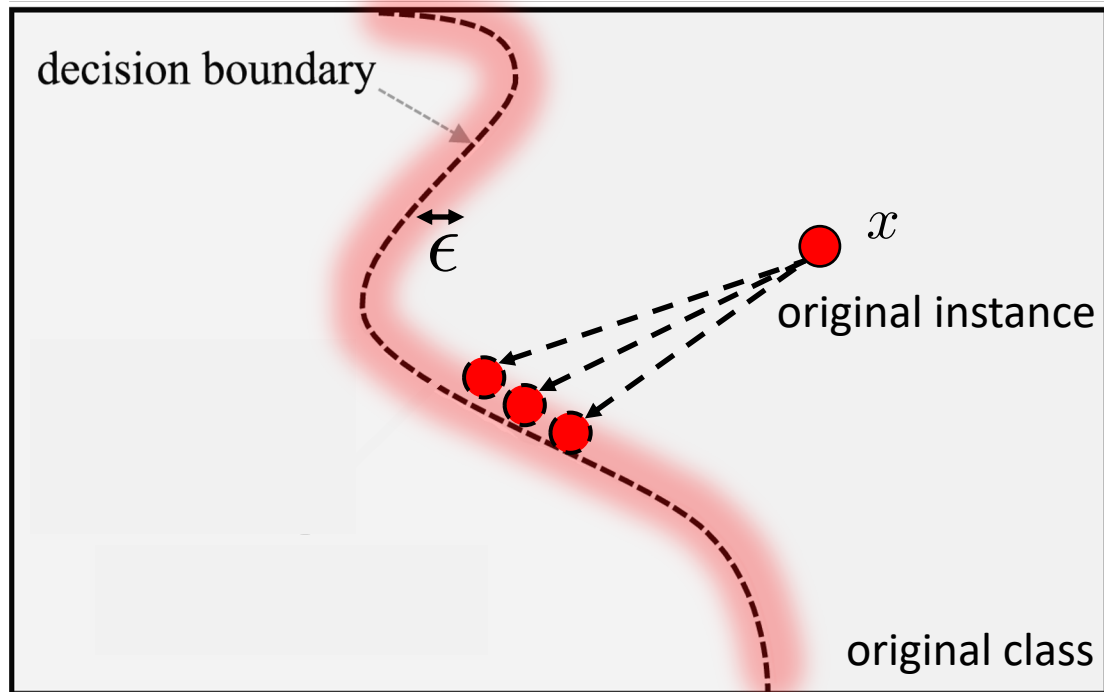


Only takes the first and second criteria into account

- Should **reach the target output** (probabilistically within a small region about it)
- Should be as **similar as possible to the instance** regarding feature values
- Should also **change as few features** as possible.
- Should produce **multiple diverse** counterfactual explanations
- Should have **feature values** that are **likely**.

Counterfactual Explanations (Wachter et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$



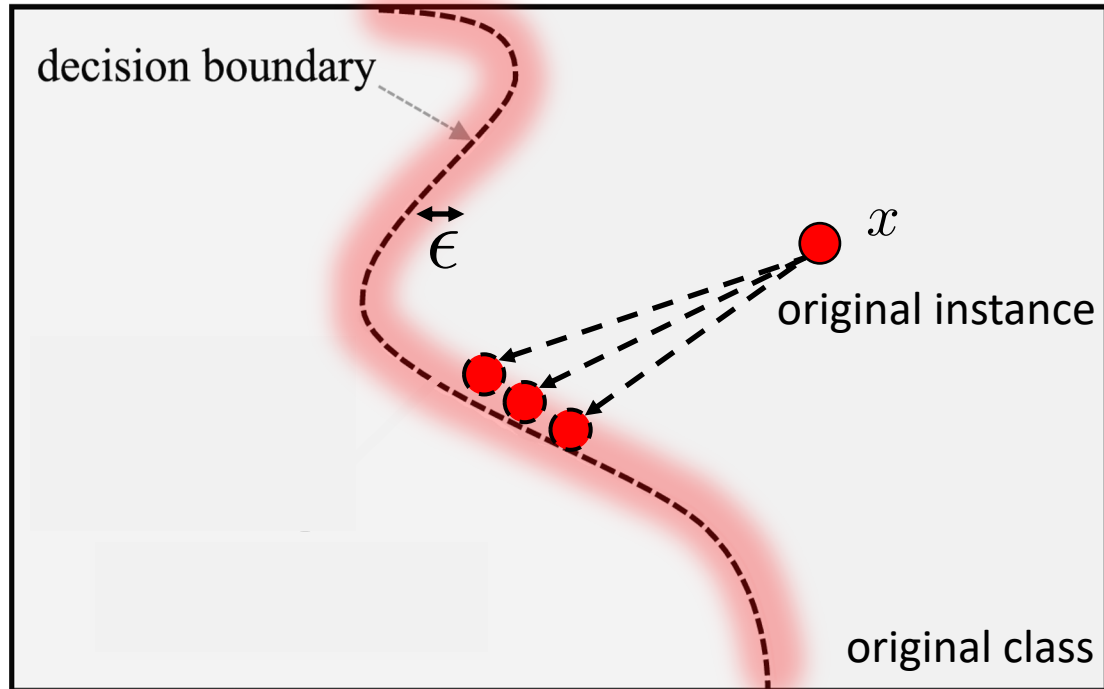
Only takes the first and second criteria into account

- Should **reach the target output** (probabilistically within a small region about it)
- Should be as **similar as possible to the instance** regarding feature values
- Should also **change as few features** as possible.
- Should produce **multiple diverse** counterfactual explanations
- Should have **feature values** that are **likely**.

There was nothing in our formalism that tried to generate diverse results.
We could therefore end up with a selection of very similar counterfactuals.

Counterfactual Explanations (Wachter et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot (\hat{f}(\mathbf{x}') - y')^2 + d(\mathbf{x}, \mathbf{x}')$$



Only takes the first and second criteria into account

- Should **reach the target output** (probabilistically within a small region about it)
- Should be as **similar as possible to the instance** regarding feature values
- Should also **change as few features** as possible.
- Should produce **multiple diverse** counterfactual explanations
- Should have **feature values** that are **likely**.

Counterfactual Explanations (Dandl et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \mathbf{X}^{\text{obs}}) = (o_1(\hat{f}(\mathbf{x}'), y'), o_2(\mathbf{x}, \mathbf{x}'), o_3(\mathbf{x}, \mathbf{x}'), o_4(\mathbf{x}', \mathbf{X}^{\text{obs}}))$$

O_1 Should be as close as possible to the desired prediction.
Manhattan metric

O_2 Should be as close to the original instance
Gower distance

O_3 Counts how many features were changed.
L0 Norm

O_4 Measures the likelihood of the point
Average Gower distance to closest real data points

We do not want to collapse the four objectives into a single objective by summing them up and weighting them, but we want to optimize all four terms simultaneously.

Counterfactual Explanations (Dandl et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \mathbf{X}^{\text{obs}}) = (o_1(\hat{f}(\mathbf{x}'), y'), o_2(\mathbf{x}, \mathbf{x}'), o_3(\mathbf{x}, \mathbf{x}'), o_4(\mathbf{x}', \mathbf{X}^{\text{obs}}))$$

O_1 Should be as close as possible to the desired prediction.
Manhattan metric

O_2 Should be as close to the original instance
Gower distance

O_3 Counts how many features were changed.
L0 Norm

O_4 Measures the likelihood of the point
Average Gower distance to closest real data points

We do not want to collapse the four objectives into a single objective by summing them up and weighting them, but we want to optimize all four terms simultaneously.

Counterfactual Explanations (Dandl et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \mathbf{X}^{\text{obs}}) = (o_1(\hat{f}(\mathbf{x}'), y'), o_2(\mathbf{x}, \mathbf{x}'), o_3(\mathbf{x}, \mathbf{x}'), o_4(\mathbf{x}', \mathbf{X}^{\text{obs}}))$$

O_1 Should be as close as possible to the desired prediction.
Manhattan metric

O_2 Should be as close to the original instance
Gower distance

O_3 Counts how many features were changed.
L0 Norm

O_4 Measures the likelihood of the point
Average Gower distance to closest real data points

We do not want to collapse the four objectives into a single objective by summing them up and weighting them, but we want to optimize all four terms simultaneously.

Counterfactual Explanations (Dandl et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \mathbf{X}^{\text{obs}}) = (o_1(\hat{f}(\mathbf{x}'), y'), o_2(\mathbf{x}, \mathbf{x}'), o_3(\mathbf{x}, \mathbf{x}'), o_4(\mathbf{x}', \mathbf{X}^{\text{obs}}))$$

O_1 Should be as close as possible to the desired prediction.
Manhattan metric

O_2 Should be as close to the original instance
Gower distance

O_3 Counts how many features were changed.
L0 Norm

O_4 Measures the likelihood of the point
Average Gower distance to closest real data points

We do not want to collapse the four objectives into a single objective by summing them up and weighting them, but we want to optimize all four terms simultaneously.

Counterfactual Explanations (Dandl et al.)

$$L(\mathbf{x}, \mathbf{x}', y', \mathbf{X}^{\text{obs}}) = (o_1(\hat{f}(\mathbf{x}'), y'), o_2(\mathbf{x}, \mathbf{x}'), o_3(\mathbf{x}, \mathbf{x}'), o_4(\mathbf{x}', \mathbf{X}^{\text{obs}}))$$

O_1 Should be as close as possible to the desired prediction.
Manhattan metric

O_2 Should be as close to the original instance
Gower distance

O_3 Counts how many features were changed.
L0 Norm

O_4 Measures the likelihood of the point
Average Gower distance to closest real data points

We do not want to collapse the four objectives into a single objective by summing them up and weighting them, but we want to optimize all four terms simultaneously.

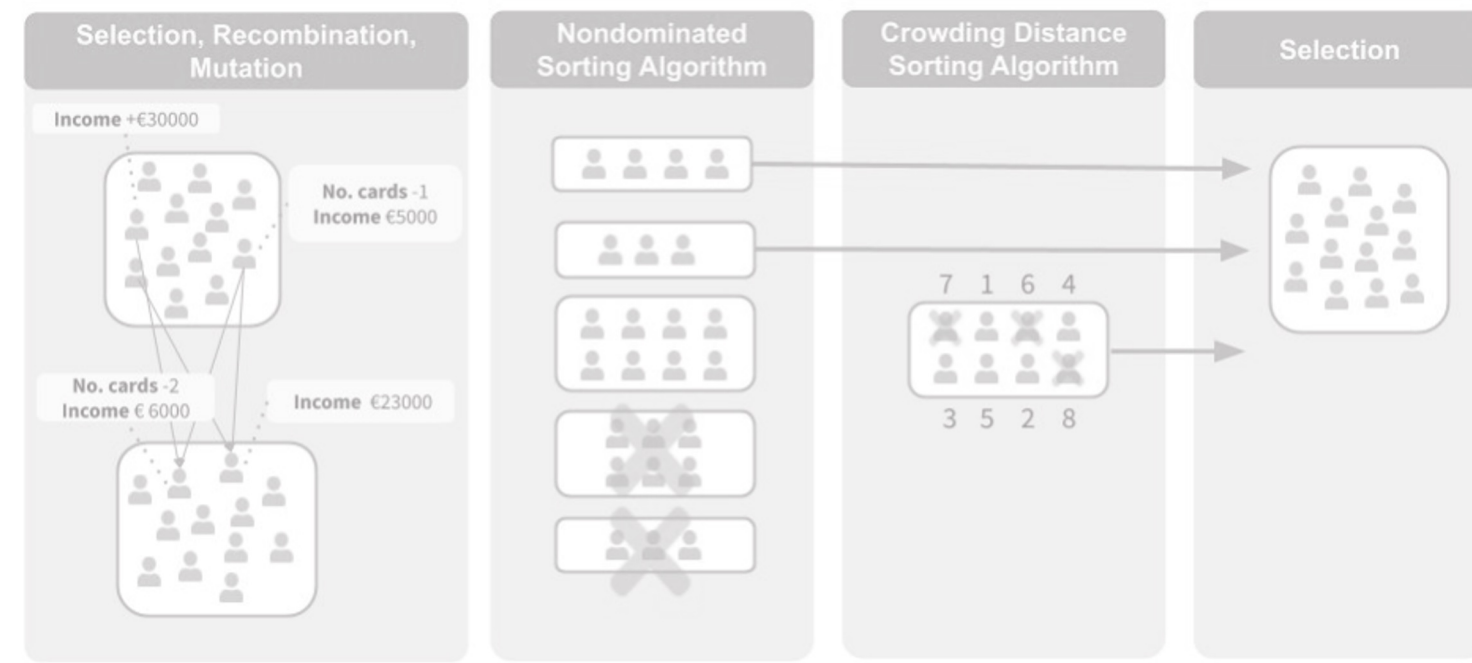
Counterfactual Explanations (Dandl et al.)

Nondominated Sorting Genetic Algorithm

Nature-inspired algorithm that applies Darwin's law of the "survival of the fittest"

$(O_1, O_2, O_3, O_4) \longrightarrow$ Fitness function (smaller is better)

Population of counterfactuals



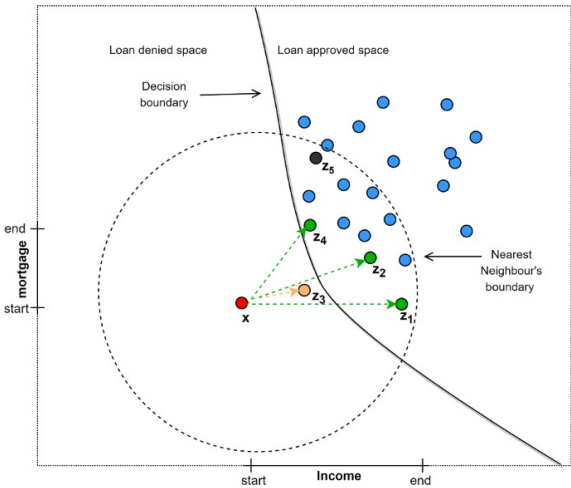
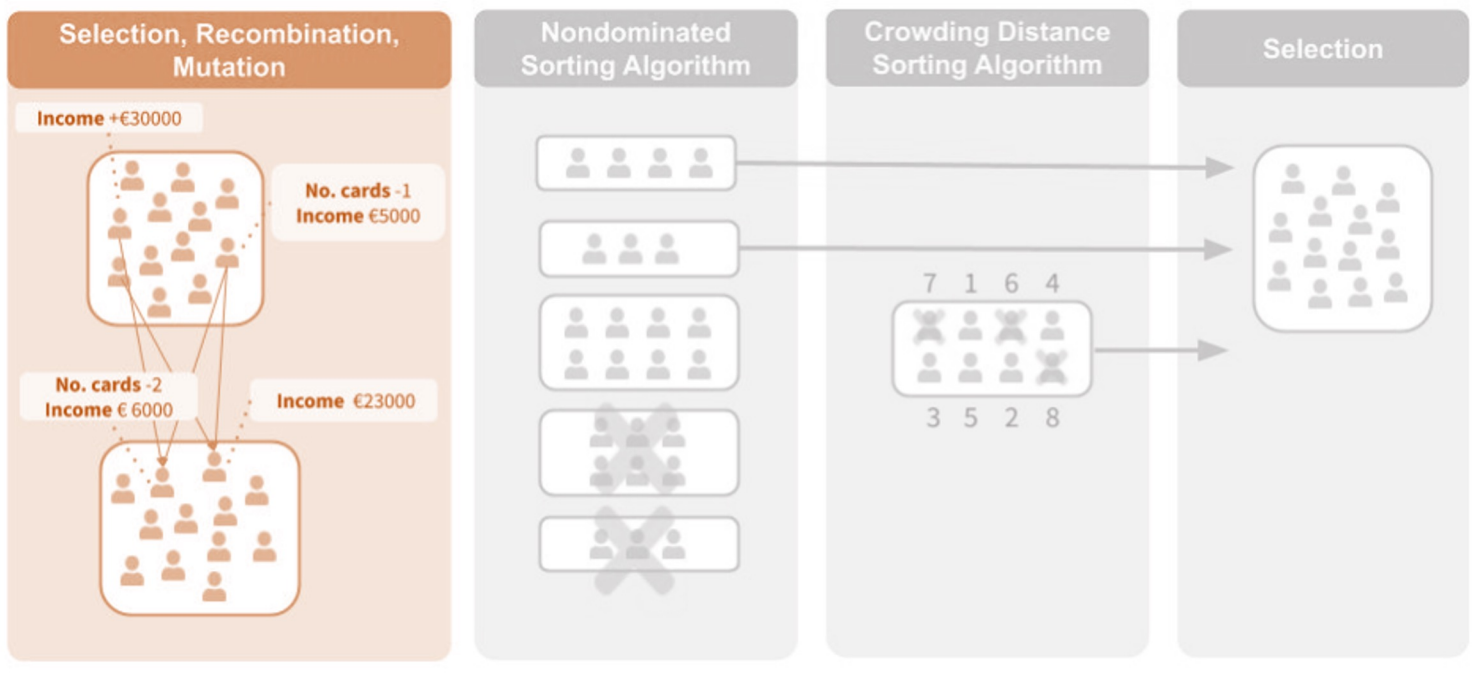
Counterfactual Explanations (Dandl et al.)

Nondominated Sorting Genetic Algorithm

Nature-inspired algorithm that applies Darwin’s law of the “survival of the fittest”

$(O_1, O_2, O_3, O_4) \longrightarrow$ Fitness function (smaller is better)

Population of counterfactuals



Step 1. Selection, Recombination, Mutation

We want solutions that are fit. Like in biology, we create new “children” solutions by:

- **Selection:** picking parents with good performance.
- **Recombination:** mixing parts of two parents.
- **Mutation:** making small random changes.

This gives us new candidate solutions with different features.

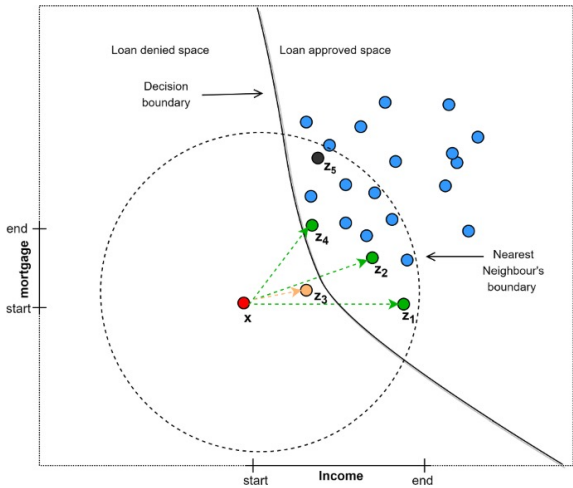
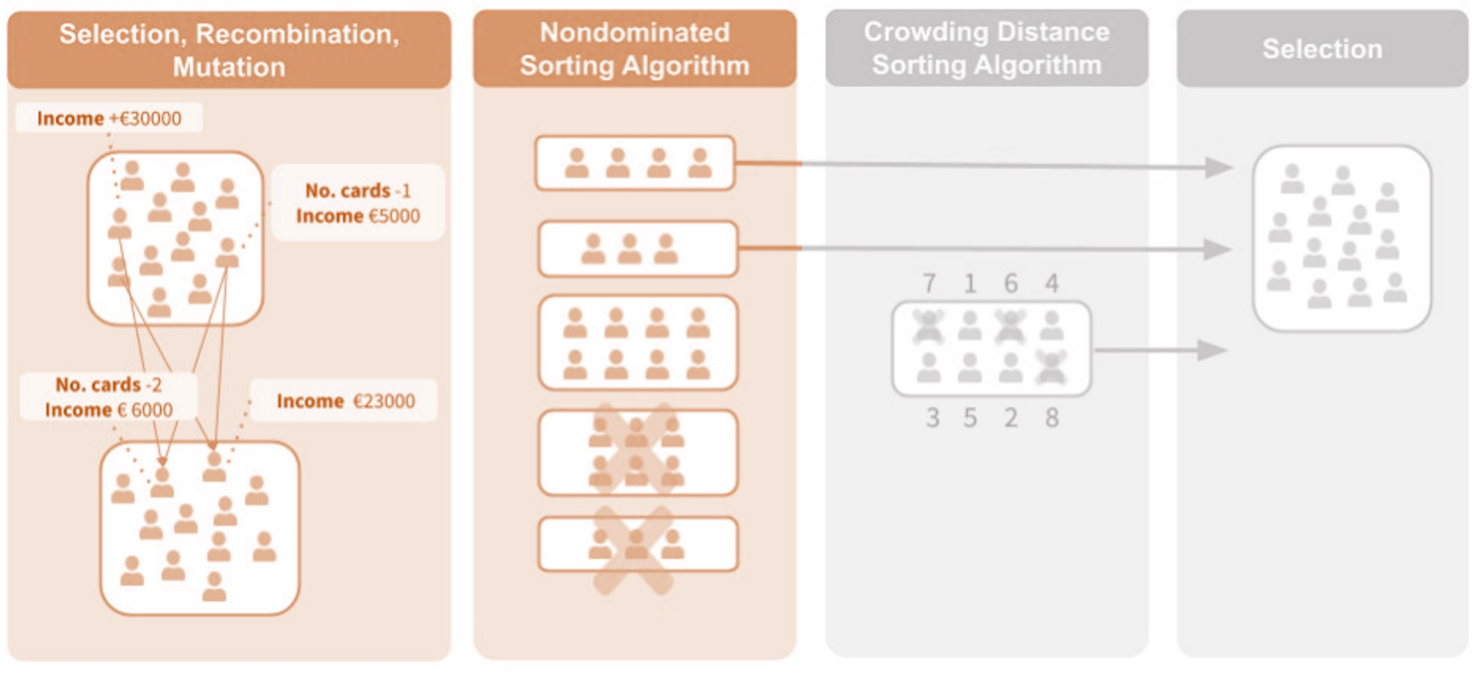
Counterfactual Explanations (Dandl et al.)

Nondominated Sorting Genetic Algorithm

Nature-inspired algorithm that applies Darwin’s law of the “survival of the fittest”

$(O_1, O_2, O_3, O_4) \longrightarrow$ Fitness function (smaller is better)

Population of counterfactuals



Step 2. Nondominated Sorting Algorithm

Now we need to rank the solutions.

Think of it as a race with multiple tracks.

A solution is considered better if there is no other solution that is strictly better all the tracks.

These “best trade-offs” are grouped together in the first row (Front 1). Once we remove those, the next best group forms Front 2, and so on.

So: Front 1 = top solutions nobody beats on all counts.

Front 2 = good, but beaten by someone in Front 1 etc..

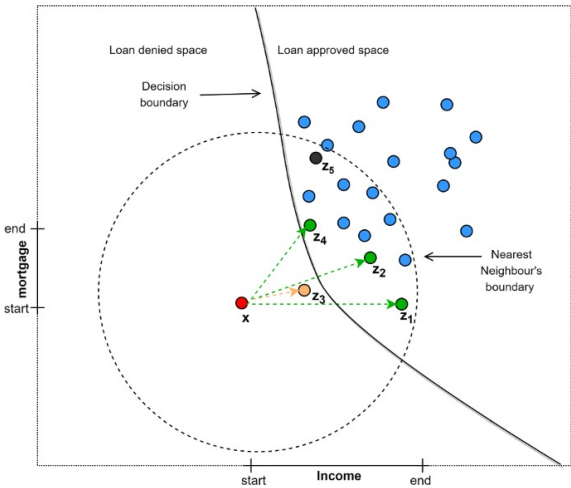
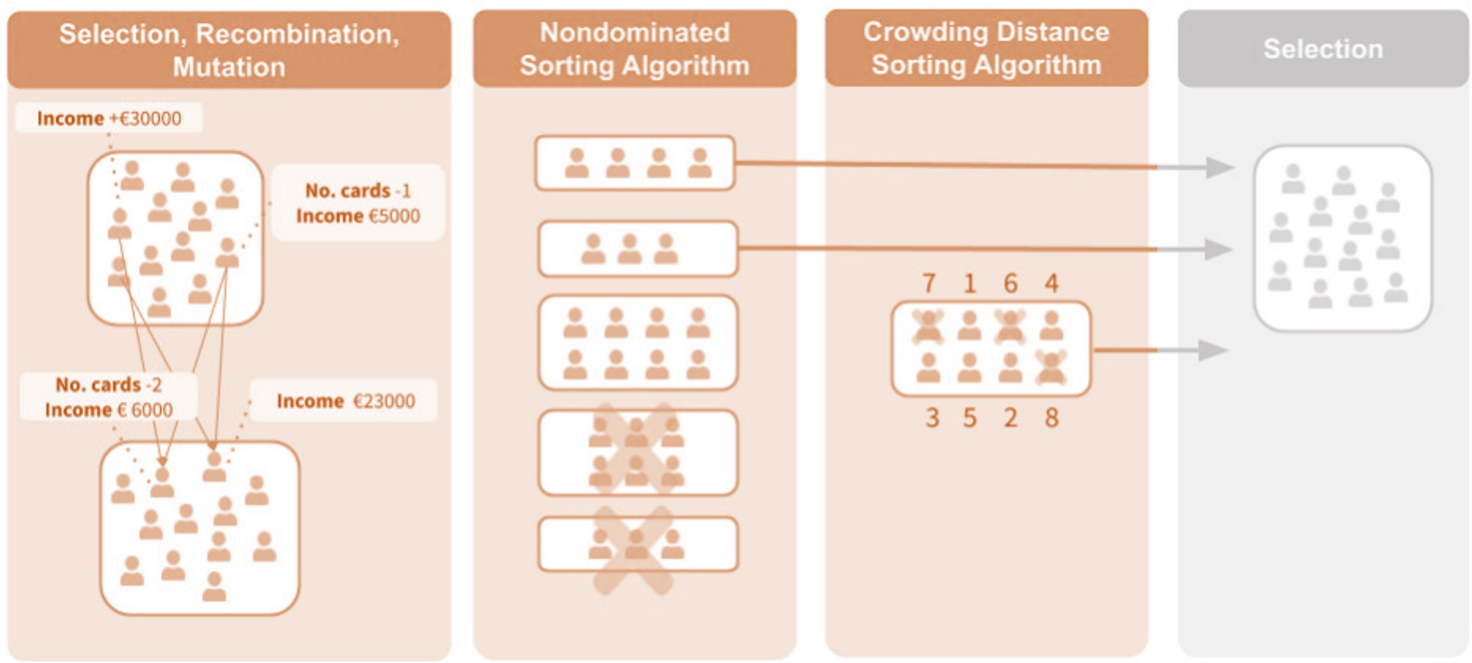
Counterfactual Explanations (Dandl et al.)

Nondominated Sorting Genetic Algorithm

Nature-inspired algorithm that applies Darwin’s law of the “survival of the fittest”

$(O_1, O_2, O_3, O_4) \longrightarrow$ Fitness function (smaller is better)

Population of counterfactuals



Step 3. Crowding Distance Sorting Algorithm

Within each group, some solutions might be very similar. To avoid keeping lots of nearly identical ones, we measure how close solutions are to each other. Imagine placing them on a line: if two are very close, one may be removed. Solutions with more space around them (more unique trade-offs) are preferred. This keeps the set of solutions diverse, covering a wide range of options.

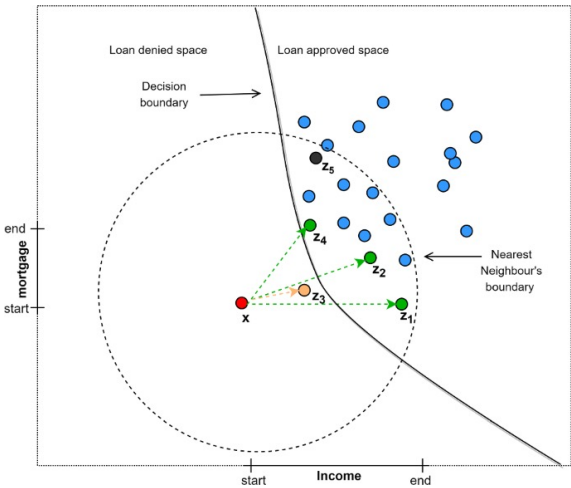
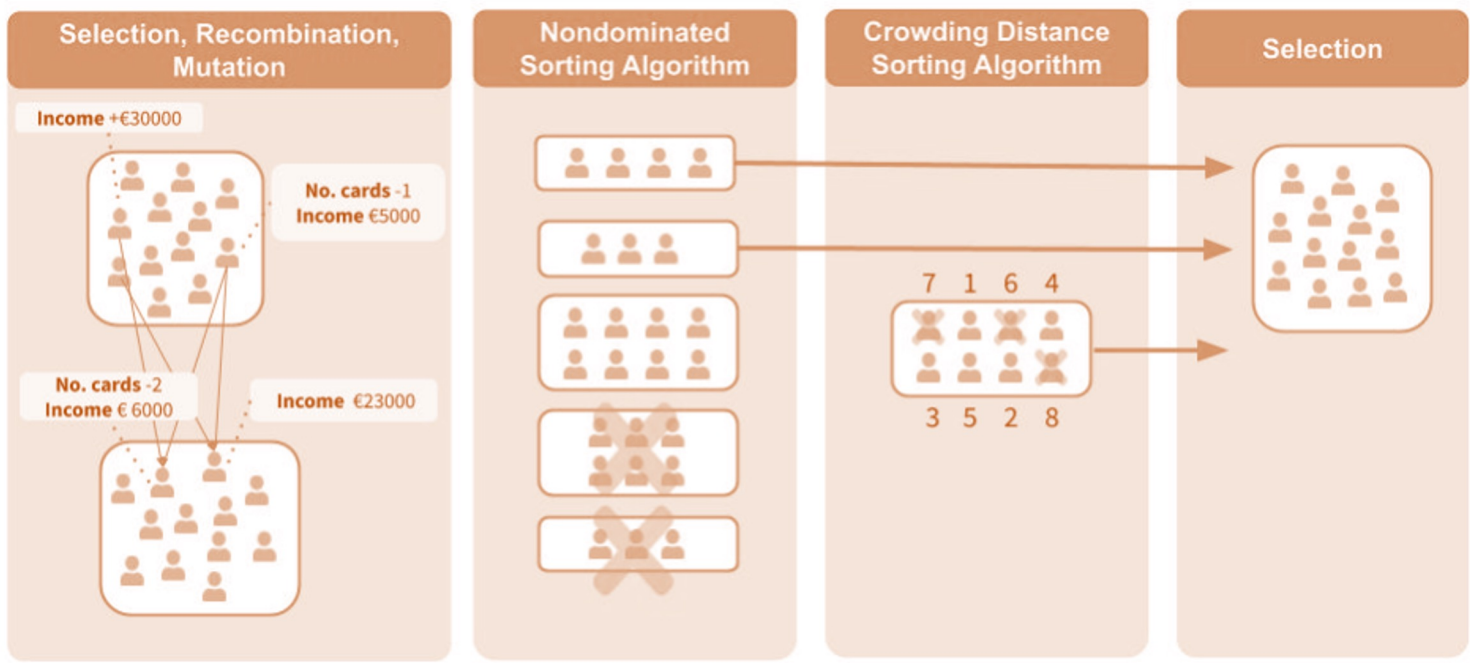
Counterfactual Explanations (Dandl et al.)

Nondominated Sorting Genetic Algorithm

Nature-inspired algorithm that applies Darwin’s law of the “survival of the fittest”

$(O_1, O_2, O_3, O_4) \longrightarrow$ Fitness function (smaller is better)

Population of counterfactuals



Step 4. Selection

Finally, we select the solutions that survive to the next generation. We keep: The best groups (Front1, then Front2). The most diverse solutions inside each group. These become the new “parents” for the next round. Over time, this process moves the population closer and closer to the best possible trade-off curve (called the Pareto front).



Counterfactual Explanations (+ / -)

Strengths

- Interpretation of counterfactual explanations is very clear
(Think of LIME, here we are not sure how safe it is to interpolate our result)
- Counterfactuals do not require access to the data or the model
(just require the model's predictive function, allows companies to protect their trade secrets, models, and data while still offering explanations)
- Works with systems that do not use machine learning
(Could be a set of hand written rules / heuristics evolutionary algorithms also used in neuromorphic ML)

Limitations

- Hyper local
(Specific to one instance and one counterfactual, meaning not useful for extracting model and data insights)

Counterfactual Explanations (Examples)

The German Credit Risk dataset, based on the credit dataset example in Dandl et al. (2020)

Trained an **SVM** to predict the probability that a person has a **good credit risk**.

Model predicted a good credit risk of 24.2 %

Table 15.1: Feature values of a particular customer

age	sex	job	housing	savings	amount	dur.	purpose
58	f	unskilled	free	little	6143	48	car

Counterfactual algorithm to find top 10 best counterfactuals to get the good risk to 50%

Table 15.2: The ten best counterfactuals found for the customer

age	sex	job	amount	dur.	o ₂	o ₃	o ₄	f(x')
		skilled		-20	0.108	2	0.036	0.501
		skilled		-24	0.114	2	0.029	0.525
		skilled		-22	0.111	2	0.033	0.513
-6		skilled		-24	0.126	3	0.018	0.505
-3		skilled		-24	0.120	3	0.024	0.515
-1		skilled		-24	0.116	3	0.027	0.522
-3	m			-24	0.195	3	0.012	0.501
-6	m			-25	0.202	3	0.011	0.501
-30	m	skilled		-24	0.285	4	0.005	0.590
-4	m		-1254	-24	0.204	4	0.002	0.506

All counterfactuals have predicted probabilities **greater than 50%**

Do **not dominate** each other, i.e., no counterfactual wins on O value

All suggest a **reduction** of the duration from 48 months to a minimum of 23

Some counterfactuals suggest **changing** the **gender** from female to male, which shows a gender **bias** of the model

Counterfactual Explanations (Examples)

The German Credit Risk dataset, based on the credit dataset example in Dandl et al. (2020)

Trained an **SVM** to predict the probability that a person has a **good credit risk**.

Model predicted a good credit risk of 24.2 %

Table 15.1: Feature values of a particular customer

age	sex	job	housing	savings	amount	dur.	purpose
58	f	unskilled	free	little	6143	48	car

Counterfactual algorithm to find top 10 best counterfactuals to get the good risk to 50%

Table 15.2: The ten best counterfactuals found for the customer

age	sex	job	amount	dur.	o ₂	o ₃	o ₄	f(x')
		skilled		-20	0.108	2	0.036	0.501
		skilled		-24	0.114	2	0.029	0.525
		skilled		-22	0.111	2	0.033	0.513
-6		skilled		-24	0.126	3	0.018	0.505
-3		skilled		-24	0.120	3	0.024	0.515
-1		skilled		-24	0.116	3	0.027	0.522
-3	m			-24	0.195	3	0.012	0.501
-6	m			-25	0.202	3	0.011	0.501
-30	m	skilled		-24	0.285	4	0.005	0.590
-4	m		-1254	-24	0.204	4	0.002	0.506

All counterfactuals have predicted probabilities **greater than 50%**

Do **not dominate** each other, i.e., no counterfactual wins on O value

All suggest a **reduction** of the duration from 48 months to a minimum of 23

Some counterfactuals suggest **changing** the **gender** from female to male, which shows a gender **bias** of the model

Counterfactual Explanations (Examples)

The German Credit Risk dataset, based on the credit dataset example in Dandl et al. (2020)

Trained an **SVM** to predict the probability that a person has a **good credit risk**.

Model predicted a good credit risk of 24.2 %

Table 15.1: Feature values of a particular customer

age	sex	job	housing	savings	amount	dur.	purpose
58	f	unskilled	free	little	6143	48	car

Counterfactual algorithm to find top 10 best counterfactuals to get the good risk to 50%

Table 15.2: The ten best counterfactuals found for the customer

age	sex	job	amount	dur.	o_2	o_3	o_4	$f(x')$
		skilled		-20	0.108	2	0.036	0.501
		skilled		-24	0.114	2	0.029	0.525
		skilled		-22	0.111	2	0.033	0.513
-6		skilled		-24	0.126	3	0.018	0.505
-3		skilled		-24	0.120	3	0.024	0.515
-1		skilled		-24	0.116	3	0.027	0.522
-3	m			-24	0.195	3	0.012	0.501
-6	m			-25	0.202	3	0.011	0.501
-30	m	skilled		-24	0.285	4	0.005	0.590
-4	m		-1254	-24	0.204	4	0.002	0.506

All counterfactuals have predicted probabilities **greater than 50%**

Do **not dominate** each other, i.e., no counterfactual wins on O value

All suggest a **reduction** of the duration from 48 months to a minimum of 23

Some counterfactuals suggest **changing** the **gender** from female to male, which shows a gender **bias** of the model

Counterfactual Explanations (Examples)

The German Credit Risk dataset, based on the credit dataset example in Dandl et al. (2020)

Trained an **SVM** to predict the probability that a person has a **good credit risk**.

Model predicted a good credit risk of 24.2 %

Table 15.1: Feature values of a particular customer

age	sex	job	housing	savings	amount	dur.	purpose
58	f	unskilled	free	little	6143	48	car

Counterfactual algorithm to find top 10 best counterfactuals to get the good risk to 50%

Table 15.2: The ten best counterfactuals found for the customer

age	sex	job	amount	dur.	o ₂	o ₃	o ₄	f(x')
		skilled		-20	0.108	2	0.036	0.501
		skilled		-24	0.114	2	0.029	0.525
		skilled		-22	0.111	2	0.033	0.513
-6		skilled		-24	0.126	3	0.018	0.505
-3		skilled		-24	0.120	3	0.024	0.515
-1		skilled		-24	0.116	3	0.027	0.522
-3	m			-24	0.195	3	0.012	0.501
-6	m			-25	0.202	3	0.011	0.501
-30	m	skilled		-24	0.285	4	0.005	0.590
-4	m		-1254	-24	0.204	4	0.002	0.506

All counterfactuals have predicted probabilities **greater than 50%**

Do **not dominate** each other, i.e., no counterfactual wins on O value

All suggest a **reduction** of the duration from 48 months to a minimum of 23

Some counterfactuals suggest **changing** the **gender** from female to male, which shows a gender **bias** of the model

Counterfactual Explanations (Examples)

The German Credit Risk dataset, based on the credit dataset example in Dandl et al. (2020)

Trained an **SVM** to predict the probability that a person has a **good credit risk**.

Model predicted a good credit risk of 24.2 %

Table 15.1: Feature values of a particular customer

age	sex	job	housing	savings	amount	dur.	purpose
58	f	unskilled	free	little	6143	48	car

Counterfactual algorithm to find top 10 best counterfactuals to get the good risk to 50%

Table 15.2: The ten best counterfactuals found for the customer

age	sex	job	amount	dur.	o ₂	o ₃	o ₄	f(x')
		skilled		-20	0.108	2	0.036	0.501
		skilled		-24	0.114	2	0.029	0.525
		skilled		-22	0.111	2	0.033	0.513
-6		skilled		-24	0.126	3	0.018	0.505
-3		skilled		-24	0.120	3	0.024	0.515
-1		skilled		-24	0.116	3	0.027	0.522
-3	m			-24	0.195	3	0.012	0.501
-6	m			-25	0.202	3	0.011	0.501
-30	m	skilled		-24	0.285	4	0.005	0.590
-4	m		-1254	-24	0.204	4	0.002	0.506

All counterfactuals have predicted probabilities **greater than 50%**

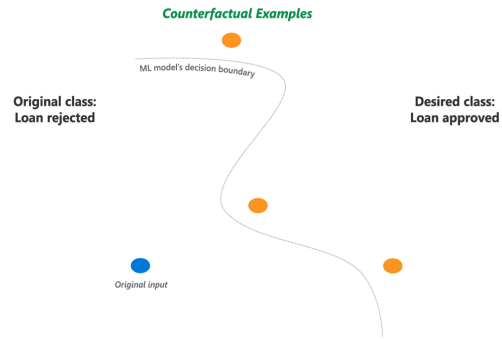
Do **not dominate** each other, i.e., no counterfactual wins on O value

All suggest a **reduction** of the duration from 48 months to a minimum of 23

Some counterfactuals suggest **changing** the **gender** from female to male, which shows a gender **bias** of the model

Counterfactual Explanations (Tutorial)

Library DiCE



Adult

Donated on 4/30/1996

Predict whether annual income of an individual exceeds \$50K/yr based on census data. Also known as "Census Income" dataset.

Dataset Characteristics

Multivariate

Subject Area

Social Science

Associated Tasks

Classification

Feature Type

Categorical, Integer

Instances

48842

Features

14

Dataset Information

Additional Information

Extraction was done by Barry Becker from the 1994 Census database. A set of reasonably clean records was extracted using the following conditions: ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HRSWK>0))

	age	workclass	education	marital_status	occupation	race	gender	hours_per_week	income
0	28	Private	Bachelors	Single	White-Collar	White	Female	60	0
1	30	Self-Employed	Assoc	Married	Professional	White	Male	65	1
2	32	Private	Some-college	Married	White-Collar	White	Male	50	0
3	20	Private	Some-college	Single	Service	White	Female	35	0
4	41	Self-Employed	Some-college	Married	White-Collar	White	Male	50	0

Code

Model-agnostic methods

- random
- genetic
- kdtree

Supports different search methods including gradient Backend sklearn

```
Dice(data, model, method="random").generate_counterfactuals(
    features_to_vary=['x1'],
    permitted_range={'x1':[0,10]})
```

Necessity and Sufficiency

Feasibility (and proximity)

Renku



Please locate the PDF called [xai_lab_instructions.pdf](#)
You will find instructions for how to run the notebooks for this course. The notebooks are hosted on [Renku Lab](#) and take several minutes to compile ([up to 15 minutes](#)).
If the environment fails to run then please follow the instructions for the local option.
Note that if you already cloned the repo from last session, then only pull (see last section).

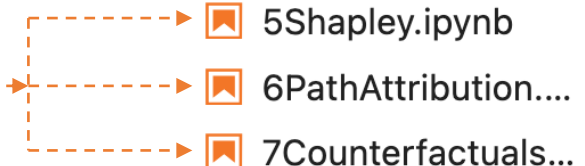
Renku documentation can be found here

<https://renku.notion.site/Documentation-db396cfc9a664cd2b161e4c6068a5ec9>

This week's notebooks

📁 / xai-course / notebooks /

Name	Modified
📁 static	6d ago
📄 1GlassBox.ipynb	6d ago
📄 2Occlusion.ipynb	6d ago
📄 3CAM.ipynb	6d ago
📄 4LIME.ipynb	6d ago
📄 5Shapley.ipynb	6d ago
📄 6PathAttribution....	6d ago
📄 7Counterfactuals...	6d ago
📄 8MutualInformati...	6d ago
📄 Challenge.ipynb	6d ago
🐍 utils.py	6d ago



CAM code walk through

```
# feature_maps captured by our hook: [1, C, H, W]
# fc_weight: [num_classes, C]
# class_idx: int, target class. If None, uses the model's top-1 prediction.

def compute_cam_for_class(fmaps: torch.Tensor, class_idx: int, fc_weight: np.ndarray):

    # 1. Get the dimensions of the feature maps
    C, H, W = fmaps.shape[1:]

    # 2. Get the weights for our target class and reshape them for broadcasting
    w = torch.from_numpy(fc_weight[class_idx]).view(C, 1, 1) # [C,1,1]

    # 3. Compute the weighted sum of feature maps
    # PyTorch will automatically "stretch" the 1x1 dimensions of the weights to match the HxW dimension
    # allowing for a clean element-wise multiplication.
    cam = torch.sum(fmaps[0] * w.to(fmaps.device), dim=0) # [H,W] , 0 because we take the first arg

    # 4. Apply ReLU to keep only positive contributions
    # We are interested in the features that have a positive influence on the classification.
    # We want to see the evidence that supports the class prediction, not what contradicts it.
    cam = F.relu(cam)

    # 5. Normalize the heatmap to be in the [0, 1] range for visualization
    cam = cam - cam.min()
    if cam.max() > 0:
        cam = cam / cam.max()

    return cam.cpu().numpy() # [H,W]
```

CAM Overlay

